

# Compact Fully Asynchronous Updatable Public Key Encryption Scheme from Hamming Quasi-Cyclic Cryptosystem

Sanajit Patra<sup>1</sup>, Ratna Dutta<sup>1</sup>, and Jayashree Dey<sup>2</sup>

<sup>1</sup> Indian Institute of Technology Kharagpur, India  
sanajitpatra@kgpian.iitkgp.ac.in, ratna@maths.iitkgp.ac.in

<sup>2</sup> Indian Institute of Technology Madras, India  
jayashreedey92@gmail.com

**Abstract.** In this work, we propose the *first* code-based uPKE from the *hamming quasi-cyclic public key encryption scheme* (hqcPKE) of Gaborit *et al.* by integrating an efficient key-update mechanism utilizing a carefully designed deterministic sampling algorithm. Our sampling algorithm exploits a structured permutation set preserving Hamming weight and satisfying specific algebraic properties that is of independent interest. More positively, our protocol allows unbounded key updates free from the cumulative error issues inherant to lattice based designs and supports asynchronous key updates, authorizing senders to update the public key independently. We formally establish the security of the proposed construction against *indistinguishability under chosen-randomness and chosen-plaintext attack* (IND-CR-CPA) in standard model considering *decisional quasi-cyclic syndrome decoding with parity* (DQCSDP) assumption. We emphasize that in comparison with existing post-quantum secure asynchronous schemes supporting unbounded updates, our construction in the standard model instead of random oracle model achieves significantly improved storage and communication efficiency, particularly in terms of public key size, ciphertext size and update ciphertext size. From a computational perspective, our design enables a more efficient public key update procedure and exhibits comparable performance for key generation, encryption, decryption and secret key update.

**Keywords:** Updatable public key encryption · Code based cryptography · Forward security.

## 1 Introduction

The problem of enabling secure communication over untrusted channels lies at the core of cryptographic research. The security of encrypted data such as emails, group messages and financial transactions relies on the confidentiality of the secret key. Once this key is compromised, the protection afforded to all past communications collapses, allowing an adversary to recover sensitive information they may have collected over months or even years. *Forward security* (FS)

mitigates the issue by ensuring that the disclosure of future keys does not compromise the secrecy of past messages. Achieving FS in symmetric-key cryptography is comparatively straightforward and well understood. In a symmetric-key setting, given an initial key  $s_0$ , one can update the key as  $(s_i, k_i) \leftarrow \text{PRG}(s_{i-1})$  using a pseudo-random generator PRG where  $s_i$  is used for further key derivation and  $k_i$  is the one-time symmetric key. Obtaining forward security is particularly challenging in *public-key encryption* (PKE) scenarios. In such schemes, time is typically partitioned into discrete intervals. The receiver updates the secret key for the next interval using the secret key of the the current interval, keeping the public key unchanged. The foundational work of Canetti *et al.* [8] introduced the concept of *forward-secure public key encryption* (FS-PKE). In a FS-PKE system, the keys evolve over time and create two synchronized chains – one a public key chain  $\text{pk}_0 \rightarrow \text{pk}_1 \rightarrow \text{pk}_2 \rightarrow \dots$  and a secret key chain  $\text{sk}_0 \rightarrow \text{sk}_1 \rightarrow \text{sk}_2 \rightarrow \dots$ , where  $\text{pk}_0$  and  $\text{sk}_0$  are respectively the initial public and secret keys. The key-update mechanism allows multiple senders to securely encrypt messages for a receiver who autonomously evolves their secret key across time periods. More precisely, the system is designed in such a way that a message encrypted with a public key  $\text{pk}_i$  can only be decrypted by its corresponding secret key  $\text{sk}_i$ . Most of the FS-PKE constructions employ *hierarchical identity-based encryption* (HIBE) techniques ([16,20] etc.). However, these designs continue to suffer from increased complexity and lower efficiency.

**Updatable public key encryption** (uPKE). To bridge this efficiency/complexity gap, the notion of uPKE was introduced by Jost *et al.* [21] and later advanced by Alwen *et al.* [3]. Beyond the functionality of standard public key encryption, uPKE schemes support asynchronous key updates in which both encryption and decryption keys are refreshed using newly generated entropy. This update mechanism enables the system to recover from key exposure by re-establishing security guarantees. A key distinction between uPKE and FS-PKE lies in who performs the update. In uPKE, updated keys may be generated at any time by any sender whereas it is restricted to the recipient in FS-PKE. More specifically, to update the public-secret key pair  $(\text{pk}_i, \text{sk}_i)$  of a recipient  $u$  at epoch  $i$ , a sender can generate and transmit a public update  $(\text{pk}_{i+1}, \text{up}_{i+1})$  where  $\text{pk}_{i+1}$  denotes the updated public key for next epoch  $i + 1$  formed using a randomness  $\mu_i$  and  $\text{up}_{i+1}$  refers to an update ciphertext that is derived by encrypting the randomness  $\mu_i$  under the public key  $\text{pk}_i$  corresponding to epoch  $i$ . Upon receiving this update, the recipient uses its secret key  $\text{sk}_i$  to decrypt  $\text{up}_{i+1}$  and recovers  $\mu_i$  and subsequently derives the updated secret key  $\text{sk}_{i+1}$  corresponding to the new public key  $\text{pk}_{i+1}$ .

Unlike standard PKE where a compromised secret key leads to a complete system reset, uPKE considers an environment where an adversary may gain access to secret key information at a epoch. The *indistinguishability under chosen-randomness and chosen-plaintext attack* (IND-CR-CPA) security allows an adversary to adaptively enforce updates on the target user’s public key while controlling the randomness used in the update algorithm. More interestingly, it ensures that if an adversary gets the information regarding the secret key

$\text{sk}_t$  at some epoch  $t$ , it could not decrypt any ciphertexts  $\text{CT}_i$  from previous epochs  $i < t$ . This guarantee remains valid provided that at least one key update in the interval between epochs  $i$  and  $t$  is carried out by an honest user, i.e., the update is generated using fresh private randomness that is unknown to the adversary. However, a more stronger security notion *indistinguishability under chosen-randomness and chosen-ciphertext attack* (IND-CR-CCA) requires that the adversary gets access to the decryption oracle corresponding to the secret key of the current epoch.

**Related Work.** Several uPKE constructions have been proposed in the past few years. Jost *et al.* [21] and Alwen *et al.* [3] proposed uPKE constructions following the framework of the hashed ElGamal public-key encryption scheme whose security relies on the *computational Diffie-Hellman* (CDH) assumption in the *random oracle model* (ROM). Later, Eaton *et al.* [13] introduced the first post-quantum secure uPKE design techniques depending on the *supersingular isogeny Diffie-Hellman* (SIDH) and *commutative supersingular isogeny Diffie-Hellman* (CSIDH) assumptions. Subsequently, Dodis *et al.* [11] eliminated the use of random oracles and presented two concrete uPKE constructions in the standard model based on the *decisional Diffie-Hellman* (DDH) and *learning with errors* (LWE) assumptions respectively. Haidar *et al.* [1] further proposed a scheme based on the *decisional composite residuosity* (DCR) assumption without random oracles. Later a more efficient LWE-based construction are presented by Haidar *et al.* [19] in the standard-model. They also proposed two generic transforms to achieve stronger security notion IND-CR-CCA refining the construction of Dodis *et al.* Later, Duparc *et al.* [12] proposed *supersingular isogeny lollipop based encryption* (SILBE) that does not rely on isogeny group actions which provides *one-way chosen-randomness chosen plaintext attack* (OW-CR-CPA) security, a weaker security framework. In 2024, Alwen *et al.* [4] proposed an efficient IND-CR-CCA secure uPKE scheme over bilinear groups in the ROM and the *algebraic group model*. In 2025, Asano and Watanabe [5] demonstrated a simple and efficient attack on Dodis *et al.*'s IND-CR-CCA secure scheme [11], suggested how to fix that and proposed a new IND-CR-CCA secure generic construction for a uPKE scheme in ROM. More recently, Albrecht *et al.* [2] achieved unbounded updatable encryption based on LWE and *permutation code equivalence* (PCE); however, their construction requires large parameters, stemming from the use of the leftover hash lemma and intricate security reductions.

## 1.1 Our Contribution

There is a growing demand for cryptographic schemes whose security is based on problems believed to be resistant to quantum attacks due to Shor [27]. Recent advances in quantum computing [28] have further emphasized the necessity of developing cryptographic primitives that remain secure in the post-quantum era. Lattice-based uPKE schemes ([11,19]) inherently suffer from noise accumulation across successive key updates. This limits the number of supported updates or necessitates expensive noise-management and compression techniques, thereby

reducing efficiency and scalability in long-term communication scenarios. On the other hand, isogeny-based constructions ([13,12]) either require coordination or interaction between senders and receivers during key updates. This prevents fully asynchronous operation or only supports a bounded number of updates. Moreover, some isogeny-based proposals rely on assumptions that have been weakened or broken, such as those underlying SIDH-based constructions. These limitations significantly hinder their suitability for practical, long-lived secure messaging applications.

Code-based cryptography stands out as an interesting option in post-quantum paradigms whose foundations rely primarily on algebraic coding theory over finite fields and the security is rooted in the well-studied hardness of decoding random linear codes. More interestingly, code-based protocols are well-suited for deployment across a wide range of platforms. Supporting efficient and scalable cryptographic functionality over extended periods is a fundamental requirement for secure messaging systems that rely on frequent public-key updates. Although updatable public-key encryption has witnessed notable progress, existing constructions often remain inefficient or impractical for real-world deployment, particularly in the post-quantum context. So far, no prior work considers uPKE in code-based environment, which motivates the exploration of new constructions that enable more efficient and practical post-quantum cryptographic solutions.

In this work, we adopt a coding theory approach and propose the *first* post-quantum secure uPKE construction for long-term secure messaging in code-based setting. Our starting point is the *hamming quasi-cyclic public key cryptosystem* (hqcPKE) of Gaborit *et al.* [15] that couples an efficiently decodable linear code with a random double-circulant code and derives security from the hardness of the *decisional quasi-cyclic syndrome decoding* (DQCSD) problem. To be more precise a parity version of DQCSD problem namely the *decisional quasi-cyclic syndrome decoding with parity* (DQCSDP) problem is assumed to be hard for their security. We removed the truncation mechanism presented in [15] and uses only the parity version assumption of DQCSDP problem. A deterministic sampling algorithm  $\text{Samp} : \mathcal{MS} \rightarrow \text{Per}(\mathcal{R})$  is used a structured permutation set  $\text{Per}(\mathcal{R})$  with few specific algebraic properties for our update mechanism where  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$  and  $\mathcal{MS} = \mathbb{F}_2^k$  is the message space. We emphasize that our scheme enables fully asynchronous public-key updates without requiring sender-receiver coordination in contrast to the isogeny-based designs ([13,12]) and does not have the cumulative error issues inherent in lattice-based designs ([11,19]). In addition, our candidate supports an unbounded number of key updates and achieves IND-CR-CPA security under the DQCSDP assumption. We outline our main findings below.

- Based on our structured permutation set  $\text{Per}(\mathcal{R})$  where each element of  $\text{Per}(\mathcal{R})$  satisfies specific algebraic properties stated below, allows the public keys and secret keys to be updated in our uhqcPKE via a transformation while preserving correctness and security. Messages are mapped to elements of  $\text{Per}(\mathcal{R})$  by a deterministic sampling algorithm  $\text{Samp}$ .

**Assumption (Informal):** Let  $(\Pi, F) \in \text{Per}(\mathcal{R})$ . We assume that an element of  $\text{Per}(\mathcal{R})$  has the following specific properties:

- (i)  $F$  and  $\Pi$  preserve Hamming weight.
- (ii)  $F(\mathbf{x} + \mathbf{h}\mathbf{y}) = F(\mathbf{x}) + \Pi(\mathbf{h})F(\mathbf{y})$  for all  $\mathbf{h}, \mathbf{x}, \mathbf{y} \in \mathcal{R}$ .
- (iii) The set  $\text{Per}(\mathcal{R})$  have sufficient entropy such that the output of **Samp** mimics random sampling.

Table 1: Storage and communication cost of asynchronous post-quantum IND-CR-CPA secure uPKE schemes.

| Scheme  | $ \text{sk} $ | $ \text{pk} $                            | $ \text{ct} $      | $ \text{up} $                           | Assumption         | UB        |
|---------|---------------|------------------------------------------|--------------------|-----------------------------------------|--------------------|-----------|
| [11]    | $m$           | $(nm + n) \log q$                        | $(m + 1) \log q$   | $m(m + 1) \log q$                       | LWE                | Bounded   |
| [19]    | $n \log q$    | $(n^2 + n) \log q$                       | $(n^2 + n) \log q$ | $(n^2 + n) \log q$                      | LWE                | Bounded   |
| [2]     | $n$           | $(nk + k) \log q + \mathcal{O}(\lambda)$ | $(n + 1) \log q$   | $\frac{\lambda}{\log p} (n + 1) \log q$ | Hollow LWE and PCE | Unbounded |
| uhqcPKE | $2\hat{n}$    | $2\hat{n}$                               | $2\hat{n}$         | $2\hat{n}$                              | DQCSDP             | Unbounded |

$\lambda$  is the security parameter,  $q$  the modulus in lattice-based schemes,  $p$  message modulus,  $n$  lattice dimension,  $\hat{n}$  code length,  $k$  dimension of hollow code,  $m(> n)$  is the number of columns of the LWE matrix, UB: update bound,  $|\text{sk}|$ : secret key size,  $|\text{pk}|$ : public key size,  $|\text{ct}|$ : ciphertext size,  $|\text{up}|$ : update ciphertext size, LWE : learning with errors, PCE: permutation code equivalence, DQCSDP: decisional quasi-cyclic syndrome decoding with parity

- Table 1 provides a comparative analysis of our uhqcPKE with existing asynchronous post-quantum secure uPKE schemes in terms of storage and communication overhead. The isogeny-based uPKE constructions ([13,12]) do not support the asynchronous property unlike our uhqcPKE. Moreover, the uPKE scheme of Duparc *et al.* [12] achieves only OW-CR-CPA security, a comparatively weaker security notion as compared to IND-CR-CPA security of our uhqcPKE. The SIDH based construction of Eaton *et al.* [13] is subsequently broken by attacks ([9,23,26]). In contrast to the lattice-based constructions ([11,19,2]), our candidate relies on DQCSDP assumption based on linear codes. As summarized in Table 1, the existing lattice-based uPKE protocols are more expensive in terms of public key size, ciphertext size as well as updated ciphertext size as compared to our uhqcPKE. When  $\hat{n} = n$  for our uhqcPKE the secret key size  $|\text{sk}| = 2n$  is comparable to others but in terms of  $|\text{pk}|, |\text{ct}|, |\text{up}|$  are linear while the others are more expensive.

The overall computation cost of the uhqcPKE is comparable as compared to the lattice-based uPKE as described in Table 2 in terms of **KeyGen**, **Enc**, **Dec**, **SkUpdate** cost while uhqcPKE performs better in terms of **PkUpdate** cost compared to other lattice-based schemes. In the Table 2, we have not included the decoding cost of uhqcPKE as it varies over codes. More favorably, our scheme uhqcPKE affords unbounded key updates whereas the works in [11,19] primarily focus on only bounded number of updates due to noise management issues after certain number of updates. In contrast,

Table 2: Computation cost of existing asynchronous post-quantum IND-CR-CPA secure uPKE schemes.

| Scheme      | KeyGen                   | Enc                      | Dec                      | PkUpdate                                      | SkUpdate                 |
|-------------|--------------------------|--------------------------|--------------------------|-----------------------------------------------|--------------------------|
| [11]        | $\mathcal{O}(nm)$        | $\mathcal{O}(nm)$        | $\mathcal{O}(m)$         | $\mathcal{O}(nm^2)$                           | $\mathcal{O}(m^2)$       |
| [19]        | $\mathcal{O}(n^2)$       | $\mathcal{O}(n^3)$       | $\mathcal{O}(n^2)$       | $\mathcal{O}(n^3)$                            | $\mathcal{O}(n^2)$       |
| [2]         | $\mathcal{O}(nk)$        | $\mathcal{O}(nk)$        | $\mathcal{O}(n)$         | $\mathcal{O}(n^3 + \frac{\lambda}{\log p}nk)$ | $\mathcal{O}(n^2)$       |
| Our uhqcPKE | $\mathcal{O}(\hat{n}^2)$ | $\mathcal{O}(\hat{n}^2)$ | $\mathcal{O}(\hat{n}^2)$ | $\mathcal{O}(\hat{n}^2)$                      | $\mathcal{O}(\hat{n}^2)$ |

$\lambda$  is the security parameter,  $p$  the message modulus,  $n$  the lattice dimension,  $m(> n)$  the number of columns of the LWE matrix,  $k$  the hollow code dimension,  $\hat{n}$  code length. All costs are reported in Big- $\mathcal{O}$  notation.

our candidate employs a structured weight preserving function which allows to update the public key and secret key without any noise accumulation. Albrecht *et al.* [2] addresses the noise-management issues by incorporating a permutation matrix and an invertible matrix into the update mechanism. However the concrete performance of their uPKE remains low for realistic choices of parameters.

- We provide a rigorous security analysis and formally establish the IND-CR-CPA security of our uhqcPKE under the 2-DQCSDP and the 3-DQCSDP assumptions. The IND-CR-CPA security model between a challenger and an adversary captures the forward security which ensures that compromise of secret keys at the current epoch does not endanger the confidentiality of past ciphertexts. The security experiment shows that the scheme will remain secure even if the keys are periodically updated and at some point of time the secret key is exposed. More formally, we prove the following Theorem.

**Theorem (Informal)** *Our proposed uhqcPKE is IND-CR-CPA under the hardness of the 2-DQCSDP and the 3-DQCSDP assumptions.*

## 1.2 Technical Overview

We construct an IND-CR-CPA secure updatable encryption scheme uhqcPKE based on IND-CPA secure hqcPKE of [15]. Our construction follows the general uPKE framework where key updates can be performed asynchronously by any sender. To enable the update functionality, we use a structured permutation set  $\text{Per}(\mathcal{R})$  where  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$ . Each element  $(\Pi, F)$  of  $\text{Per}(\mathcal{R})$  satisfies few specific properties as stated in Section 1.1.

We use a deterministic sampling function  $\text{Samp} : \mathbb{F}_2^k \rightarrow \text{Per}(\mathcal{R})$  to sample an element  $(\Pi, F)$  of  $\text{Per}(\mathcal{R})$  using a vector  $\mu_i$  from the message space  $\mathbb{F}_2^k$ , use it to update the public key at epoch  $i$  and encrypt  $\mu_i$  to form an update ciphertext  $\text{up}_{i+1}$ . To update a secret key at epoch  $i$  the update ciphertext  $\text{up}_{i+1}$  is de-

crypted using the secret key  $\mathbf{sk}_i$  at epoch  $i$  to recover the randomness  $\mu_i$  and use  $\text{Samp}(\mu_i) \in \text{Per}(\mathcal{R})$  to update the secret key at epoch  $i+1$ . More explicitly, a user samples its initial secret key  $\mathbf{sk}_0 = (\mathbf{x}_0, \mathbf{y}_0)$  by selecting  $\mathbf{x}_0, \mathbf{y}_0 \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2)$  and computes the corresponding initial public key  $\mathbf{pk}_0 = (\mathbf{h}_0, \mathbf{s}_0 = \mathbf{x}_0 + \mathbf{h}_0 \mathbf{y}_0) \in \mathcal{R} \times \mathcal{R}$  at epoch 0 where  $\mathcal{S}_w^n(\mathbb{F}_2) = \{\mathbf{v} \in \mathbb{F}_2^n \mid \text{wt}(\mathbf{v}) = w\}$  and  $\mathbf{h}_0 \xleftarrow{\$} \mathcal{R}$  is a uniformly random public polynomial. To encrypt a message  $\mathbf{m} \in \mathbb{F}^k$  under the public key  $\mathbf{pk}_i = (\mathbf{h}_i, \mathbf{s}_i)$  at epoch  $i$ , the sender computes the uhqcPKE ciphertext  $\text{CT}_{\mathbf{m}} = (\mathbf{u}_{\mathbf{m}} = \mathbf{r}_1 + \mathbf{h}_i \mathbf{r}_2, \mathbf{v}_{\mathbf{m}} = \mathbf{m} \mathbf{G} + \mathbf{s}_i \mathbf{r}_2 + \mathbf{e})$  for some random  $(\mathbf{r}_1, \mathbf{r}_2) \in \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\mathbf{e} \in \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  where  $\mathbf{G}$  denotes the public generator matrix of the  $[n, k]$  linear code,  $w, w_r, w_e$  are fixed weights. To update the public key  $\mathbf{pk}_i$  at epoch  $i$ , a sender samples a randomness  $\mu_i \xleftarrow{\$} \mathbb{F}_2^k$  and encrypts it under  $\mathbf{pk}_i = (\mathbf{h}_i, \mathbf{s}_i)$  to generate the update ciphertext  $\text{up}_{i+1} = \text{CT}_{\mu_i} = (\mathbf{u}_{\mu_i} = \mathbf{r}'_1 + \mathbf{h}_i \mathbf{r}'_2, \mathbf{v}_{\mu_i} = \mu_i \mathbf{G} + \mathbf{s}_i \mathbf{r}'_2 + \mathbf{e}')$  for randomly chosen  $(\mathbf{r}'_1, \mathbf{r}'_2) \in \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\mathbf{e}' \in \mathcal{S}_{w_e}^n(\mathbb{F}_2)$ . The updated public key is then computed as  $\mathbf{pk}_{i+1} = (\mathbf{h}_{i+1}, \mathbf{s}_{i+1})$  by setting  $\mathbf{h}_{i+1} = \Pi_{\mu_i}(\mathbf{h}_i)$ ,  $\mathbf{s}_{i+1} = \mathbf{F}_{\mu_i}(\mathbf{s}_i)$  and generating  $\text{Samp}(\mu_i) = (\Pi_{\mu_i}, \mathbf{F}_{\mu_i}) \in \text{Per}(\mathcal{R})$ . Upon receiving  $(\mathbf{pk}_{i+1}, \text{up}_{i+1})$ , the receiver decrypting  $\text{up}_{i+1}$ , recovers the update randomness  $\mu_i$ , generates  $\text{Samp}(\mu_i) = (\Pi_{\mu_i}, \mathbf{F}_{\mu_i}) \in \text{Per}(\mathcal{R})$  and updates the old secret key  $\mathbf{sk}_i = (\mathbf{x}_i, \mathbf{y}_i)$  by new secret key  $\mathbf{sk}_{i+1} = (\mathbf{x}_{i+1}, \mathbf{y}_{i+1})$  by setting  $\mathbf{x}_{i+1} = \mathbf{F}_{\mu_i}(\mathbf{x}_i)$ ,  $\mathbf{y}_{i+1} = \mathbf{F}_{\mu_i}(\mathbf{y}_i)$ . The weight preserving property of  $\Pi, \mathbf{F}$  and the property  $\mathbf{F}(\mathbf{x} + \mathbf{h}\mathbf{y}) = \mathbf{F}(\mathbf{x}) + \Pi(\mathbf{h})\mathbf{F}(\mathbf{y})$  for all  $\mathbf{x}, \mathbf{y}, \mathbf{h} \in \mathcal{R}$  keeps the distribution of the public-secret key pair unaltered.

We prove that our uhqcPKE is IND-CR-CPA secure via a game-based reduction under the hardness assumptions of the 2-DQCSDP and the 3-DQCSDP. The proof proceeds through a sequence of hybrid games in which the adversary's view is modified in a gradual manner. For every pair of consecutive games, we show that no efficient adversary can distinguish them. We argue that any adversary capable of distinguishing two adjacent games can be transformed into a distinguisher that breaks either the 2-DQCSDP assumption or the 3-DQCSDP assumption. In particular, if we consider a 2-DQCSDP instance  $(\mathbf{H}, \mathbf{s})$  where  $\mathbf{H} = (\mathbf{I}_n \text{ rot}(\mathbf{h}))$  is an  $n \times 2n$  matrix and  $\mathbf{s}$  is a vector of length  $n$ . From this instance, we derive the public key  $\mathbf{pk}_0 = (\mathbf{h}, \mathbf{s}) = (\mathbf{h}_0, \mathbf{s}_0)$  where  $\mathbf{h}, \mathbf{s} \in \mathcal{R}$  and simulate an adversary that attempts to distinguish two consecutive games. If such an adversary succeeds, then we obtain a distinguisher that breaks the 2-DQCSDP assumption. Consequently, the hardness of 2-DQCSDP implies that the corresponding games are computationally indistinguishable. Similarly, we consider a 3-DQCSDP instance  $(\mathbf{H}, (\mathbf{a}_1, \mathbf{a}_2))$  where  $\mathbf{H}$  is a  $2n \times 3n$  matrix of the form  $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$  and  $(\mathbf{a}_1, \mathbf{a}_2)$  is a vector of length  $2n$  with  $\mathbf{a}_1 = \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2 \in \mathcal{R}$  and  $\mathbf{a}_2 = \mathbf{s}\mathbf{r}_2 + \mathbf{e} \in \mathcal{R}$  for some unknown  $\mathbf{r}_1, \mathbf{r}_2 \in \mathcal{R}$  and  $\mathbf{e} \in \mathcal{R}$  satisfying a fixed parity condition. For a message  $\mathbf{m}$ , we define the ciphertext as  $\text{CT} = (\mathbf{a}_1, \mathbf{m}\mathbf{G} + \mathbf{a}_2) = (\mathbf{r}_1 + \mathbf{h}\mathbf{r}_2, \mathbf{m}\mathbf{G} + \mathbf{s}\mathbf{r}_2 + \mathbf{e})$ . The resulting distribution of CT is identical to that of an honestly generated ciphertext. We exploit this property to simulate the challenge ciphertext in the security experiment. We show that if there exists an adversary capable of distinguishing two

adjacent games in this hybrid sequence, then one can construct a distinguisher that breaks the 3-DQCSDP assumption. Hence, the hardness of 3-DQCSDP guarantees that the corresponding games are computationally indistinguishable. In the final game, the challenge ciphertext is statistically independent of the challenge bit  $b$  and therefore combining all the arguments, we conclude that our uhcPKE is IND-CR-CPA secure.

## 2 Preliminaries

**Notations.** Let  $\mathbb{Z}$  be the set of integers,  $\mathbb{F}$  be finite field and  $\mathbb{F}_2$  be the finite binary field. The Hamming weight of a vector, denoted by  $\text{wt}(\cdot)$ , refers to the number of non-zero entries in that vector. We define the set of all binary vectors of length  $n$  with Hamming weight exactly  $w$  as  $\mathcal{S}_w^n(\mathbb{F}_2) = \{\mathbf{v} \in \mathbb{F}_2^n \mid \text{wt}(\mathbf{v}) = w\}$ .

The notation  $x \xleftarrow{\$} \mathcal{S}$  indicates that  $x$  is sampled uniformly at random from the set  $\mathcal{S}$ . The transpose of a vector  $\mathbf{u}$  is denoted by  $\mathbf{u}^T$ . Also for any two functions  $f, g$ , we denote the composition by  $f \circ g = f(g)$ . Let  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$  denote the quotient ring of polynomials where  $n \in \mathbb{Z}^+$ . We use bold lowercase or uppercase letters to denote vectors, polynomials, or matrices. A prime  $n$  is primitive if  $(x^n - 1)/(x - 1)$  is irreducible in  $\mathcal{R}$ . An element  $\mathbf{u} \in \mathcal{R}$  is often denoted as the vector  $\mathbf{u} = (u_0, u_1, \dots, u_{n-1}) \in \mathbb{F}_2^n$  where  $u_i$  denotes the coefficient of  $x^i$  for  $i = 0, 1, \dots, n-1$ . For any  $\mathbf{u}, \mathbf{v} \in \mathcal{R}$ , where  $\mathbf{u} = (u_0, u_1, \dots, u_{n-1}) \in \mathbb{F}_2^n$  and  $\mathbf{v} = (v_0, v_1, \dots, v_{n-1}) \in \mathbb{F}_2^n$  we define their product  $\mathbf{u}\mathbf{v} = \mathbf{w} \in \mathcal{R}$ , where  $\mathbf{w} = (w_0, w_1, \dots, w_n)$  is given component-wise as  $w_k = \sum_{i+j \equiv k \pmod{n}} u_i v_j$  for  $k \in \{0, 1, \dots, n-1\}$ .

### 2.1 Basics Related to Coding Theory

**Definition 1 (Circulant Matrix [24]).** Let  $\mathbf{v} = (v_0, v_1, \dots, v_{n-1}) \in \mathbb{F}_2^n$ . The circulant matrix generated by  $\mathbf{v}$ , denoted  $\text{rot}(\mathbf{v})$ , is defined as

$$\text{rot}(\mathbf{v}) = \begin{pmatrix} v_0 & v_{n-1} & \cdots & v_1 \\ v_1 & v_0 & \cdots & v_2 \\ \cdots & \cdots & \cdots & \cdots \\ v_{n-1} & v_{n-2} & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}.$$

An important consequence of this construction is that the product of any two vectors  $\mathbf{u}, \mathbf{v} \in \mathcal{R}$  can be represented as a standard matrix-vector product involving the circulant form. That is,  $\mathbf{u}\mathbf{v} = \mathbf{u} \times \text{rot}(\mathbf{v})^T = (\text{rot}(\mathbf{u}) \times \mathbf{v}^T)^T = \mathbf{v} \times \text{rot}(\mathbf{u})^T = \mathbf{v}\mathbf{u}$ . Here, ‘ $\times$ ’ denotes the usual matrix multiplication. This equivalence reflects the commutativity of polynomial multiplication in  $\mathcal{R}$  and allows efficient implementation via matrix operations.

**Definition 2 (Linear Code [22]).** A linear  $[n, k]$  code  $\mathcal{C}$  of length  $n$  and dimension  $k$  is a subspace of the vector space  $\mathbb{F}^n$  of dimension  $k$  over the field  $\mathbb{F}$ . The elements of  $\mathcal{C}$  are referred to as codewords. An  $[n, k]$  linear code is said to be a binary linear code if  $\mathbb{F} = \mathbb{F}_2$ .

**Definition 3 (Generator Matrix [22]).** A matrix  $\mathbf{G} \in \mathbb{F}_2^{k \times n}$  is said to be a generator matrix for the  $[n, k]$  binary linear code  $\mathcal{C}$  if  $\mathcal{C} = \{\mathbf{m}\mathbf{G} \mid \mathbf{m} \in \mathbb{F}_2^k\}$ .

**Definition 4 (Parity-Check Matrix [22]).** Let  $\mathcal{C}$  be an  $[n, k]$  linear code over  $\mathbb{F}_2$ . A matrix  $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$  is said to be a parity-check matrix of  $\mathcal{C}$  if  $\mathcal{C}$  is the kernel of  $\mathbf{H}$  i.e.,  $\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \mid \mathbf{H}\mathbf{v}^T = \mathbf{0}\}$ . Equivalently,  $\mathbf{H}$  is a generator matrix for the dual code  $\mathcal{C}^\perp$  where  $\mathcal{C}^\perp = \{\mathbf{u}\mathbf{H} \mid \mathbf{u} \in \mathbb{F}_2^{n-k}\}$ .

**Definition 5 (Syndrome [22]).** Let  $\mathcal{C}$  be an  $[n, k]$  linear code over  $\mathbb{F}_2$  and  $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$  be a parity-check matrix of  $\mathcal{C}$ . For any vector  $\mathbf{v} \in \mathbb{F}_2^n$ , the syndrome of  $\mathbf{v}$  is defined as  $\mathbf{s} = \mathbf{H}\mathbf{v}^T$ . A vector  $\mathbf{v}$  is a codeword in  $\mathcal{C}$  if and only if its syndrome is zero, i.e.,  $\mathbf{v} \in \mathcal{C} \iff \mathbf{H}\mathbf{v}^T = \mathbf{0}$ .

**Definition 6 (Minimum Distance [22]).** Let  $\mathcal{C}$  be an  $[n, k]$  linear code over  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$  and  $\text{wt}(\cdot)$  denote a weight function (typically the Hamming weight) on  $\mathcal{R}$ . The minimum distance of  $\mathcal{C}$  is defined as

$$d = \min_{\substack{\mathbf{u}, \mathbf{v} \in \mathcal{C} \\ \mathbf{u} \neq \mathbf{v}}} \text{wt}(\mathbf{u} - \mathbf{v}).$$

A linear code of length  $n$ , dimension  $k$  and minimum distance  $d$  is denoted by  $[n, k, d]$ . Such a code can uniquely correct any error pattern affecting up to  $\tau = \lfloor \frac{d-1}{2} \rfloor$  positions.

**Definition 7 (Quasi-Cyclic Codes [25]).** Let  $\mathbf{c} = (\mathbf{c}_0, \mathbf{c}_1, \dots, \mathbf{c}_{s-1}) \in \mathbb{F}_2^{sn}$ , where each block  $\mathbf{c}_i \in \mathbb{F}_2^n$  for  $0 \leq i < s$ . A linear code  $\mathcal{C} \subseteq \mathbb{F}_2^{sn}$  of length  $sn$ , dimension  $k$  and minimum distance  $d$  denoted  $[sn, k, d]$  is said to be a quasi-cyclic (QC) code of index  $s$  if for every codeword  $\mathbf{c} \in \mathcal{C}$  the vector obtained by applying a simultaneous cyclic shift to each block  $\mathbf{c}_0, \mathbf{c}_1, \dots, \mathbf{c}_{s-1}$  also belongs to  $\mathcal{C}$ .

Alternatively, viewing each block  $\mathbf{c}_i$  as a polynomial in the ring  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$ , the code  $\mathcal{C}$  is quasi-cyclic of index  $s$  if for any codeword  $\mathbf{c} = (\mathbf{c}_0, \dots, \mathbf{c}_{s-1}) \in \mathcal{C}$ , the tuple  $(x\mathbf{c}_0, \dots, x\mathbf{c}_{s-1})$  also lies in  $\mathcal{C}$ .

**Definition 8 (Systematic Quasi-Cyclic Codes [18]).** A systematic quasi-cyclic  $[sn, n]$  code of index  $s$  and rate  $\frac{1}{s}$  is a quasi-cyclic code with an  $(s-1)n \times sn$  parity-check matrix of the form:

$$\begin{bmatrix} \mathbf{I}_n & 0 & \cdots & 0 & \mathbf{A}_0 \\ 0 & \mathbf{I}_n & \cdots & 0 & \mathbf{A}_1 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & \cdots & \mathbf{I}_n & \mathbf{A}_{s-2} \end{bmatrix}$$

where,  $\mathbf{A}_i$  for  $i \in \{0, 1, \dots, s-2\}$  are circulant matrix.

**Remark 1** The definition of systematic quasi-cyclic codes of index  $s$  can be generalized to all rates  $\ell/s$  for  $\ell = 1, 2, \dots, s-1$ .

## 2.2 Hardness Assumptions

This section establishes the computational foundations for the security of the cryptographic protocols.

**Definition 9 (Syndrome Decoding (SD) Distribution [7]).** *The  $\text{SD}(n, k, w)$  distribution parameterized by positive integers  $n, k$  and  $w$  is defined as the distribution over pairs  $(\mathbf{H}, \mathbf{y})$  where  $\mathbf{H} \leftarrow \mathbb{F}_2^{(n-k) \times n}$  is sampled randomly and the syndrome  $\mathbf{y}$  satisfies  $\mathbf{y}^T = \mathbf{H}\mathbf{x}^T$  which is computed by sampling a random vector  $\mathbf{x} \xleftarrow{\$} \mathbb{F}_2^n$  such that  $\text{wt}(\mathbf{x}) = w$ .*

**Definition 10 (Computational SD(CSD) Problem [14]).** *Let  $n, k, w$  be positive integers. The  $\text{CSD}(n, k, w)$  problem parametrized by  $n, k, w$  states that given a pair  $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$  sampled according to the distribution  $\text{SD}(n, k, w)$  defined in definition 9, find a vector  $\mathbf{x} \in \mathbb{F}_2^n$  such that  $\mathbf{H}\mathbf{x}^T = \mathbf{y}^T$  and  $\text{wt}(\mathbf{x}) = w$ .*

The computational version of the SD problem is known to be NP-complete in the Hamming metric, as shown by Berlekamp, McEliece and van Tilborg [6].

**Definition 11 (Decisional SD (DSD) Problem [14]).** *The decisional syndrome decoding  $\text{DSD}(n, k, w)$  problem parametrized by  $n, k, w$  is defined as the problem of deciding, whether a pair  $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$  is sampled from the distribution  $\text{SD}(n, k, w)$  or from the uniform distribution over  $\mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$  with non-negligible advantage.*

**Proposition 1.** *Let  $\mathbf{f}(x) = \sum_{i=0}^{n-1} f_i x^i \in \mathcal{R}$  be a polynomial where  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$ . We define its parity as  $\text{par}(\mathbf{f}) = \mathbf{f} \cdot \mathbf{1} = \sum_{i=0}^{n-1} f_i \bmod 2$ . Then, for all  $\mathbf{h} = \mathbf{h}(x), \mathbf{y} = \mathbf{y}(x) \in \mathcal{R}$ , we have  $\text{par}(\mathbf{h}\mathbf{y}) = \text{par}(\mathbf{h}) \cdot \text{par}(\mathbf{y}) \bmod 2$ . Moreover, if  $\mathbf{x}, \mathbf{y}, \mathbf{h} \in \mathcal{R}$  satisfy  $\text{wt}(\mathbf{x}) = \text{wt}(\mathbf{y}) = w$  and  $\text{par}(\mathbf{h}) = b_1$  and if  $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y}$ , then  $\text{par}(\mathbf{s}) = w + b_1 \times w \bmod 2$ .*

*Proof.* Let  $\mathbf{h}(x) = \sum_{i=0}^{n-1} h_i x^i \in \mathcal{R}$  and  $\mathbf{y}(x) = \sum_{j=0}^{n-1} y_j x^j \in \mathcal{R}$  where  $h_i, y_j \in \mathbb{F}_2$  and  $\mathbf{h}\mathbf{y} = \sum_{k=0}^{n-1} c_k x^k \in \mathcal{R}$  with

$$c_k = \sum_{i+j \equiv k \pmod{n}} h_i y_j \in \mathbb{F}_2.$$

Now,

$$\text{par}(\mathbf{h}\mathbf{y}) = \sum_{k=0}^{n-1} c_k = \sum_{k=0}^{n-1} \sum_{i+j \equiv k} h_i y_j \bmod 2.$$

Since each ordered pair  $(i, j) \in \{0, \dots, n-1\}^2$  appears exactly once in the above sum, this simplifies to

$$\text{par}(\mathbf{h}\mathbf{y}) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} h_i y_j \bmod 2$$

which factors as

$$\left(\sum_{i=0}^{n-1} h_i\right) \left(\sum_{j=0}^{n-1} y_j\right) \bmod 2.$$

Hence  $\text{par}(\mathbf{hy}) = \text{par}(\mathbf{h}) \cdot \text{par}(\mathbf{y}) \bmod 2$ .

For the second claim, parity is linear over  $\mathbb{F}_2$  as  $\text{par}(\mathbf{x} + \mathbf{y}) = (\mathbf{x} + \mathbf{y}) \cdot \mathbf{1} = \mathbf{x} \cdot \mathbf{1} + \mathbf{y} \cdot \mathbf{1} = \text{par}(\mathbf{x}) + \text{par}(\mathbf{y})$  and thus  $\text{par}(\mathbf{s}) = \text{par}(\mathbf{x}) + \text{par}(\mathbf{hy}) \bmod 2$ . Since  $\text{par}(\mathbf{x}) = \text{wt}(\mathbf{x}) \equiv w \bmod 2$  and  $\text{par}(\mathbf{hy}) = \text{par}(\mathbf{h}) \cdot \text{par}(\mathbf{y}) = b_1 w \bmod 2$ , we conclude  $\text{par}(\mathbf{s}) = w + b_1 w = w(1 + b_1) \bmod 2$ .

We use the following notations

$$\begin{aligned} - \mathbb{F}_{2,b_1}^n &= \{\mathbf{h} \in \mathbb{F}_2^n \mid \mathbf{h} \cdot \mathbf{1} = b_1 \bmod 2\} \mid b_1 \in \{0, 1\}. \\ - \mathbb{F}_{2,b_1}^{n \times 2n} &= \left\{ \mathbf{H} = [\mathbf{I}_n \mid \text{rot}(\mathbf{h})] \in \mathbb{F}_2^{n \times 2n} \mid \mathbf{h} \in \mathbb{F}_{2,b_1}^n \right\} \text{ for } b_1 \in \{0, 1\}. \\ - \mathbb{F}_{2,b_1,b_2}^{2n \times 3n} &= \left\{ \mathbf{H} = \begin{bmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}_1) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{h}_2) \end{bmatrix} \in \mathbb{F}_2^{2n \times 3n} \mid \mathbf{h}_1 \in \mathbb{F}_{2,b_1}^n, \mathbf{h}_2 \in \mathbb{F}_{2,b_2}^n \right\} \text{ for } b_1, b_2 \in \{0, 1\}. \end{aligned}$$

The first denotes a set of binary vectors of length  $n$  whose entries sum to a parity  $b_1 \in \{0, 1\}$ . Here,  $\mathbf{1}$  denotes the vector whose all entries are 1 and of length  $n$  and  $\text{rot}(\mathbf{h})$ ,  $\text{rot}(\mathbf{h}_1)$ ,  $\text{rot}(\mathbf{h}_2)$  denote the circulant matrices generated by the vector  $\mathbf{h}$ ,  $\mathbf{h}_1$ ,  $\mathbf{h}_2$  respectively.

**Definition 12 (2-Quasi-Cyclic Syndrome Decoding with Parity (2-QCSDP) Distribution [24]).** Let  $n, w$  be positive integers and let  $b_1 \in \{0, 1\}$  be a parity bit. Define  $b_2 = (w + b_1 w) \bmod 2$ . The 2-QCSDP( $n, w, b_1, b_2$ ) problem is defined as the distribution over pairs  $(\mathbf{H}, \mathbf{y})$  where,  $\mathbf{H} \leftarrow \mathbb{F}_{2,b_1}^{n \times 2n}$  is sampled randomly and the syndrome  $\mathbf{y}$  satisfies  $\mathbf{y}^T = \mathbf{H}\mathbf{x}^T$  which is computed by sampling  $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2) \in \mathbb{F}_2^{2n}$  such that  $\text{wt}(\mathbf{x}_1) = \text{wt}(\mathbf{x}_2) = w$ .

**Definition 13 (2-Decisional Quasi-Cyclic Syndrome Decoding with Parity (2-DQCSDP) Problem [24]).** Let  $n, w$  be positive integers, and let  $b_1 \in \{0, 1\}$ . Define  $b_2 = (w + b_1 w) \bmod 2$ . The 2-DQCSDP( $n, w, b_1, b_2$ ) problem states that given a pair  $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_{2,b_1}^{n \times 2n} \times \mathbb{F}_{2,b_2}^n$ , decide with non-negligible advantage whether  $(\mathbf{H}, \mathbf{y})$  is sampled from the distribution 2-QCSDP( $n, w, b_1, b_2$ ) or from the uniform distribution over  $\mathbb{F}_{2,b_1}^{n \times 2n} \times \mathbb{F}_{2,b_2}^n$ .

More formally, the 2-DQCSDP problem is said to be hard if for any PPT distinguisher  $\mathcal{D}$ , the advantage  $\text{Adv}_{\mathcal{D}}^{2\text{-DQCSDP}}(1^\lambda)$  in solving the 2-DQCSDP problem is negligible, i.e.,

$$\text{Adv}_{\mathcal{D}}^{2\text{-DQCSDP}}(1^\lambda) = \left| \Pr[\mathcal{D}(\mathbf{H}, \mathbf{y}) = 1 \mid \mathbf{H} \in \mathbb{F}_{2,b_1}^{n \times 2n}, \mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)] - \Pr[\mathcal{D}(\mathbf{H}, \mathbf{y}) = 1 \mid \mathbf{H} \xleftarrow{\$} \mathbb{F}_{2,b_1}^{n \times 2n}, \mathbf{y} \xleftarrow{\$} \mathbb{F}_{2,b_2}^n] \right| < \varepsilon \text{ for some negligible function } \varepsilon.$$

**Definition 14 (3-Quasi Cyclic Syndrome Decoding with Parity (3-QCSDP) Distribution [24]).** Let  $n, w, b_1, b_2$  be positive integers and define  $b_3 = (w + b_1w) \bmod 2$ . The  $3\text{-QCSDP}(n, w, b_1, b_2)$  distribution is defined as the distribution over pairs  $(\mathbf{H}, (\mathbf{y}_1, \mathbf{y}_2))$  where  $\mathbf{H} \xleftarrow{\$} \mathbb{F}_{2, b_1, b_2}^{2n \times 3n}$  is sampled randomly and the syndrome  $\mathbf{y} = (\mathbf{y}_1, \mathbf{y}_2)$  with  $(\mathbf{y}_1, \mathbf{y}_2) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$  satisfies  $\mathbf{y}^T = \mathbf{H}\mathbf{x}^T$  which is computed by sampling a random vector  $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3) \xleftarrow{\$} \mathbb{F}_2^{3n}$  such that  $\text{wt}(\mathbf{x}_1) = \text{wt}(\mathbf{x}_2) = \text{wt}(\mathbf{x}_3) = w$ .

**Definition 15 (3-Decisional Quasi-Cyclic Syndrome Decoding problem with Parity (3-DQCSDP) Problem [24]).** Let  $n, w, b_1, b_2$  be positive integers and define  $b_3 = (w + b_1w) \bmod 2$  and  $b'_3 = (w + b_2w) \bmod 2$ . The  $3\text{-DQCSDP}(n, w, b_1, b_2)$  problem states that given a pair  $(\mathbf{H}, (\mathbf{y}_1, \mathbf{y}_2)) \in \mathbb{F}_{2, b_1, b_2}^{2n \times 3n} \times (\mathbb{F}_{2, b_3}^n \times \mathbb{F}_{2, b'_3}^n)$ , decide with non-negligible advantage whether the pair is sampled from the  $3\text{-QCSDP}(n, w, b_1, b_2)$  distribution or from the uniform distribution over  $\mathbb{F}_{2, b_1, b_2}^{2n \times 3n} \times (\mathbb{F}_{2, b_3}^n \times \mathbb{F}_{2, b'_3}^n)$ .

More formally, the 3-DQCSDP problem is said to be hard if for any PPT distinguisher  $\mathcal{D}$ , the advantage  $\text{Adv}_{\mathcal{D}}^{3\text{-DQCSDP}}(1^\lambda)$  in solving the 3-DQCSDP problem is negligible, i.e.,

$$\text{Adv}_{\mathcal{D}}^{3\text{-DQCSDP}}(1^\lambda) = |\Pr[\mathcal{D}(\mathbf{H}, \mathbf{y} = \mathbf{x}\mathbf{H}^T) = 1 \mid \mathbf{H} \in \mathbb{F}_{2, b_1, b_2}^{2n \times 3n}, \mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)] - \Pr[\mathcal{D}(\mathbf{H}, \mathbf{y}) = 1 \mid \mathbf{H} \xleftarrow{\$} \mathbb{F}_{2, b_1, b_2}^{2n \times 3n}, \mathbf{y} \xleftarrow{\$} \mathbb{F}_{2, b_3}^n \times \mathbb{F}_{2, b'_3}^n]| < \varepsilon$$

for some negligible function  $\varepsilon$ .

### 2.3 Public Key Encryption

**Definition 16 (Public Key Encryption [10]).** A public key encryption (PKE) scheme consists of the following four PPT algorithms  $\text{PKE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$  associated with a public key space  $\mathcal{PKS}$ , secret key space  $\mathcal{SKS}$ , message space  $\mathcal{MS}$  and a ciphertext space  $\mathcal{CS}$  and randomness space  $\mathcal{RS}$ .

- $\text{PKE.Setup}(1^\lambda) \rightarrow \text{pp}_{\text{PKE}}$ : On input of the security parameter  $\lambda$ , a trusted third party runs this algorithm and publishes public parameters  $\text{pp}_{\text{PKE}}$ .
- $\text{PKE.KeyGen}(\text{pp}_{\text{PKE}}) \rightarrow (\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}})$ : A user runs this algorithm on input of the public parameter  $\text{pp}_{\text{PKE}}$  and outputs a pair of secret key and public key  $(\text{sk}_{\text{PKE}}, \text{pk}_{\text{PKE}}) \in \mathcal{SKS} \times \mathcal{PKS}$ . It publishes the public key  $\text{pk}_{\text{PKE}}$  while keeping the secret key  $\text{sk}_{\text{PKE}}$  secret.
- $\text{PKE.Enc}(\text{pp}_{\text{PKE}}, \text{pk}_{\text{PKE}}, \mathbf{m}; \theta) \rightarrow \text{CT}_{\mathbf{m}}$ : An encryptor runs this PPT algorithm by taking as input the public parameter  $\text{pp}_{\text{PKE}}$ , public key  $\text{pk}_{\text{PKE}}$  and a message  $\mathbf{m} \in \mathcal{MS}$  and returns a ciphertext  $\text{CT}_{\mathbf{m}} \in \mathcal{CS}$  which is sent through an insecure channel. Here  $\theta \in \mathcal{RS}$  is an encryption randomness chosen by the algorithm during encryption.

- $\text{PKE.Dec}(\text{pp}_{\text{PKE}}, \text{sk}_{\text{PKE}}, \text{CT}_{\mathbf{m}}) \longrightarrow \mathbf{m}/\perp$  : On input of the public parameter  $\text{pp}_{\text{PKE}}$ , the secret key  $\text{sk}_{\text{PKE}}$  and the ciphertext  $\text{CT}_{\mathbf{m}}$ , the decryptor runs this algorithm and recovers either the message  $\mathbf{m}$  or returns  $\perp$ , a designated symbol indicating failure.

**Correctness:** We say that a public key encryption scheme  $\text{PKE}$  is correct if for any security parameter  $\lambda$ ,

$$\Pr \left[ \text{PKE.Dec}(\text{pp}_{\text{PKE}}, \text{sk}_{\text{PKE}}, \text{CT}_{\mathbf{m}}) = \mathbf{m} \mid \begin{array}{l} \text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda), \\ (\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}}), \\ \text{CT}_{\mathbf{m}} \leftarrow \text{PKE.Enc}(\text{pp}_{\text{PKE}}, \text{pk}_{\text{PKE}}, \mathbf{m}; \theta) \end{array} \right] = 1.$$

**Definition 17 (Indistinguishability under Chosen Plaintext Attack (IND-CPA) [17]).** A public key encryption scheme ( $\text{PKE}$ ) is said to be IND-CPA secure if for any PPT adversary  $\mathcal{A}$ , the advantage

$$\text{Adv}_{\text{PKE}, \mathcal{A}}^{\text{IND-CPA}}(1^\lambda) = \left| \Pr \left[ \text{EXP}_{\text{PKE}, \mathcal{A}}^{\text{IND-CPA}}(1^\lambda) = 1 \right] - \frac{1}{2} \right| = \left| \Pr [b = b'] - \frac{1}{2} \right|$$

is negligible, where the IND-CPA security experiment  $\text{EXP}_{\text{PKE}, \mathcal{A}}^{\text{IND-CPA}}(1^\lambda)$  between a challenger  $\mathcal{C}$  and the adversary  $\mathcal{A}$  is described in Figure 1.

**Setup Phase:** The challenger  $\mathcal{C}$  computes a public parameter  $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$  and a pair of public-secret key  $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$ . It forwards the public parameter  $\text{pp}_{\text{PKE}}$  and the public key  $\text{pk}_{\text{PKE}}$  to the adversary  $\mathcal{A}$  while keeping the secret key  $\text{sk}_{\text{PKE}}$  to itself.

**Challenge Phase:** After receiving  $(\mathbf{m}_0^*, \mathbf{m}_1^*) \in \mathcal{MS} \times \mathcal{MS}$  a message pair from the adversary  $\mathcal{A}$ , the challenger  $\mathcal{C}$  uniformly samples  $b \xleftarrow{\$} \{0, 1\}$  and sends  $\text{CT}_{\mathbf{m}_b}^* \leftarrow \text{PKE.Enc}(\text{pp}_{\text{PKE}}, \text{pk}_{\text{PKE}}, \mathbf{m}_b^*; \theta)$  to  $\mathcal{A}$  where  $\theta \in \mathcal{RS}$  is an encryption randomness chosen by the algorithm during encryption.

**Guess Phase:** The adversary  $\mathcal{A}$  eventually returns a bit  $b'$ . If  $b = b'$ , the experiment outputs 1, otherwise outputs 0.

Fig. 1:  $\text{EXP}_{\text{PKE}, \mathcal{A}}^{\text{IND-CPA}}(1^\lambda)$ : IND-CPA security experiment between a challenger and an adversary

## 2.4 Hamming Quasi-Cyclic Public Key Encryption (hqcpKE) [24]

The public key encryption scheme  $\text{hqcpKE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$  associated a message space  $\mathcal{MS} = \mathbb{F}_2^k$ , a ciphertext space  $\mathcal{CS} = \mathcal{R} \times \mathcal{R}$ , randomness space  $\mathcal{RS} = \mathbb{F}_2^k$  and key space (both public key and secret key)  $\mathcal{KS} =$

$\mathcal{R} \times \mathcal{R}$ . It employs two types of codes—a decodable linear  $[n, k]$  code  $\mathbf{C}$  over  $\mathbb{F}_2$  having a generator matrix  $\mathbf{G} \in \mathbb{F}_2^{k \times n}$  and error-correcting capacity  $\tau$  and a random double-circulant  $[2n, n]$  code having a parity check matrix in the form  $(\mathbf{1}, \mathbf{h})$  consists of four PPT algorithms satisfying the following requirements where  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$  and we use the notation  $\mathcal{S}_w^n(\mathbb{F}_2)$  defined as  $\mathcal{S}_w^n(\mathbb{F}_2) = \{\mathbf{v} \in \mathbb{F}_2^n \mid \text{wt}(\mathbf{v}) = w\}$ .

- $\text{hqcpKE.Setup}(1^\lambda) \rightarrow \text{pp}_{\text{hqcpKE}}$ : Given a security parameter  $\lambda$ , a trusted authority executes this PPT algorithm to generate the public parameters  $\text{pp}_{\text{hqcpKE}}$  as follows.
  1. Chooses non-negative integers  $n, k, \alpha, \tau, w, w_r, w_e$  where  $n$  is the length and  $k$  is the dimension of the code. Here  $\tau$  denotes the error-correcting capability  $\alpha$  denotes the length of the encryption randomness and  $w, w_r, w_e$  denote the weight of the secret key components and error vectors.
  2. Selects a generator matrix  $\mathbf{G} \in \mathbb{F}_2^{k \times n}$  of a decodable  $[n, k]$  linear code  $\mathbf{C}$  with error correcting capability  $\tau$  and a randomness generator  $\mathbf{P} : \{0, 1\}^* \rightarrow \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$ .
  3. Publishes the public parameter  $\text{pp}_{\text{hqcpKE}} = (n, k, \alpha, \tau, w, w_r, w_e, \mathbf{G}, \mathbf{P})$ .
- $\text{hqcpKE.KeyGen}(\text{pp}_{\text{hqcpKE}}) \rightarrow (\text{pk}_{\text{hqcpKE}}, \text{sk}_{\text{hqcpKE}})$ : On input the public parameters  $\text{pp}_{\text{hqcpKE}}$ , a user executes the key generation algorithm to generate public-secret key pair  $\text{pk}_{\text{hqcpKE}}, \text{sk}_{\text{hqcpKE}}$  by performing the following steps.
  1. Samples  $\mathbf{h} \xleftarrow{\$} \mathcal{R}$  and  $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$ . Note that,  $\mathbf{x} \in \mathbb{F}_2^n, \mathbf{y} \in \mathbb{F}_2^n$  with  $\text{wt}(\mathbf{x}) = w$  and  $\text{wt}(\mathbf{y}) = w$ .
  2. Computes  $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y} \in \mathcal{R}$ .
  3. Sets  $\text{pk}_{\text{hqcpKE}} = (\mathbf{h}, \mathbf{s})$  and  $\text{sk}_{\text{hqcpKE}} = (\mathbf{x}, \mathbf{y})$ .
  4. Publishes the public key  $\text{pk}_{\text{hqcpKE}} = (\mathbf{h}, \mathbf{s})$  and keep the secret  $\text{sk}_{\text{hqcpKE}} = (\mathbf{x}, \mathbf{y})$  secret to itself.
- $\text{hqcpKE.Enc}(\text{pp}_{\text{hqcpKE}}, \text{pk}_{\text{hqcpKE}}, \mathbf{m}; \theta) \rightarrow \text{CT}_{\mathbf{m}}$ : An encryptor runs this procedure with input of the public parameter  $\text{pp}_{\text{hqcpKE}}$ , the public key  $\text{pk}_{\text{hqcpKE}}$  and a message  $\mathbf{m} \in \mathbb{F}_2^k$  and returns a ciphertext  $\text{CT}_{\mathbf{m}}$  by carrying out the following steps.
  1. Samples a random  $\alpha$ -bit string  $\theta \xleftarrow{\$} \mathbb{F}_2^\alpha$  and generates  $\mathbf{P}(\theta) = (\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2)$  where  $\mathbf{r}_1, \mathbf{r}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2), \mathbf{e} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$ .
  2. Computes  $\mathbf{u}_{\mathbf{m}} = \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2 \in \mathcal{R}$  and  $\mathbf{v}_{\mathbf{m}} = \mathbf{m}\mathbf{G} + \mathbf{s}\mathbf{r}_2 + \mathbf{e} \in \mathcal{R}$ .
  3. Returns the ciphertext  $\text{CT}_{\mathbf{m}} = (\mathbf{u}_{\mathbf{m}}, \mathbf{v}_{\mathbf{m}})$ .
- $\text{hqcpKE.Dec}(\text{pp}_{\text{hqcpKE}}, \text{sk}_{\text{hqcpKE}}, \text{CT}_{\mathbf{m}}) \rightarrow \mathbf{m}/\perp$ : The decryptor, having the public parameter  $\text{pp}_{\text{hqcpKE}}$  and secret key  $\text{sk}_{\text{hqcpKE}} = (\mathbf{x}, \mathbf{y})$ , runs this algorithm to decrypt the ciphertext  $\text{CT}_{\mathbf{m}} = (\mathbf{u}_{\mathbf{m}}, \mathbf{v}_{\mathbf{m}})$  with  $\mathbf{u}_{\mathbf{m}} = \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2 \in \mathcal{R}$  and  $\mathbf{v}_{\mathbf{m}} = \mathbf{m}\mathbf{G} + \mathbf{s}\mathbf{r}_2 + \mathbf{e} \in \mathcal{R}$ .
  1. Performs  $\mathbf{C.decode}(\mathbf{v}_{\mathbf{m}} - \mathbf{u}_{\mathbf{m}}\mathbf{y}) \rightarrow \mathbf{m}/\perp$  where the procedure  $\mathbf{C.decode}()$  is the decoding algorithm of the  $[n, k]$  linear code  $\mathbf{C}$  with generator matrix  $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ . We provided an instantiation of the encoding-decoding algorithm for Reed-Solomon code in Subsection 2.5.

**Correctness.** The decryption of hqcPKE depends on the decoding technique of the  $[n, k]$  binary linear code  $\mathbf{C}$  and the decryption occurs when the decoding algorithm  $\mathbf{C.decode}$  correctly decodes  $\mathbf{v}_m - \mathbf{u}_m \mathbf{y}$  as,

$$\begin{aligned} \mathbf{v}_m - \mathbf{u}_m \mathbf{y} &= \mathbf{mG} + \mathbf{sr}_2 + \mathbf{e} - (\mathbf{r}_1 + \mathbf{hr}_2)\mathbf{y} = \mathbf{mG} + (\mathbf{x} + \mathbf{hy})\mathbf{r}_2 + \mathbf{e} - (\mathbf{r}_1 + \mathbf{hr}_2)\mathbf{y} \\ &= \mathbf{mG} + \mathbf{xr}_2 + \mathbf{e} - \mathbf{r}_1 \mathbf{y}. \end{aligned}$$

The decoding correctly executes if  $\text{wt}(\mathbf{xr}_2 + \mathbf{e} - \mathbf{r}_1 \mathbf{y}) \leq \tau$ ,  $\tau$  being the error-correcting capability of the code  $\mathbf{C}$ .

**Theorem 1 ([24]).** Let  $\mathbf{x} = (x_0, x_1, \dots, x_{n-1}) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2)$ ,  $\mathbf{r} = (r_0, r_1, \dots, r_{n-1}) \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2)$  be mutually independent and  $\mathbf{z} = \mathbf{xr} = (z_0, z_1, \dots, z_{n-1})$ . Then for every  $k \in \{0, 1, \dots, n-1\}$ , the  $k$ -th coordinate  $z_k$  of  $\mathbf{z}$  is Bernoulli distributed with parameter  $\tilde{p} = \Pr[z_k = 1]$  where

$$\tilde{p} = \frac{T}{\binom{n}{w} \binom{n}{w_r}}, \quad T = \sum_{\substack{1 \leq \ell \leq \min(w, w_r) \\ \ell \text{ odd}}} C_\ell, \quad C_\ell = \binom{n}{\ell} \binom{n-\ell}{w-\ell} \binom{n-w}{w_r-\ell}.$$

**Theorem 2 ([24]).** Let  $\mathbf{x}, \mathbf{y} \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2)$ ,  $\mathbf{r}_1, \mathbf{r}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\mathbf{e} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  be mutually independent random vectors and  $\mathbf{e}' = \mathbf{xr}_2 - \mathbf{r}_1 \mathbf{y} + \mathbf{e} = (e'_0, e'_1, \dots, e'_{n-1})$ . Then for every  $k \in \{0, 1, \dots, n-1\}$ , the  $k$ th coordinate  $e'_k$  of  $\mathbf{e}'$  satisfies the following distribution.

$$\begin{aligned} \Pr[e'_k = 1] &= 2\tilde{p}(1 - \tilde{p})(1 - \frac{w_e}{n}) + ((1 - \tilde{p})^2 + \tilde{p}^2) \frac{w_e}{n}, \\ \Pr[e'_k = 0] &= ((1 - \tilde{p})^2 + \tilde{p}^2)(1 - \frac{w_e}{n}) + 2\tilde{p}(1 - \tilde{p}) \frac{w_e}{n}, \\ \text{with } \tilde{p} &= \frac{T}{\binom{n}{w} \binom{n}{w_r}}, \quad T = \sum_{\substack{1 \leq \ell \leq \min(w, w_r) \\ \ell \text{ odd}}} C_\ell, \quad C_\ell = \binom{n}{\ell} \binom{n-\ell}{w-\ell} \binom{n-w}{w_r-\ell}. \end{aligned}$$

**Theorem 3 ([24]).** The encryption scheme hqcPKE presented above is IND-CPA secure as presented in Definition 17, assuming the hardness of 2-DQCSDP and 3-DQCSDP problems defined in Definitions 13 and 15.

## 2.5 Encoding-Decoding of Reed-Solomon Code

The scheme hqcPKE can be instantiated by BCH code, Reed-Solomon code, Reed-Muller code [24]. We discuss the encoding-decoding algorithm for Reed-Solomon code [15].

A Reed-Solomon code  $\text{RS}[n, k, d_{\min}]$  over  $\mathbb{F}_{2^m}$  is a linear code with block length  $n = 2^m - 1$ , the number of information bits  $k$  (dimension), error-correcting capacity  $\tau$  and minimum distance  $d_{\min} = 2\tau + 1$ , number of parity check digits  $n - k = 2\tau$ . The generator polynomial for  $\text{RS}[n, k, d_{\min}]$  code is  $g(x) =$

$(x + \alpha)(x + \alpha^2) \cdots (x + \alpha^{2^\tau})$  where  $\alpha$  is a primitive element in  $\mathbb{F}_{2^m}$ . A message  $\mathbf{u} = (u_0, u_1, \dots, u_{k-1}) \in \mathbb{F}_2^k$  which is represented as a polynomial  $u(x) = u_0 + u_1x + \cdots + u_{k-1}x^{k-1} \in \mathbb{F}_2[x]$ . The encoding procedure consists of the three steps. The decoding procedure for  $\text{RS}[n, k, d_{\min}]$  code is presented in Fig-

1.  $u(x) = u_0 + u_1x + \cdots + u_{k-1}x^{k-1} \in \mathbb{F}_2[x]$  will be multiplied by  $x^{n-k}$ .
2. Divide  $x^{n-k}u(x)$  by the generator polynomial  $g(x)$  and obtain the remainder  $b(x)$  with  $\deg(b(x)) \leq n - k = 2\tau$ .
3. The code polynomial is  $c(x) = b(x) + x^{n-k}u(x) \in \mathbb{F}_2^n$ .

Fig. 2: Encoding procedure for the Reed-Solomon Code [15]

ure 3. Suppose the codeword  $c(x) = c_0 + c_1x + \cdots + c_{n-1}x^{n-1} \in \mathbb{F}_2^n[x]$  and the received codeword be  $r(x) = r_0 + r_1x + r_2x^2 + \cdots + r_{n-1}x^{n-1} \in \mathbb{F}_2^n[x]$ . Let  $r(x) = c(x) + e(x)$  when  $e(x) = e_0 + e_1x + \cdots + e_{n-1}x^{n-1} \in \mathbb{F}_2^n[x]$  be the error polynomial.

1. As the first step, the syndrome values  $S_1, S_2, \dots, S_{2\tau}$  of the received code  $r(x)$  are calculated where  $\tau$  is the error-correcting capacity of the code. Note that  $2\tau$  number of syndromes of the received codeword is taken so that we can construct the error location polynomial using Berlekamp's algorithm [7].
2. Given the polynomial  $r(x)$ , compute  $S_i = r(\alpha^i)$  for  $i = \{1, 2, \dots, 2\tau\}$  where  $\alpha \in \mathbb{F}_{2^m}$  is a primitive element.
3. Obtain  $S_i = c(\alpha^i) + e(\alpha^i) = 0 + e(\alpha^i) = e(\alpha^i)$  as every valid codeword  $c(x)$  satisfies,  $c(\alpha^i) = 0$  for  $i = 1, 2, \dots, 2\tau$ . Thus, the syndromes depend only on the error polynomial  $e(x)$ .
4. We assume the error polynomial has  $\delta(\leq \tau)$  errors  $e_{j_1}, e_{j_2}, \dots, e_{j_\delta}$  at locations  $j_1, j_2, \dots, j_\delta$ . These syndromes form a system of  $2\tau$  nonlinear equations.

$$\begin{cases} S_1 = e_{j_1}\alpha^{j_1} + e_{j_2}\alpha^{j_2} + \cdots + e_{j_\delta}\alpha^{j_\delta}, \\ S_2 = e_{j_1}(\alpha^{j_1})^2 + e_{j_2}(\alpha^{j_2})^2 + \cdots + e_{j_\delta}(\alpha^{j_\delta})^2, \\ S_3 = e_{j_1}(\alpha^{j_1})^3 + e_{j_2}(\alpha^{j_2})^3 + \cdots + e_{j_\delta}(\alpha^{j_\delta})^3, \\ \vdots \\ S_{2\tau} = e_{j_1}(\alpha^{j_1})^{2\tau} + e_{j_2}(\alpha^{j_2})^{2\tau} + \cdots + e_{j_\delta}(\alpha^{j_\delta})^{2\tau}. \end{cases} \quad (1)$$

5. Defining  $\beta_i = \alpha^{j_i} \in \mathbb{F}_{2^m}$  for  $i = \{1, 2, \dots, 2\tau\}$  into the system we get

$$\begin{cases} S_1 = e_{j_1}\beta_1 + e_{j_2}\beta_2 + \dots + e_{j_\delta}\beta_\delta, \\ S_2 = e_{j_1}\beta_1^2 + e_{j_2}\beta_2^2 + \dots + e_{j_\delta}\beta_\delta^2, \\ S_3 = e_{j_1}\beta_1^3 + e_{j_2}\beta_2^3 + \dots + e_{j_\delta}\beta_\delta^3, \\ \vdots \\ S_{2\tau} = e_{j_1}\beta_1^{2\tau} + e_{j_2}\beta_2^{2\tau} + \dots + e_{j_\delta}\beta_\delta^{2\tau}. \end{cases} \quad (2)$$

1. Compute the  $2\tau$  syndromes  $S_i$  as in the equation 1 using the received polynomial  $r(x) \in \mathbb{F}_2^n[x]$  by evaluating  $r(\alpha^i)$  for each  $i$  for  $i \in \{1, 2, \dots, \tau\}$ .
2. Compute the error-location polynomial  $\zeta(x) = (1 + \beta_1 x)(1 + \beta_2 x) \dots (1 + \beta_\delta x) = 1 + \zeta_1 x + \zeta_2 x^2 + \dots + \zeta_\delta x^\delta$  from the  $2\tau$  syndromes from equation 2 using the Berlekamp's algorithm [7].
3. Find the error-location numbers  $(\beta_1^{-1}, \beta_2^{-1}, \dots, \beta_\delta^{-1})$  by calculating the roots of the polynomial  $\zeta(x)$  and find their inverses.
4. Compute the error-evaluator polynomial  $Z(x) = 1 + (S_1 + \zeta_1)x + (S_2 + \zeta_1 S_1 + \zeta_2)x^2 + \dots + (S_\delta + \zeta_1 S_{\delta-1} + \dots + \zeta_\delta)x^\delta$ .
5. Compute the error values  $e_{j_l} = \frac{Z(\beta_l^{-1})}{\prod_{\substack{i=1 \\ i \neq l}}^\delta (1 + \beta_i \beta_l^{-1})}$  at the location  $\beta_l$ .
6. Correct the errors in the received polynomial  $r(x)$  by computing  $r(x) - e(x)$  where  $e(x) = (e_{j_1}, e_{j_2}, \dots, e_{j_\delta})$ .

Fig. 3: Decoding procedure for the Reed-Solomon Code [15]

## 2.6 Updatable Public Key Encryption

**Definition 18.** [11] An updatable public key encryption (uPKE) scheme associated with a message space  $\mathcal{MS}$ , a ciphertext space  $\mathcal{CT}$ , a public key space  $\mathcal{PKS}$ , a secret key space  $\mathcal{SKS}$  and randomness space  $\mathcal{RS}$ ,  $\mathcal{RS}'$  is specified by the following six PPT algorithms  $\text{uPKE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{PkUpdate}, \text{SkUpdate})$ .

- $\text{uPKE.Setup}(1^\lambda) \rightarrow \text{pp}_{\text{uPKE}}$ : A trusted authority runs the PPT algorithm taking the security parameter  $\lambda$  as input and publishes a public parameter  $\text{pp}_{\text{uPKE}}$ .
- $\text{uPKE.KeyGen}(\text{pp}_{\text{uPKE}}) \rightarrow (\text{pk}_0^{(u)}, \text{sk}_0^{(u)})$ : This PPT algorithm, run by a user  $u$ , takes the public parameter  $\text{pp}_{\text{uPKE}}$  as input and outputs a fresh secret key  $\text{sk}_0^{(u)}$  and a fresh public key  $\text{pk}_0^{(u)}$  at epoch 0. It keeps the secret key  $\text{sk}_0^{(u)}$  to itself and publishes the public key  $\text{pk}_0^{(u)}$ .

- $\text{uPKE.Enc}(\text{pp}_{\text{uPKE}}, \text{pk}_i^{(u)}, \mathbf{m}; \theta) \longrightarrow \text{CT}_{\mathbf{m}}$ : The encryptor runs this PPT algorithm, taking the public parameter  $\text{pp}_{\text{uPKE}}$ , public key  $\text{pk}_i^{(u)}$  at epoch  $i$  and a message  $\mathbf{m} \in \mathcal{MS}$  as input and outputs a ciphertext  $\text{CT}_{\mathbf{m}} \in \mathcal{CS}$  where  $\theta \in \mathcal{RS}'$  is an encryption randomness chosen by the algorithm during encryption.
- $\text{uPKE.Dec}(\text{pp}_{\text{uPKE}}, \text{sk}_i^{(u)}, \text{CT}_{\mathbf{m}}) \longrightarrow \mathbf{m}/\perp$ : The decryptor runs this PPT algorithm, which takes the public parameter  $\text{pp}_{\text{uPKE}}$ , the secret key  $\text{sk}_i^{(u)}$  at epoch  $i$  and the ciphertext  $\text{CT}_{\mathbf{m}}$  as input and returns either the message  $\mathbf{m}$  or  $\perp$  which indicates the decryption failure.
- $\text{uPKE.PkUpdate}(\text{pp}_{\text{uPKE}}, \text{pk}_i^{(u)}; \mu_i) \longrightarrow (\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)})$ : Any user who wants to update the public key can run this PPT algorithm which takes the public parameter  $\text{pp}_{\text{uPKE}}$ , the public key  $\text{pk}_i^{(u)}$  at epoch  $i$  as input and outputs an update ciphertext  $\text{up}_{i+1}^{(u)}$  and a new public key  $\text{pk}_{i+1}^{(u)}$  at epoch  $i+1$  where  $\mu_i \in \mathcal{RS}$  is randomness chosen by the algorithm during public key update.
- $\text{uPKE.SkUpdate}(\text{pp}_{\text{uPKE}}, \text{up}_{i+1}^{(u)}, \text{sk}_i^{(u)}) \longrightarrow \text{sk}_{i+1}^{(u)}$ : A user  $u$  runs this PPT algorithm on input the public parameter  $\text{pp}_{\text{uPKE}}$ , the update ciphertext  $\text{up}_{i+1}^{(u)}$  at epoch  $i+1$  and the secret key  $\text{sk}_i^{(u)}$  at epoch  $i$  and outputs an updated secret key  $\text{sk}_{i+1}^{(u)}$  for epoch  $i+1$ .

**Correctness:** For any sequence of randomness  $\{\mu_i\}_{i=1}^q$ , where  $q$  is any positive integer representing an arbitrary chain of updates and  $\mu_i \in \mathcal{RS}$  the scheme  $\text{uPKE}$  is said to be correct if for any message  $\mathbf{m} \in \mathcal{MS}$  and for any  $i \in \{0, 1, 2, \dots, q\}$ ,

$$\Pr \left[ \begin{array}{l} \text{uPKE.Dec}(\text{pp}_{\text{uPKE}}, \text{sk}_{i+1}^{(u)}, \text{CT}_{\mathbf{m}}) = \mathbf{m} \\ \text{pp}_{\text{uPKE}} \leftarrow \text{uPKE.Setup}(1^\lambda), \\ (\text{pk}_0^{(u)}, \text{sk}_0^{(u)}) \leftarrow \text{uPKE.KeyGen}(\text{pp}_{\text{uPKE}}), \\ (\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)}) \leftarrow \text{uPKE.PkUpdate}(\text{pp}_{\text{uPKE}}, \text{pk}_i^{(u)}; \mu_i), \\ \text{sk}_{i+1}^{(u)} \leftarrow \text{uPKE.SkUpdate}(\text{pp}_{\text{uPKE}}, \text{up}_{i+1}^{(u)}, \text{sk}_i^{(u)}), \\ \text{CT}_{\mathbf{m}} \leftarrow \text{uPKE.Enc}(\text{pp}_{\text{uPKE}}, \text{pk}_{i+1}^{(u)}, \mathbf{m}; \theta) \end{array} \right] \geq 1 - \delta(\lambda)$$

where  $\delta(\lambda)$  denotes the decryption failure rate.

**Definition 19 (Indistinguishability under Chosen Randomness Chosen Plaintext Attack (IND-CR-CPA)[11]).** Consider an updatable public key encryption scheme  $\text{uPKE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{PkUpdate}, \text{SkUpdate})$ . The scheme  $\text{uPKE}$  is said to be IND-CR-CPA secure if for any PPT adversary  $\mathcal{A}$  the advantage

$$\text{Adv}_{\text{uPKE}, \mathcal{A}}^{\text{IND-CR-CPA}}(1^\lambda) = \left| \Pr \left[ \text{EXP}_{\text{uPKE}, \mathcal{A}}^{\text{IND-CR-CPA}}(1^\lambda) = 1 \right] - \frac{1}{2} \right| = \left| \Pr [b = b'] - \frac{1}{2} \right|$$

is negligible where the IND-CR-CPA security experiment  $\text{EXP}_{\text{uPKE}, \mathcal{A}}^{\text{IND-CR-CPA}}(1^\lambda)$  is the security game between a challenger  $\mathcal{C}$  and the adversary  $\mathcal{A}$  as presented in Figure 4.

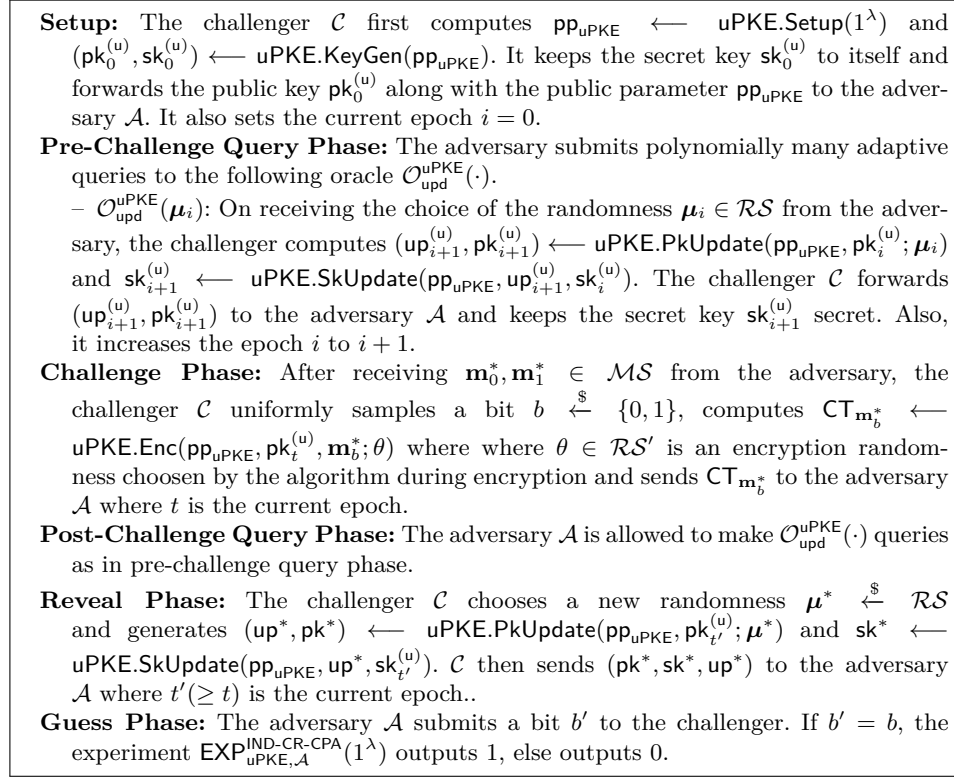


Fig. 4:  $\text{EXP}_{\text{uPKE}, \mathcal{A}}^{\text{IND-CR-CPA}}(1^\lambda)$  : IND-CR-CPA security experiment between a challenger  $\mathcal{C}$  and the adversary  $\mathcal{A}$

### 3 uhqcPKE: Updatable Hamming Quasi Cyclic Public Encryption

#### 3.1 Protocol Requirements:

Let  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$  and  $\text{Samp} : \mathcal{MS} \rightarrow \text{Per}(\mathcal{R})$  be a deterministic sampling algorithm where  $\mathcal{MS}$  is the message space and the set  $\text{Per}(\mathcal{R})$  is defined as the collection of special structured permutation maps  $\text{Per}(\mathcal{R}) = \{(\Pi, F)\}$  with the following properties:

- **Linearity/Homomorphism:** For any  $\mathbf{x}, \mathbf{y}, \mathbf{h} \in \mathcal{R}$ , it holds that  $F(\mathbf{x} + \mathbf{h}\mathbf{y}) = F(\mathbf{x}) + \Pi(\mathbf{h})F(\mathbf{y})$ .

- **Weight-Preservation:** Both  $\Pi$  and  $F$  preserve the Hamming weight of their inputs. For any  $\mathbf{x}, \mathbf{y} \in \mathcal{R}$ ,  $\text{wt}(\mathbf{x}) = \text{wt}(\Pi(\mathbf{x}))$  and  $\text{wt}(\mathbf{y}) = \text{wt}(F(\mathbf{y}))$ .
- **Distributional Equivalence:** The set  $\text{Per}(\mathcal{R})$  must have sufficient entropy such that the output of **Samp** mimics random sampling. Specifically, for  $\mu \xleftarrow{\$} \mathbb{F}_2^k$ , let  $(\Pi_\mu, F_\mu) \leftarrow \text{Samp}(\mu)$ . For uniformly sampled  $\mathbf{x} \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2)$  and  $\mathbf{h} \xleftarrow{\$} \mathcal{R}$ , the distributions of  $F_\mu(\mathbf{x})$  and  $\Pi_\mu(\mathbf{h})$  must be computationally indistinguishable from vectors sampled uniformly at random from  $\mathcal{S}_w^n(\mathbb{F}_2)$  and  $\mathcal{R}$ , respectively.

### 3.2 Description of the protocol

Our proposed updatable hamming quasi cyclic public key encryption **uhqcPKE** = (Setup, KeyGen, Enc, Dec, PkUpdate, SkUpdate) works as follows.

- **uhqcPKE.Setup**( $1^\lambda$ )  $\rightarrow$   $\text{pp}_{\text{uhqcPKE}}$  : On input the security parameter  $\lambda$ , a trusted authority runs this PPT algorithm and publishes a set of public parameters  $\text{pp}_{\text{uhqcPKE}}$  as follows.
  1. Chooses non-negative integers  $n, k, \alpha, \tau, w, w_r, w_e$  where  $n$  is the length and  $k$  is the dimension of a  $\tau$  error correcting decodable linear code  $C$  over  $\mathbb{F}_2$  and  $w, w_r, w_e$  denotes fixed weight of vectors and  $\alpha$  denotes the length of encryption randomness.
  2. Selects a generator matrix  $\mathbf{G}$  of order  $k \times n$  for the  $[n, k]$  binary linear code  $C$ .
  3. Selects a **Samp** :  $\mathbb{F}_2^k \rightarrow \text{Per}(\mathcal{R})$  where  $\text{Per}(\mathcal{R})$  is described as in Subsection 3.1 and Section 5 where  $\mathcal{R} = \mathbb{F}_2[x]/x^n - 1$ .
  4. Selects a randomness generator  $\mathbf{P} : \{0, 1\}^* \rightarrow \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ , we use the notation  $\mathcal{S}_w^n(\mathbb{F}_2) = \{\mathbf{v} \in \mathbb{F}_2^n \mid \text{wt}(\mathbf{v}) = w\}$ .
  5. Publishes public parameter  $\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_r, w_e, \text{Samp}, \mathbf{G}, \mathbf{P})$ .
- **uhqcPKE.KeyGen**( $\text{pp}_{\text{uhqcPKE}}$ )  $\rightarrow (\text{pk}_0^{(u)}, \text{sk}_0^{(u)})$  : A user  $u$  runs this PPT algorithm taking  $\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_r, w_e, \text{Samp}, \mathbf{G}, \mathbf{P})$ , the public parameter, as input and returns a fresh pair of public key and secret key  $(\text{pk}_0^{(u)}, \text{sk}_0^{(u)})$  at epoch 0 by computing the following steps.
  1. Samples  $\mathbf{h}_0 \xleftarrow{\$} \mathcal{R}$  and  $(\mathbf{x}_0, \mathbf{y}_0) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$ . Note that  $\mathbf{x}_0, \mathbf{y}_0 \in \mathbb{F}_2^n$  with  $\text{wt}(\mathbf{x}_0) = w = \text{wt}(\mathbf{y}_0)$ .
  2. Computes  $\mathbf{s}_0 = \mathbf{x}_0 + \mathbf{h}_0 \mathbf{y}_0 \in \mathcal{R}$ .
  3. Sets the public key  $\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0)$  and secret key  $\text{sk}_0^{(u)} = (\mathbf{x}_0, \mathbf{y}_0)$ .
  4. Publishes the public key  $\text{pk}_0^{(u)}$  while keeping the secret key  $\text{sk}_0^{(u)}$  secret to itself.
- **uhqcPKE.Enc**( $\text{pp}_{\text{uhqcPKE}}, \text{pk}_i^{(u)}, \mathbf{m}; \theta$ )  $\rightarrow \text{CT}_{\mathbf{m}}$  : The encryptor runs the algorithm, taking public parameter  $\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_r, w_e, \text{Samp}, \mathbf{G}, \mathbf{P})$ , public key  $\text{pk}_i^{(u)} = (\mathbf{h}_i, \mathbf{s}_i)$  at epoch  $i$ , a message  $\mathbf{m} \in \mathbb{F}_2^k$  as input and outputs  $\text{CT}_{\mathbf{m}} = (\mathbf{u}_{\mathbf{m}}, \mathbf{v}_{\mathbf{m}}) \in \mathcal{R} \times \mathcal{R}$  by executing the following steps.

1. Choose a  $\alpha$ -bit string  $\theta \in \mathbb{F}_2^\alpha$  and compute  $P(\theta) = (\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2)$  where  $\mathbf{e} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  and  $\mathbf{r} = (\mathbf{r}_1, \mathbf{r}_2) \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ . Note that,  $\text{wt}(\mathbf{e}) = w_e$  and  $\text{wt}(\mathbf{r}_1) = \text{wt}(\mathbf{r}_2) = w_r$ .
  2. Computes  $\mathbf{u}_m = \mathbf{r}_1 + \mathbf{h}_i \mathbf{r}_2 \in \mathcal{R}$  and  $\mathbf{v}_m = \mathbf{m} \mathbf{G} + \mathbf{s}_i \mathbf{r}_2 + \mathbf{e} \in \mathcal{R}$ .
  3. Returns  $\text{CT}_m = (\mathbf{u}_m, \mathbf{v}_m)$ .
- $\text{uhqcPKE.Dec}(\text{pp}_{\text{uhqcPKE}}, \text{sk}_i^{(u)}, \text{CT}_m) \longrightarrow \mathbf{m}/\perp$  : The decryptor runs this algorithm with the public parameter  $\text{pp}_{\text{uhqcPKE}} = (n, k, \tau, w, w_r, w_e, \text{Samp}, \mathbf{G}, \mathbf{P})$ , its secret key  $\text{sk}_i^{(u)} = (\mathbf{x}_i, \mathbf{y}_i)$  at epoch  $i$  and the ciphertext  $\text{CT}_m = (\mathbf{u}_m, \mathbf{v}_m)$  as input and outputs  $\mathbf{m}$  or  $\perp$  by running the decoding algorithm  $\text{C.decode}(\mathbf{v}_m - \mathbf{u}_m \mathbf{y}_i)$  as in the decryption algorithm (for RS code) of hqcPKE presented in Subsection 2.4.
- $\text{uhqcPKE.PkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_i^{(u)}; \boldsymbol{\mu}_i) \longrightarrow (\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)})$  : Taking as input the public parameter  $\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_r, w_e, \text{Samp}, \mathbf{G}, \mathbf{P})$ , the public key  $\text{pk}_i^{(u)} = (\mathbf{h}_i, \mathbf{s}_i = \mathbf{x}_i + \mathbf{h}_i \mathbf{y}_i)$  at epoch  $i$ , a user runs this algorithm and updates the public key by computing the following steps to output a new pair of an update ciphertext and public key  $(\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)})$ .
1. Samples a randomness  $\boldsymbol{\mu}_i \xleftarrow{\$} \mathbb{F}_2^k$  and generates  $(\Pi_{\boldsymbol{\mu}_i}, \mathbf{F}_{\boldsymbol{\mu}_i}) = \text{Samp}(\boldsymbol{\mu}_i) \in \text{Per}(\mathcal{R})$ .
  2. Computes  $\Pi_{\boldsymbol{\mu}_i}(\mathbf{h}_i) = \mathbf{h}_{i+1} \in \mathcal{R}$  and  $\mathbf{s}_{i+1} = \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{s}_i)$ . Note that  $\mathbf{s}_{i+1} = \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{s}_i) = \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{x}_i + \mathbf{h}_i \mathbf{y}_i) = \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{x}_i) + \Pi_{\boldsymbol{\mu}_i}(\mathbf{h}_i) \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{y}_i) = \mathbf{x}_{i+1} + \mathbf{h}_{i+1} \mathbf{y}_{i+1} \in \mathcal{R}$  for epoch  $i+1$ .
  3. Encrypts  $\boldsymbol{\mu}_i$  by running the  $\text{uhqcPKE.Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_i^{(u)}, \boldsymbol{\mu}_i; \theta')$  which outputs  $\text{up}_{i+1}^{(u)} = \text{CT}_{\boldsymbol{\mu}_i} = (\mathbf{u}_{\boldsymbol{\mu}_i}, \mathbf{v}_{\boldsymbol{\mu}_i}) = (\hat{\mathbf{r}}_1 + \mathbf{h}_i \hat{\mathbf{r}}_2, \boldsymbol{\mu}_i \mathbf{G} + \mathbf{s}_i \hat{\mathbf{r}}_2 + \hat{\mathbf{e}})$  where  $P(\theta') = (\hat{\mathbf{e}}, \hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2)$  for  $\theta' \in \mathbb{F}_2^\alpha$ .
  4. Updates the public key as  $\text{pk}_{i+1}^{(u)} = (\mathbf{h}_{i+1}, \mathbf{s}_{i+1})$ .
  5. Returns update ciphertext-public key pair  $(\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)})$ .
- $\text{uhqcPKE.SkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{up}_{i+1}^{(u)}, \text{sk}_i^{(u)}) \longrightarrow \text{sk}_{i+1}^{(u)}$  : The user  $u$  runs this algorithm on input the public parameter  $\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_r, w_e, \text{Samp}, \mathbf{G}, \mathbf{P})$ , its update ciphertext  $\text{up}_{i+1}^{(u)}$  at epoch  $i+1$ , secret key  $\text{sk}_i^{(u)}$  at epoch  $i$  and generates its updated secret key  $\text{sk}_{i+1}^{(u)}$  for epoch  $i+1$  by executing the following steps.
1. Executes the algorithm  $\text{uhqcPKE.Dec}(\text{pp}_{\text{uhqcPKE}}, \text{sk}_i^{(u)}, \text{up}_{i+1}^{(u)})$  returns  $\boldsymbol{\mu}_i$  by running the algorithm  $\text{C.Decode}(\mathbf{v}_{\boldsymbol{\mu}_i} - \mathbf{u}_{\boldsymbol{\mu}_i} \mathbf{y}_i)$ .
  2. Computes  $(\Pi_{\boldsymbol{\mu}_i}, \mathbf{F}_{\boldsymbol{\mu}_i}) = \text{Samp}(\boldsymbol{\mu}_i) \in \text{Per}(\mathcal{R})$ ,  $\mathbf{x}_{i+1} = \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{x}_i) \in \mathcal{R}$  and  $\mathbf{y}_{i+1} = \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{y}_i) \in \mathcal{R}$ . Note that  $\text{wt}(\mathbf{x}_{i+1}) = \text{wt}(\mathbf{x}_i) = w$  and  $\text{wt}(\mathbf{y}_{i+1}) = \text{wt}(\mathbf{y}_i) = w$ .
  3. Update the secret key as  $\text{sk}_{i+1}^{(u)} = (\mathbf{x}_{i+1}, \mathbf{y}_{i+1})$ .

### 3.3 Correctness

For a given security parameter  $\lambda$ , let  $\text{pp}_{\text{uhqcPKE}} \leftarrow \text{uhqcPKE.Setup}(1^\lambda)$ . The scheme is correct if, for any non-negative integer  $q$  representing an arbitrary chain of updates, the decryption failure rate is bounded by  $\delta(1^\lambda)$ .

We demonstrate correctness by tracking the evolution of keys from epoch 0 to an arbitrary epoch  $i + 1$  where  $i \in \{0, 1, 2, \dots, q\}$ .

*Epoch 0:*

$$(\text{sk}_0^{(u)}, \text{pk}_0^{(u)}) \leftarrow \text{uhqcPKE.KeyGen}(\text{pp}_{\text{uhqcPKE}})$$

where  $\text{sk}_0^{(u)} = (\mathbf{x}_0, \mathbf{y}_0) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$  and  $\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0 = \mathbf{x}_0 + \mathbf{h}_0 \mathbf{y}_0) \in \mathcal{R} \times \mathcal{R}$ .

*Epochs 0 to  $i$ :* Assume a sequence of  $i + 1$  updates generated by sampling update messages  $\boldsymbol{\mu}_j \xleftarrow{\$} \mathbb{F}_2^k$ . For  $j \in \{0, 1, \dots, i\}$ , the keys recursively evolve as follows:

$$(\text{up}_{j+1}^{(u)}, \text{pk}_{j+1}^{(u)}) \leftarrow \text{uhqcPKE.PkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_j^{(u)}; \boldsymbol{\mu}_j)$$

$$\text{sk}_{j+1}^{(u)} \leftarrow \text{uhqcPKE.SkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{up}_{j+1}^{(u)}, \text{sk}_j^{(u)})$$

where  $\text{Samp}(\boldsymbol{\mu}_j) = (\Pi_{\boldsymbol{\mu}_j}, \mathbf{F}_{\boldsymbol{\mu}_j}) \in \text{Per}(\mathcal{R})$ . By the properties defined in Section 3.1, the keys at epoch  $i + 1$  are:

$$\text{sk}_{i+1}^{(u)} = (\mathbf{x}_{i+1}, \mathbf{y}_{i+1}) = (\mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{x}_i), \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{y}_i)) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$$

$$\text{pk}_{i+1}^{(u)} = (\mathbf{h}_{i+1}, \mathbf{s}_{i+1}) = (\Pi_{\boldsymbol{\mu}_i}(\mathbf{h}_i), \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{x}_i + \mathbf{h}_i \mathbf{y}_i) = \mathbf{x}_{i+1} + \mathbf{h}_{i+1} \mathbf{y}_{i+1}) \in \mathcal{R} \times \mathcal{R}.$$

Because  $\text{Samp}$  ensures distributional equivalence, the key pair  $(\text{sk}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)})$  is computationally indistinguishable from a freshly generated key pair from  $\text{uhqcPKE.KeyGen}$ .

*Encryption and Decryption at Epoch  $i + 1$ :* For any message  $\mathbf{m} \in \mathbb{F}_2^k$ , let  $\text{CT}_{\mathbf{m}}$  be encrypted under the updated public key  $\text{pk}_{i+1}^{(u)}$ :

$$\text{CT}_{\mathbf{m}} \leftarrow \text{uhqcPKE.Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_{i+1}^{(u)}, \mathbf{m}; \theta)$$

where  $\text{CT}_{\mathbf{m}} = (\mathbf{u}_{\mathbf{m}}, \mathbf{v}_{\mathbf{m}}) = (\mathbf{r}_1 + \mathbf{h}_{i+1} \mathbf{r}_2, \mathbf{mG} + \mathbf{s}_{i+1} \mathbf{r}_2 + \mathbf{e}) \in \mathcal{R} \times \mathcal{R}$  for randomness  $(\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) \leftarrow \text{P}(\theta)$  with appropriate weights  $w_e$  and  $w_r$ .

During decryption using  $\text{sk}_{i+1}^{(u)} = (\mathbf{x}_{i+1}, \mathbf{y}_{i+1})$ , the algebraic structure yields:

$$\begin{aligned} \mathbf{v}_{\mathbf{m}} - \mathbf{u}_{\mathbf{m}} \mathbf{y}_{i+1} &= (\mathbf{mG} + \mathbf{s}_{i+1} \mathbf{r}_2 + \mathbf{e}) - (\mathbf{r}_1 + \mathbf{h}_{i+1} \mathbf{r}_2) \mathbf{y}_{i+1} \\ &= \mathbf{mG} + (\mathbf{x}_{i+1} + \mathbf{h}_{i+1} \mathbf{y}_{i+1}) \mathbf{r}_2 + \mathbf{e} - (\mathbf{r}_1 + \mathbf{h}_{i+1} \mathbf{r}_2) \mathbf{y}_{i+1} \\ &= \mathbf{mG} + \mathbf{x}_{i+1} \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \mathbf{y}_{i+1} \end{aligned}$$

The decoding algorithm  $\text{C.decode}(\cdot)$  successfully recovers  $\mathbf{m}$  provided that the noise term satisfies  $\text{wt}(\mathbf{x}_{i+1} \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \mathbf{y}_{i+1}) \leq \tau$  where  $\tau$  is the error-correcting

capability of the code  $C$ . Since the distributions of the updated keys match those of the standard  $\text{hqcPKE}$ , the probability of a decoding failure is identically bounded by the standard  $\text{hqcPKE}$  decryption failure rate  $\delta$ .

Therefore,

$$\Pr \left[ \text{uhqcPKE.Dec}(\text{pp}_{\text{uhqcPKE}}, \text{sk}_{i+1}^{(u)}, \text{CT}_{\mathbf{m}}) = \mathbf{m} \right] \geq 1 - \delta.$$

## 4 Security Analysis

**Theorem 4.** Our proposed updatable public key encryption  $\text{uhqcPKE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{PkUpdate}, \text{SkUpdate})$  presented above is IND-CR-CPA secure as per Definition 19 under the hardness of the 2-DQCSDP and the 3-DQCSDP assumptions presented in Subsection 2.2.

*Proof.* To prove the IND-CR-CPA security of  $\text{uhqcPKE}$ , we consider a sequence of games in which the adversary's view is gradually modified and we show that no efficient adversary can distinguish between any two consecutive games. Across these games, the challenge ciphertext is slowly transformed from the original game to the one where it is independent of the bit  $b$  in the IND-CR-CPA security experiment. We argue that any adversary capable of distinguishing one game from the subsequent can be used to construct a distinguisher that breaks either the 2-DQCSDP assumption or the 3-DQCSDP assumption. For each game  $\mathcal{G}_i$ ,  $i = 0, 1, \dots, 5$ , let  $S_i$  denotes the event that the adversary correctly guesses the challenge bit, i.e.,  $b' = b$ .

- *Game  $\mathcal{G}_0$ :* This is the original IND-CR-CPA security game  $\text{EXP}_{\text{uhqcPKE}, \mathcal{A}}^{\text{IND-CR-CPA}}(1^\lambda)$  between a challenger  $\mathcal{C}$  and an adversary  $\mathcal{A}$  as shown below.

**Setup:** The challenger  $\mathcal{C}$  computes public parameter

$$\text{pp}_{\text{uhqcPKE}} \leftarrow \text{uhqcPKE.Setup}(1^\lambda)$$

and initial public key-secret key pair

$$(\text{pk}_0^{(u)}, \text{sk}_0^{(u)}) \leftarrow \text{uhqcPKE.KeyGen}(\text{pp}_{\text{uhqcPKE}})$$

where  $\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_r, w_e, \text{Samp}, \mathbf{G}, \mathbf{P})$ ,

$$\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) \in \mathcal{R} \times \mathcal{R}, \text{sk}_0^{(u)} = (\mathbf{x}_0, \mathbf{y}_0) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$$

satisfying  $\mathbf{s}_0 = \mathbf{x}_0 + \mathbf{h}_0 \mathbf{y}_0$ . Let  $b_1 = \mathbf{h}_0 \cdot \mathbf{1} \pmod{2}$  and  $b_2 = w + b_1 \times w \pmod{2}$ . Then  $\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n$  as  $\mathbf{h}_0$  has parity  $b_0$  and  $\mathbf{s}_0$  has parity  $b_1$ . It initializes the epoch  $i = 0$  and keeps the secret key  $\text{sk}_0^{(u)}$  to itself and forwards the public key  $\text{pk}_0^{(u)}$  along with the public parameter  $\text{pp}_{\text{uhqcPKE}}$  to the adversary  $\mathcal{A}$ .

**Pre-Challenge Query Phase:** The adversary submits  $t-1$  many adaptive queries to the oracle  $\mathcal{O}_{\text{upd}}^{\text{uhqcPKE}}(\cdot)$  which are simulated as follows by  $\mathcal{C}$ .

–  $\mathcal{O}_{\text{upd}}^{\text{uhqcPKE}}(\mu_i)$ : Upon receiving the choice of randomness  $\mu_i \in \mathbb{F}_2^k$  for  $i = \{0, 1, \dots, t-1\}$  from the adversary  $\mathcal{A}$ , the challenger  $\mathcal{C}$  computes

$$(\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)}) \leftarrow \text{uhqcPKE.PkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_i^{(u)}; \mu_i),$$

$$\text{sk}_{i+1}^{(u)} \leftarrow \text{uhqcPKE.SkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{up}_{i+1}^{(u)}, \text{sk}_i^{(u)})$$

where  $\text{pk}_{i+1}^{(u)} = (\mathbf{h}_{i+1} = \Pi_{\mu_i}(\mathbf{h}_i), \mathbf{s}_{i+1} = F_{\mu_i}(\mathbf{s}_i)) \in \mathcal{R} \times \mathcal{R}$ ,

$$\text{up}_{i+1}^{(u)} = \text{CT}_{\mu_i} \leftarrow \text{uhqcPKE.Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_i^{(u)}, \mu_i; \theta'),$$

where  $\text{CT}_{\mu_i} = (\mathbf{u}_{\mu_i} = \mathbf{r}'_1 + \mathbf{h}_i \mathbf{r}'_2, \mathbf{v}_{\mu_i} = \mu_i \mathbf{G} + \mathbf{s}_i \mathbf{r}'_2 + \mathbf{e}')$

$$\text{sk}_{i+1}^{(u)} = (\mathbf{x}_{i+1}, \mathbf{y}_{i+1}) = (F_{\mu_i}(\mathbf{x}_i), F_{\mu_i}(\mathbf{y}_i)) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$$

satisfying  $\mathbf{s}_{i+1} = \mathbf{x}_{i+1} + \mathbf{h}_{i+1} \mathbf{y}_{i+1}$ ,  $\text{pk}_i^{(u)} = (\mathbf{h}_i, \mathbf{s}_i) \in \mathcal{R} \times \mathcal{R}$ ,  $\text{sk}_i^{(u)} = (\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$  with  $\mathbf{s}_i = \mathbf{x}_i + \mathbf{h}_i \mathbf{y}_i$ ,  $(\Pi_{\mu_i}, F_{\mu_i}) = \text{Samp}(\mu_i) \in \text{Per}(\mathcal{R})$ ,  $P(\theta') = (\mathbf{e}', \mathbf{r}'_1, \mathbf{r}'_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\theta' \xleftarrow{\$} \mathbb{F}_2^\alpha$  is a random  $\alpha$ -bit string. It increases the epoch  $i$  to  $i+1$ , keeps  $\text{sk}_{i+1}^{(u)}$  secret to itself and forwards  $(\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)})$  to  $\mathcal{A}$ . Note that  $(\mathbf{h}_0, \mathbf{s}_0) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  implies  $\mathbf{h}_i, \mathbf{h}_{i+1} \in \mathbb{F}_{2,b_1}^n$  as they will have the same parity by the weight preserving property of  $\Pi_{\mu_i}$  and  $\mathbf{s}_i, \mathbf{s}_{i+1} \in \mathbb{F}_{2,b_2}^n$  as they have the same parity  $b_2$  by the weight preserving property of  $F_{\mu_i}$ .

**Challenge Phase:** On receiving the message pair  $(\mathbf{m}_0^*, \mathbf{m}_1^*) \in \mathbb{F}_2^k \times \mathbb{F}_2^k$  from  $\mathcal{A}$  at epoch  $t$ , the challenger  $\mathcal{C}$  samples a bit  $b \xleftarrow{\$} \{0, 1\}$ , computes

$$\text{uhqcPKE.Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_t^{(u)}, \mathbf{m}_b^*; \theta) \rightarrow \text{CT}_{\mathbf{m}_b^*}$$

where  $\text{CT}_{\mathbf{m}_b^*} = (\mathbf{u}_{\mathbf{m}_b^*} = \mathbf{r}_1 + \mathbf{h}_t \mathbf{r}_2, \mathbf{v}_{\mathbf{m}_b^*} = \mathbf{m}_b^* \mathbf{G} + \mathbf{s}_t \mathbf{r}_2 + \mathbf{e}) \in \mathcal{R} \times \mathcal{R}$  and sends  $\text{CT}_{\mathbf{m}_b^*}$  to  $\mathcal{A}$  where  $\text{pk}_t = (\mathbf{h}_t, \mathbf{s}_t)$  is the public key at epoch  $t$  satisfying  $\mathbf{s}_t = \mathbf{x}_t + \mathbf{h}_t \mathbf{y}_t$ ,  $\theta \in \mathbb{F}_2^\alpha$  is a random  $\alpha$ -bit string,  $P(\theta) = (\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ .

**Post-Challenge Query Phase:** The adversary  $\mathcal{A}$  makes  $\mathcal{O}_{\text{upd}}^{\text{uhqcPKE}}(\mu_i)$  queries for  $i = \{t+1, t+2, \dots, t'-1\}$  as in the pre-challenge query phase for  $\mu_i \in \mathbb{F}_2^k$ .

**Reveal Phase:** The challenger  $\mathcal{C}$  chooses a new randomness  $\mu^* \xleftarrow{\$} \mathbb{F}_2^k$  at epoch  $t'+1$  then generates

$$(\text{up}^*, \text{pk}^*) \leftarrow \text{uhqcPKE.PkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_{t'}^{(u)}; \mu^*)$$

and  $\text{sk}^* \leftarrow \text{uhqcPKE.SkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{up}^*, \text{sk}_{t'}^{(u)})$  and sends  $(\text{pk}^*, \text{sk}^*, \text{up}^*)$  to  $\mathcal{A}$  where

$$\text{up}^* = \text{CT}_{\mu^*} \leftarrow \text{uhqcPKE.Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_{t'}^{(u)}, \mu^*; \theta''),$$

where  $\text{CT}_{\mu^*} = (\mathbf{u}_{\mu^*} = \mathbf{r}_1'' + \mathbf{h}_{t'}\mathbf{r}_2'', \mathbf{v}_{\mu^*} = \mu^*\mathbf{G} + \mathbf{s}_{t'}\mathbf{r}_2'' + \mathbf{e}'')$ ,

$$\text{pk}^* = (\mathbf{h}^* = \Pi_{\mu^*}(\mathbf{h}_{t'}), \mathbf{s}^* = \mathbf{F}_{\mu^*}(\mathbf{s}_{t'})) \in \mathcal{R} \times \mathcal{R},$$

$$\text{sk}^* = (\mathbf{F}_{\mu^*}(\mathbf{x}_{t'}), \mathbf{F}_{\mu^*}(\mathbf{y}_{t'})) = (\mathbf{x}^*, \mathbf{y}^*) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$$

satisfying  $\mathbf{s}^* = \mathbf{x}^* + \mathbf{h}^*\mathbf{y}^*$ ,  $\alpha$ -bit string  $\theta'' \in \mathbb{F}_2^\alpha$ ,  $\mathbf{P}(\theta'') = (\mathbf{e}'', \mathbf{r}_1'', \mathbf{r}_2'') \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $(\Pi_{\mu^*}, \mathbf{F}_{\mu^*}) = \text{Samp}(\mu^*) \in \text{Per}(\mathcal{R})$ ,  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathcal{R} \times \mathcal{R}$ ,  $\text{sk}_{t'}^{(u)} = (\mathbf{x}_{t'}, \mathbf{y}_{t'}) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$  with  $\mathbf{s}_{t'} = \mathbf{x}_{t'} + \mathbf{h}_{t'}\mathbf{y}_{t'}$ .

As  $(\mathbf{h}_0, \mathbf{s}_0) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  it follows that  $\mathbf{h}_{t'}, \mathbf{h}^* \in \mathbb{F}_{2,b_1}^n$  (see Proposition 3) and  $\mathbf{s}_{t'}, \mathbf{s}^* \in \mathbb{F}_{2,b_2}^n$  (see Proposition 1).

**Guess Phase:** The adversary outputs a bit  $b' \in \{0,1\}$  as a guess for  $b \in \{0,1\}$  chosen by  $\mathcal{C}$  during the challenge phase.

- *Game  $\mathcal{G}_1$ :* This game is similar to  $\mathcal{G}_0$  except that the secret key-public key pair  $(\text{sk}^*, \text{pk}^*)$  is set as follows by  $\mathcal{C}$  in the reveal phase at epoch  $t' + 1$  by choosing a randomness  $\mu^* \xleftarrow{\$} \mathbb{F}_2^k$  and generating  $(\Pi_{\mu^*}, \mathbf{F}_{\mu^*}) = \text{Samp}(\mu^*) \in \text{Per}(\mathcal{R})$  where  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathcal{R} \times \mathcal{R}$ .

$$\text{sk}^* = (\rho^*, \gamma^*) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$$

$$\text{pk}^* = (\mathbf{h}^* = \Pi_{\mu^*}(\mathbf{h}_{t'}), \mathbf{s}^* = \rho^* + \mathbf{h}^*\gamma^*) \in \mathcal{R} \times \mathcal{R}$$

Note that in game  $\mathcal{G}_0$ ,

$$\text{sk}^* = (\mathbf{x}^* = \mathbf{F}_{\mu^*}(\mathbf{x}_{t'}), \mathbf{y}^* = \mathbf{F}_{\mu^*}(\mathbf{y}_{t'})) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2),$$

$$\text{pk}^* = (\mathbf{h}^* = \Pi_{\mu^*}(\mathbf{h}_{t'}), \mathbf{s}^* = \mathbf{x}^* + \mathbf{h}^*\mathbf{y}^*) \in \mathcal{R} \times \mathcal{R}$$

where  $\text{sk}_{t'}^{(u)} = (\mathbf{x}_{t'}, \mathbf{y}_{t'}) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$  and  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathcal{R} \times \mathcal{R}$  satisfying  $\mathbf{s}_{t'} = \mathbf{x}_{t'} + \mathbf{h}_{t'}\mathbf{y}_{t'}$ . Besides  $\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) \in \mathcal{R} \times \mathcal{R}$ ,  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathcal{R} \times \mathcal{R}$  and  $\text{sk}_0^{(u)} = (\mathbf{x}_0, \mathbf{y}_0) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$ ,  $\text{sk}_{t'}^{(u)} = (\mathbf{x}_{t'}, \mathbf{y}_{t'}) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$  in game  $\mathcal{G}_1$  are set exactly in the same way as in game  $\mathcal{G}_0$ . The  $n$  length vectors  $\mathbf{h}_0, \mathbf{h}_{t'}, \mathbf{h}^* \in \mathbb{F}_{2,b_1}^n$  in both  $\mathcal{G}_0$  and  $\mathcal{G}_1$  as they have the same parity  $b_1 = \mathbf{h}_0 \cdot \mathbf{1} \bmod 2$  by the property of  $\Pi_{\mu^*}$  by Proposition 3 for  $\mathbf{h}_0 \in \mathcal{R}$  and the  $n$  length vectors  $\mathbf{s}_0 = \mathbf{x}_0 + \mathbf{h}_0\mathbf{y}_0$ ,  $\mathbf{s}_{t'} = \mathbf{x}_{t'} + \mathbf{h}_{t'}\mathbf{y}_{t'}$ ,  $\mathbf{s}^* = \mathbf{x}^* + \mathbf{h}^*\mathbf{y}^* \in \mathbb{F}_{2,b_2}^n$  in both  $\mathcal{G}_0$  and  $\mathcal{G}_1$  (Proposition 1) as they have the same parity  $b_2 = w + b_1 \times w \bmod 2$ .

Here, we rely on computational hiding and statistical arguments. First, by the IND-CPA security of the underlying hqcPKE scheme, the update ciphertext  $\text{up}^*$  hides the true value of  $\mu^*$  from any PPT adversary. Therefore, from the adversary's perspective,  $\mu^*$  acts as a hidden uniform random variable.

Since  $\mu^*$  is hidden, the application of  $(\Pi_{\mu^*}, F_{\mu^*}) = \text{Samp}(\mu^*)$  to the previous key state acts as a random transformation over  $\text{Per}(\mathcal{R})$ . By the distributional equivalence property of  $\text{Per}(\mathcal{R})$ , the honestly derived key is statistically close to the uniformly sampled key  $(\rho^*, \gamma^*)$ , bounded by  $\varepsilon_1$ . Therefore, the adversary can only distinguish this game by either breaking the IND-CPA security of the ciphertext or detecting the statistical variance. Thus

$$|\Pr[S_0] - \Pr[S_1]| \leq \text{Adv}_{\text{hqCPKE}}^{\text{IND-CPA}}(1^\lambda) + \varepsilon_1.$$

- *Game  $\mathcal{G}_2$* : This game is identical to  $\mathcal{G}_1$  except for a change in the generation of the initial public key  $\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  during the setup. The component  $\mathbf{s}_0 = \mathbf{x}_0 + \mathbf{h}_0 \mathbf{y}_0 \in \mathbb{F}_{2,b_2}^n$  is made independent of  $\mathbf{h}_0 \in \mathbb{F}_{2,b_1}^n$  and replaced by a uniformly sampled vector  $\mathbf{s}'_0 \xleftarrow{\$} \mathbb{F}_{2,b_2}^n$ , thereby affecting the simulation of  $\text{CT}_{\mathbf{m}_b^*}$  in challenge phase and  $\text{up}^*$  in the reveal phase and  $\text{pk}_{i+1}^{(u)}, \text{up}_{i+1}^{(u)}, \text{sk}_{i+1}^{(u)}$  in the pre-challenge query and post-challenge query phase. The challenger does not set any initial secret key  $\text{sk}_0^{(u)}$  and  $(\mathbf{h}_0, \mathbf{s}'_0) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  are unrelated unlike  $\mathcal{G}_1$ . Thus in Game  $\mathcal{G}_2$ ,

$$\begin{aligned} \text{pk}_0^{(u)} &= (\mathbf{h}_0, \mathbf{s}'_0) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n, \\ \text{pk}_{i+1}^{(u)} &= (\mathbf{h}_{i+1} = \Pi_{\mu_i}(\mathbf{h}_i), \mathbf{s}'_{i+1} = F_{\mu_i}(\mathbf{s}'_i)) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n, \\ \text{up}_{i+1}^{(u)} &= \text{CT}_{\mu_i} = (\mathbf{u}_{\mu_i} = \mathbf{r}'_1 + \mathbf{h}_i \mathbf{r}'_2, \mathbf{v}_{\mu_i} = \mu_i \mathbf{G} + \mathbf{s}'_i \mathbf{r}'_2 + \mathbf{e}') \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_i^{(u)}, \mu_i; \theta') \\ \text{CT}_{\mathbf{m}_b^*} &= (\mathbf{u}_{\mathbf{m}_b^*} = \mathbf{r}_1 + \mathbf{h}_t \mathbf{r}_2, \mathbf{v}_{\mathbf{m}_b^*} = \mathbf{m}_b^* \mathbf{G} + \mathbf{s}'_t \mathbf{r}_2 + \mathbf{e}) \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_t^{(u)}, \mathbf{m}_b^*; \theta) \\ \text{up}^* &= \text{CT}_{\mu^*} = (\mathbf{u}_{\mu^*} = \mathbf{r}'_1 + \mathbf{h}_{t'} \mathbf{r}'_2, \mathbf{v}_{\mu^*} = \mu^* \mathbf{G} + \mathbf{s}'_{t'} \mathbf{r}'_2 + \mathbf{e}'') \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_{t'}^{(u)}, \mu^*; \theta'') \\ \text{sk}^* &= (\rho^*, \gamma^*) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2) \\ \text{pk}^* &= (\mathbf{h}^* = \Pi_{\mu^*}(\mathbf{h}_{t'}), \mathbf{s}^* = \rho^* + \mathbf{h}^* \gamma^*) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n, \end{aligned}$$

Here  $(\Pi_{\mu_i}, F_{\mu_i}) = \text{Samp}(\mu_i) \in \text{Per}(\mathcal{R})$ ,  $\mu_i \in \mathbb{F}_2^k$  is the choice of randomness of adversary  $\mathcal{A}$  for  $i \in \{1, 2, \dots, t-1\}$ ,  $\text{pk}_i^{(u)} = (\mathbf{h}_i, \mathbf{s}'_i) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  denotes the public key at epoch  $i$ ,  $\text{P}(\theta') = (\mathbf{e}', \mathbf{r}'_1, \mathbf{r}'_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\theta' \xleftarrow{\$} \mathbb{F}_2^\alpha$ ,  $\text{pk}_t^{(u)} = (\mathbf{h}_t, \mathbf{s}'_t) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$ ,  $\text{P}(\theta) = (\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\theta \xleftarrow{\$} \mathbb{F}_2^\alpha$ .  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  is the public key at epoch  $t'$ ,  $\text{P}(\theta'') = (\mathbf{e}'', \mathbf{r}''_1, \mathbf{r}''_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\theta'' \xleftarrow{\$} \mathbb{F}_2^\alpha$ .  $(\Pi_{\mu^*}, F_{\mu^*}) = \text{Samp}(\mu^*) \in \text{Per}(\mathcal{R})$  where  $\mu^* \xleftarrow{\$} \mathbb{F}_2^k$  is the randomness chosen by the challenger  $\mathcal{C}$ .

**Claim 1:**  $|\Pr[S_1] - \Pr[S_2]| < \varepsilon_2$  for some negligible function  $\varepsilon_2$ .

- *Game  $\mathcal{G}_3$* : This game proceeds in the same way as before except for a change in  $\text{up}_{i+1}^{(u)}$  during the pre-challenge and post-challenge query phase.

Instead of using  $\mathcal{A}$ 's choice of randomness  $\mu_i \in \mathbb{F}_2^k$  in generating  $\text{up}_{i+1}^{(u)}$  for  $i = \{0, 1, 2, \dots, t-1\}$  and invoking  $\text{uhqcPKE.Enc}$ , the challenger  $\mathcal{D}$  samples randomly weighted vectors  $\hat{\mathbf{r}}'_1, \hat{\mathbf{r}}'_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\hat{\mathbf{e}}' \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$ , computes

$$\text{up}_{i+1}^{(u)} = (\hat{\mathbf{r}}'_1 + \mathbf{h}_i \hat{\mathbf{r}}'_2, \mu_i \mathbf{G} + \mathbf{s}'_i \hat{\mathbf{r}}'_2 + \hat{\mathbf{e}}') \text{ and sends it } \mathcal{A}.$$

**Claim 2:**  $|\Pr[S_2] - \Pr[S_3]| < \varepsilon_3$  for some negligible function  $\varepsilon_3$ .

- *Game  $\mathcal{G}_4$*  : This game is the same as the previous game  $\mathcal{G}_3$  except for one change in the challenge phase. The challenge ciphertext is set as

$$\text{CT}_{\mathbf{m}_b^*} = (\mathbf{u}_{\mathbf{m}_b^*} = \hat{\mathbf{r}}_1 + \mathbf{h}_t \hat{\mathbf{r}}_2, \mathbf{v}_{\mathbf{m}_b^*} = \mathbf{m}_b^* \mathbf{G} + \mathbf{s}'_t \hat{\mathbf{r}}_2 + \hat{\mathbf{e}})$$

by randomly sampling  $\hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\hat{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$  where  $\text{pk}_t^{(u)} = (\mathbf{h}_t, \mathbf{s}_t) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  as in  $\mathcal{G}_3$ . The challenge ciphertext is  $\mathcal{G}_3$  is

$$\text{CT}_{\mathbf{m}_b^*} \leftarrow \text{uhqcPKE.Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_t^{(u)}, \mathbf{m}_b^*; \theta)$$

where  $\text{CT}_{\mathbf{m}_b^*} = (\mathbf{u}_{\mathbf{m}_b^*} = \mathbf{r}_1 + \mathbf{h}_t \mathbf{r}_2, \mathbf{v}_{\mathbf{m}_b^*} = \mathbf{m}_b^* \mathbf{G} + \mathbf{s}'_t \mathbf{r}_2 + \mathbf{e})$  and  $(\mathbf{r}_1, \mathbf{r}_2, \mathbf{e}) \in \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  is generated using randomness generator  $\mathbf{P}$  on input  $\theta \xleftarrow{\$} \mathbb{F}_2^\alpha$  sampled during the invocation of the encryption algorithm.

**Claim 3:**  $|\Pr[S_3] - \Pr[S_4]| < \varepsilon_3$  for some negligible function  $\varepsilon_3$ .

- *Game  $\mathcal{G}_5$* : This game proceeds as before except for a change in the  $\text{up}^*$  component in the reveal phase. Here, we replace the correctly weighted vector  $\mathbf{r}_1'', \mathbf{r}_2''$  and  $\mathbf{e}''$  with random vectors  $\hat{\mathbf{r}}_1'', \hat{\mathbf{r}}_2'' \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\hat{\mathbf{e}}'' \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$ . Thus,
- $$\text{up}^* = (\mathbf{u}^* = \hat{\mathbf{r}}_1' + \mathbf{h}_{t'} \hat{\mathbf{r}}_2', \mathbf{v}^* = \mu^* \mathbf{G} + \mathbf{s}'_{t'} \hat{\mathbf{r}}_2' + \hat{\mathbf{e}}').$$

**Claim 4:**  $|\Pr[S_4] - \Pr[S_5]| = \text{Adv}_{\mathcal{D}}^{3\text{-DQCDP}}(1^\lambda) < \varepsilon_3$  for some negligible function  $\varepsilon_3$ .

In game  $\mathcal{G}_5$  all values revealed to the adversary are uniformly randomized. In particular, the challenge ciphertext  $\text{CT}_{\mathbf{m}_b^*}$  is statistically independent of the message bit  $b$  as  $\mathbf{u}_{\mathbf{m}_b^*}$  is fully independent of the bit  $b$  and  $\mathbf{v}_{\mathbf{m}_b^*}$  is masked by a random vector  $\mathbf{s}'_{t'} \hat{\mathbf{r}}_2' + \hat{\mathbf{e}}'$ . Hence,  $\Pr[S_5] = \frac{1}{2}$ .

Thus, summing up all the results we get

$$\begin{aligned} |\Pr[S_0] - \Pr[S_5]| &\leq |\Pr[S_0] - \Pr[S_1]| + |\Pr[S_1] - \Pr[S_2]| \\ &\quad + |\Pr[S_2] - \Pr[S_3]| + |\Pr[S_3] - \Pr[S_4]| \\ &\quad + |\Pr[S_4] - \Pr[S_5]| \end{aligned}$$

Therefore,

$$|\Pr[S_0] - \frac{1}{2}| < \text{Adv}_{\text{hqcPKE}}^{\text{IND-CPA}}(1^\lambda) + \varepsilon_1 + \varepsilon_2 + 3\varepsilon_3 = \varepsilon$$

where  $\text{Adv}_{\text{hqcPKE}}^{\text{IND-CPA}}(1^\lambda) + \varepsilon_1 + \varepsilon_2 + 3\varepsilon_3 = \varepsilon$ . Therefore,

$$\text{Adv}_{\text{uhqcPKE}, \mathcal{A}}^{\text{IND-CR-CPA}}(1^\lambda) = |\Pr[\text{EXP}_{\text{uhqcPKE}, \mathcal{A}}^{\text{IND-CR-CPA}}(1^\lambda) = 1] - \frac{1}{2}| < \varepsilon$$

Hence, the theorem follows.

#### 4.1 Proof of Claims

• *Proof of Claim 1:* We show that if the adversary  $\mathcal{A}$  can distinguish between the games  $\mathcal{G}_1$  and  $\mathcal{G}_2$ , then one can construct a distinguisher  $\mathcal{D}$  for the 2-DQCSDP( $n, w, b_1, b_2$ ) (see Definition 13) problem. Let the 2-DQCSDP( $n, w, b_1, b_2$ ) challenger  $\mathcal{B}$  provides an instance  $\text{Ins} = (\mathbf{H}, \mathbf{s}) \in \mathbb{F}_{2, b_1}^{n \times 2n} \times \mathbb{F}_{2, b_2}^n$  of the 2-DQCSDP( $n, w, b_1, b_2$ ) problem to the distinguisher  $\mathcal{D}$  where  $\mathbf{H} = (\mathbf{I}_n \text{ rot}(\mathbf{h}))$ ,  $b_1 = \mathbf{h} \cdot \mathbf{1} \bmod 2$ ,  $\mathbf{h} \in \mathbb{F}_{2, b_1}^n$ ,  $b_2 = w + b_1 \times w \bmod 2$  and  $\mathbf{s}$  is either a random element from  $\mathbb{F}_{2, b_2}^n$  or of the form  $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y}$  for some  $\mathbf{x}, \mathbf{y} \in \mathcal{S}_w^n(\mathbb{F}_2)$ . The distinguisher  $\mathcal{D}$  plays the role of the challenger to  $\mathcal{A}$  and the simulates  $\mathcal{A}'$ 's view as follows.

– **Setup:** The distinguisher  $\mathcal{D}$  sets

$$\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_e, w_r, \text{Samp}, \mathbf{G}, \mathbf{P})$$

by choosing a  $\tau$  error correcting decodable  $[n, k]$  linear code  $\mathbf{C}$  with a generator matrix  $\mathbf{G}$ , a positive integer a size  $\alpha$  of encryption randomness, fixed weights  $w, w_e, w_r$ , a randomness generator  $\mathbf{P} : \{0, 1\}^* \rightarrow \mathcal{S}_{w_e}^n(\mathbb{F}_2^n) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2^n) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2^n)$  and a sampling function  $\text{Samp} : \mathbb{F}_2^k \rightarrow \text{Per}(\mathcal{R})$ . It extracts  $\mathbf{h}, \mathbf{s}$  from the given instance  $\text{Ins} = (\mathbf{H}, \mathbf{s})$ , sets

$$\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) = (\mathbf{h}, \mathbf{s}) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n$$

and forwards  $\text{pp}_{\text{uhqcPKE}}, \text{pk}_0^{(u)}$  to  $\mathcal{A}$ .

– **Pre-Challenge Query Phase:** The adversary  $\mathcal{A}$  is allowed  $\mathcal{O}_{\text{upd}}^{\text{uhqcPKE}}(\cdot)$  queries as described in the original game. On receiving a query  $\boldsymbol{\mu}_i \in \mathbb{F}_2^k$  for  $i \in \{0, 1, 2, \dots, t-1\}$  from  $\mathcal{A}$ , the challenger  $\mathcal{D}$  generates

$$(\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)}) \leftarrow \text{uhqcPKE.PkUpdate}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_i^{(u)}, \boldsymbol{\mu}_i)$$

and forwards it to  $\mathcal{A}$  where

$$\text{pk}_{i+1}^{(u)} = (\mathbf{h}_{i+1} = \Pi_{\boldsymbol{\mu}_i}(\mathbf{h}_i), \mathbf{s}_{i+1} = \mathbf{F}_{\boldsymbol{\mu}_i}(\mathbf{s}_i)) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n,$$

$$\text{up}_{i+1}^{(u)} = \text{CT}_{\mu_i} \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_i^{(u)}, \mu_i; \theta')$$

where  $\text{CT}_{\mu_i} = (\mathbf{u}_{\mu_i} = \mathbf{r}'_1 + \mathbf{h}_i \mathbf{r}'_2, \mathbf{v}_{\mu_i} = \mu_i \mathbf{G} + \mathbf{s}_i \mathbf{r}'_2 + \mathbf{e}')$  and  $(\Pi_{\mu_i}, \mathbf{F}_{\mu_i}) = \text{Samp}(\mu_i) \in \text{Per}(\mathcal{R})$ ,  $\theta' \in \mathbb{F}_2^\alpha$ ,  $\mathbf{P}(\theta') = (\mathbf{e}', \mathbf{r}'_1, \mathbf{r}'_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2^n) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2^n) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2^n)$  and  $\text{pk}_i^{(u)} = (\mathbf{h}_i, \mathbf{s}_i) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  for  $i \in \{1, 2, \dots, t-1\}$ .

– **Challenge Phase:** On receiving a pair of challenge messages  $(\mathbf{m}_0^*, \mathbf{m}_1^*) \in \mathbb{F}_2^k \times \mathbb{F}_2^k$  from  $\mathcal{A}$  at epoch  $t$ , the distinguisher  $\mathcal{D}$  samples a bit  $b \xleftarrow{\$} \{0, 1\}$ , computes

$$\text{CT}_{\mathbf{m}_b^*} \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_t^{(u)}, \mathbf{m}_b^*; \theta)$$

where  $\text{CT}_{\mathbf{m}_b^*} = (\mathbf{u}_{\mathbf{m}_b^*} = \mathbf{r}_1 + \mathbf{h}_t \mathbf{r}_2, \mathbf{v}_{\mathbf{m}_b^*} = \mathbf{m}_b^* \mathbf{G} + \mathbf{s}_t \mathbf{r}_2 + \mathbf{e})$  and forwards it to  $\mathcal{A}$  where  $\mathbf{P}(\theta) = (\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\theta \in \mathbb{F}_2^\alpha$  and  $\text{pk}_t^{(u)} = (\mathbf{h}_t, \mathbf{s}_t) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$ .

– **Post-Challenge Query Phase:** The adversary  $\mathcal{A}$  is allowed to make  $\mathcal{O}_{\text{upd}}^{\text{uhqcPKE}}(\mu_i)$  queries for  $i \in \{t+1, \dots, t'-1\}$  which is described and simulated in the pre-challenge query phase for  $\mu_i \in \mathbb{F}_2^k$ .

– **Reveal Phase:** The distinguisher  $\mathcal{D}$  samples a  $k$ -bit string  $\mu^* \xleftarrow{\$} \mathbb{F}_2^k$  at epoch  $t'$  and picks

$$\text{sk}^* = (\rho^*, \gamma^*) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$$

independent of  $\mu^*$ , computes

$$\text{up}^* = \text{CT}_{\mu^*} \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_{t'}^{(u)}, \mu^*; \theta'')$$

$$\text{pk}^* = (\mathbf{h}^* = \Pi_{\mu^*}(\mathbf{h}_{t'}), \mathbf{s}^* = \rho^* + \mathbf{h}^* \gamma^*) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$$

where  $\text{CT}_{\mu^*} = (\mathbf{u}_{\mu^*} = \mathbf{r}_1'' + \mathbf{h}_{t'} \mathbf{r}_2'', \mathbf{v}_{\mu^*} = \mu^* \mathbf{G} + \mathbf{s}_{t'} \mathbf{r}_2'' + \mathbf{e}'')$  and forwards the tuple  $(\text{pk}^*, \text{sk}^*, \text{up}^*)$  to  $\mathcal{A}$  where  $\theta'' \in \mathbb{F}_2^\alpha$ ,  $\mathbf{P}(\theta'') = (\mathbf{e}'', \mathbf{r}_1'', \mathbf{r}_2'') \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$ ,  $(\Pi_{\mu^*}, \mathbf{F}_{\mu^*}) = \text{Samp}(\mu^*) \in \text{Per}(\mathcal{R})$ .

– **Guess Phase:** After receiving a guess bit  $b'$  from  $\mathcal{A}$  for bit  $b$  chosen by  $\mathcal{D}$  in the challenge phase,  $\mathcal{D}$  forwards  $\beta$ ,  $\beta = 0$  if  $b' \neq b$  and  $\beta = 1$  if  $b' = b$ .

Observe that if the public key component  $\mathbf{s}_i \in \mathbb{F}_{2,b_2}^n$  is sampled from the  $2\text{-QCSDP}(n, w, b_1, b_2)$  distribution then  $\mathbf{s}_i$  is of the form  $\mathbf{s}_i = \mathbf{x}_i + \mathbf{h}_i \mathbf{y}_i$  for some unknown vectors  $\mathbf{x}_i, \mathbf{y}_i \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$  which implies that  $\mathbf{s}_{i+1} \in \mathbb{F}_{2,b_2}^n$  belongs to the  $2\text{-QCSDP}(n, w, b_1, b_2)$  distribution as  $\mathbf{s}_{i+1} = \mathbf{F}_{\mu_i}(\mathbf{s}_i) = \mathbf{F}_{\mu_i}(\mathbf{x}_i + \mathbf{h}_i \mathbf{y}_i) = \mathbf{F}_{\mu_i}(\mathbf{x}_i) + \Pi_{\mu_i}(\mathbf{h}_i) \mathbf{F}_{\mu_i}(\mathbf{y}_i) = \mathbf{x}_{i+1} + \mathbf{h}_{i+1} \mathbf{y}_{i+1} \in \mathbb{F}_{2,b_2}^n$  where  $\mathbf{x}_{i+1}, \mathbf{y}_{i+1} \in \mathcal{S}_w^n(\mathbb{F}_2)$  for all  $i \in \{1, 2, \dots, t'\}$ . On the other hand, if  $\mathbf{s}_i \in \mathbb{F}_{2,b_2}^n$  is sampled from the uniform distribution over  $\mathbb{F}_{2,b_2}^n$  then  $\mathbf{s}_{i+1} \in \mathbb{F}_{2,b_2}^n$  follows

the uniform distribution over  $\mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  for all  $i \in \{1, 2, \dots, t'\}$ .

In the setup phase, the public parameters  $\mathbf{pp}_{\text{uhqcPKE}}$  and the initial public key  $\mathbf{pk}_0^{(u)} \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  are identically distributed in  $\mathcal{G}_1$  and  $\mathcal{G}_2$ . During the pre-challenge query phase and post challenge query phase, the generated update ciphertext  $\mathbf{up}_{i+1}^{(u)} \in \mathcal{R} \times \mathcal{R}$  and updated public key  $\mathbf{pk}_{i+1}^{(u)} \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  follow the same distribution in both the games. Moreover, the distribution of the challenge ciphertext  $\mathbf{CT}_{\mathbf{m}_b^*} \in \mathcal{R} \times \mathcal{R}$  in the challenge phase are identical in  $\mathcal{G}_1$  and  $\mathcal{G}_2$  and the simulation of  $(\mathbf{pk}^*, \mathbf{sk}^*, \mathbf{up}^*)$  in the reveal phase. Therefore, the adversary's view to the distributions are same in both the games which leads to the following two cases.

**Case  $\beta = 0$ :** If  $\mathbf{Ins} = (\mathbf{H}, \mathbf{s} = \mathbf{s}')$  for  $\mathbf{s}' \xleftarrow{\$} \mathbb{F}_{2,b_2}^n$ , then  $\mathcal{D}$  perfectly simulates game  $\mathcal{G}_2$  and therefore

$$\Pr \left[ \mathcal{D}(\mathbf{H}, \mathbf{s}) = 1 \mid \mathbf{H} = (\mathbf{I}_n \text{ rot}(\mathbf{h})), \mathbf{s} = \mathbf{s}' \xleftarrow{\$} \mathbb{F}_{2,b_2}^n \right] = \Pr[S_2].$$

**Case  $\beta = 1$ :** If  $\mathbf{Ins} = (\mathbf{H}, \mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y})$  for  $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y} \in \mathbb{F}_{2,b_2}^n$  with some  $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$ , then  $\mathcal{D}$  simulates game  $\mathcal{G}_1$  and hence,

$$\Pr \left[ \mathcal{D}(\mathbf{H}, \mathbf{s}) = 1 \mid \mathbf{H} = (\mathbf{I}_n \text{ rot}(\mathbf{h})), \mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y}, \mathbf{x}, \mathbf{y} \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \right] = \Pr[S_1].$$

Therefore, the advantage of distinguishing game  $\mathcal{G}_1$  and game  $\mathcal{G}_2$  is the same as the advantage of breaking the 2-DQCSDP( $n, w, b_1, b_2$ ) problem. Thus,  $|\Pr[S_1] - \Pr[S_2]| = \text{Adv}_{\mathcal{D}}^{2\text{-DQCSDP}}(1^\lambda) < \varepsilon_2$  by the 2-DQCSDP( $n, w, b_1, b_2$ ) assumption for some negligible function  $\varepsilon_2$ .

• *Proof of Claim 2:* We show that if there exists an adversary  $\mathcal{A}$  who can distinguish between the games  $\mathcal{G}_2$  and  $\mathcal{G}_3$  then one can construct a distinguisher  $\mathcal{D}$  for the 3-DQCSDP( $n, w_e, w_r, b_1, b_2$ ) problem (see Definition 15). Let  $\mathcal{B}$  be the 3-DQCSDP( $n, w_e, w_r, b_1, b_2$ ) challenger who provides an instance  $\mathbf{Ins} = (\mathbf{H}, \mathbf{a} = (\mathbf{a}_1, \mathbf{a}_2)) \in \mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n)$  of the 3-DQCSDP( $n, w_e, w_r, b_1, b_2$ ) problem to the distinguisher  $\mathcal{D}$  where

$$\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, \mathbf{h} \in \mathbb{F}_{2,b_1}^n, \mathbf{s} \in \mathbb{F}_{2,b_2}^n,$$

$$b_1 = \mathbf{h} \cdot \mathbf{1} \bmod 2, b_2 = w + b_1 w \bmod 2, b_3 = w_r + b_1 w_r \bmod 2,$$

$$b'_3 = w_e + b_2 w_r \bmod 2, \mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n,$$

and  $(\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2, \tilde{\mathbf{e}})$  is either a random tuple from  $\mathbb{F}_{2,w_r}^n \times \mathbb{F}_{2,w_r}^n \times \mathbb{F}_{2,w_e}^n$  or from  $\mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_e}^n(\mathbb{F}_2)$ . The distinguisher  $\mathcal{D}$  acts as a challenger to  $\mathcal{A}$  and simulates its views as follows.

– **Setup:** The distinguisher  $\mathcal{D}$  chooses a  $\tau$  error correcting decodable  $[n, k]$

linear code  $\mathbf{C}$  with a generator matrix is  $\mathbf{G}$ , a size  $\alpha$  of encryption randomness, fixed weights  $w, w_e, w_r$ , a randomness generator  $\mathbf{P} : \{0, 1\}^* \rightarrow \mathcal{S}_{w_e}^n(\mathbb{F}_2^n) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2^n) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2^n)$ , a sampling function  $\mathbf{Samp} : \mathbb{F}_2^k \rightarrow \text{Per}(\mathcal{R})$  and sets the public parameter as

$$\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_e, w_r, \mathbf{Samp}, \mathbf{G}, \mathbf{P}).$$

It extracts  $\mathbf{h}, \mathbf{s}$  from  $\mathbf{H}$  in the given instance  $(\mathbf{H}, \mathbf{a})$ , sets

$$\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) = (\mathbf{h}, \mathbf{s}) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n$$

and forwards  $\text{pp}_{\text{uhqcPKE}}, \text{pk}_0^{(u)}$  to  $\mathcal{A}$ . The distinguisher sets  $\mathbf{a}_{1,0} = \mathbf{a}_1$  and  $\mathbf{a}_{2,0} = \mathbf{a}_2$  extracting  $\mathbf{a} = (\mathbf{a}_1, \mathbf{a}_2) \in \mathbb{F}_{2, b_3}^n \times \mathbb{F}_{2, b'_3}^n$  from the given instance  $\text{Ins} = (\mathbf{H}, \mathbf{a})$ .  $\mathcal{D}$  initializes a list  $\mathbf{a\_List} = \mathbf{a\_List} \cup \{\mathbf{a}_{1,0}, \mathbf{a}_{2,0}\}$ .

– **Pre-Challenge Query Phase:** On receiving a query  $\mu_i \in \mathbb{F}_2^k$  for  $i = \{0, 1, 2, \dots, t-1\}$  from  $\mathcal{A}$ , the distinguisher  $\mathcal{D}$  updates the list  $\mathbf{a\_List} = \mathbf{a\_List} \cup \{\mathbf{a}_{1,i}, \mathbf{a}_{2,i}\}$  and computes

$$\mathbf{a}_{1,i+1} = \Pi_{\mu_i}(\mathbf{a}_{1,i}), \quad \mathbf{a}_{2,i+1} = \mathbf{F}_{\mu_i}(\mathbf{a}_{2,i}),$$

$$\boxed{\text{up}_{i+1}^{(u)} = (\mathbf{u}_{\mu_i}, \mathbf{v}_{\mu_i}) = (\mathbf{a}_{1,i}, \mu_i \mathbf{G} + \mathbf{a}_{2,i}) \in \mathcal{R} \times \mathcal{R},}$$

$$\text{pk}_{i+1}^{(u)} = (\Pi_{\mu_i}(\mathbf{h}_i), \mathbf{F}_{\mu_i}(\mathbf{s}_i)) = (\mathbf{h}_{i+1}, \mathbf{s}_{i+1}) \in \mathcal{R} \times \mathcal{R}$$

and forwards  $(\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)})$  to  $\mathcal{A}$  where  $\text{pk}_i^{(u)} = (\mathbf{h}_i, \mathbf{s}_i) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n$ ,  $(\Pi_{\mu_i}, \mathbf{F}_{\mu_i}) = \mathbf{Samp}(\mu_i) \in \text{Per}(\mathcal{R})$ . Note that  $\mathbf{a}_{1,i}, \mathbf{a}_{2,i}$  are the updated instance component at epoch  $i$  and  $\mathbf{a}_{1,0} = \mathbf{a}_1, \mathbf{a}_{2,0} = \mathbf{a}_2$ .

– **Challenge Phase:** On receiving challenge message pair  $(\mathbf{m}_0^*, \mathbf{m}_1^*) \in \mathbb{F}_2^k \times \mathbb{F}_2^k$  from  $\mathcal{A}$  at epoch  $t$ , the distinguisher  $\mathcal{D}$  samples a bit  $b \xleftarrow{\$} \{0, 1\}$ , computes

$$\text{CT}_{\mathbf{m}_b^*} \leftarrow \text{uhqcPKE.Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_t^{(u)}, \mathbf{m}_b^*; \theta),$$

where  $\text{CT}_{\mathbf{m}_b^*} = (\mathbf{u}_{\mathbf{m}_b^*} = \mathbf{r}_1 + \mathbf{h}_t \mathbf{r}_2, \mathbf{v}_{\mathbf{m}_b^*} = \mathbf{m}_b^* \mathbf{G} + \mathbf{s}_t \mathbf{r}_2 + \mathbf{e})$  and forwards it to  $\mathcal{A}$  where  $\theta \in \mathbb{F}_2^\alpha$ ,  $\mathbf{P}(\theta) = (\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$  and  $\text{pk}_t^{(u)} = (\mathbf{h}_t, \mathbf{s}_t) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n$ .

– **Post-Challenge Query Phase:** The adversary  $\mathcal{A}$  can make  $\mathcal{O}_{\text{upd}}^{\text{uhqcPKE}}(\mu_i)$  queries for  $i = \{t, t+1, \dots, t'-1\}$  which are simulated in the same manner by  $\mathcal{D}$  as described in the pre-challenge query phase for  $\mu_i \in \mathbb{F}_2^k$ .

– **Reveal Phase:** The distinguisher  $\mathcal{D}$  samples  $\mu^* \xleftarrow{\$} \mathbb{F}_2^k$  at epoch  $t' + 1$  and selects

$$\begin{aligned} \text{sk}^* &= (\rho^*, \gamma^*) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2) \\ \text{pk}^* &= (\mathbf{h}^* = \Pi_{\mu^*}(\mathbf{h}_{t'}), \mathbf{s}^* = \rho^* + \mathbf{h}^* \gamma^*) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n \end{aligned}$$

$$\text{up}^* = \text{CT}_{\mu^*} \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_{t'}^{(u)}, \mu^*; \theta''),$$

where  $\text{CT}_{\mu^*} = (\mathbf{u}_{\mu^*} = \mathbf{r}_1'' + \mathbf{h}_{t'} \mathbf{r}_2'', \mathbf{v}_{\mu^*} = \mu^* \mathbf{G} + \mathbf{s}_{t'} \mathbf{r}_2'' + \mathbf{e}'')$  and forwards  $(\text{pk}^*, \text{sk}^*, \text{up}^*)$  to the adversary  $\mathcal{A}$  where  $(\Pi_{\mu^*}, F_{\mu^*}) = \text{Samp}(\mu^*) \in \text{Per}(\mathcal{R})$ ,  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathcal{R} \times \mathcal{R}$ ,  $\theta'' \xleftarrow{\$} \mathbb{F}_2^\alpha$ ,  $P(\theta'') = (\mathbf{e}'', \mathbf{r}_1'', \mathbf{r}_2'') \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ .

– **Guess Phase:** After receiving a guess bit  $b'$  from  $\mathcal{A}$  for bit  $b$  chosen by  $\mathcal{D}$  in the challenge phase,  $\mathcal{D}$  forwards to  $\mathcal{B}$ ,  $\beta = 0$  if  $b' \neq b$  and  $\beta = 1$  if  $b' = b$ .

Note that  $\mathcal{D}$  faithfully simulates the experiment using the given 3-DQCSDP( $n, w_e, w_r, b_1, b_2$ ) instance. In the setup phase, the generated public parameters  $\text{pp}_{\text{uhqcPKE}}$  and the initial public key  $\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  each have identical distribution in game  $\mathcal{G}_2$  and game  $\mathcal{G}_3$ . During the pre-challenge query phase and post-challenge query phase, the update ciphertext  $\text{up}_{i+1}^{(u)} \in \mathcal{R} \times \mathcal{R}$  and updated public key  $\text{pk}_{i+1}^{(u)} = (\mathbf{h}_{i+1}, \mathbf{s}_{i+1}) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  are simulated following to the same distribution in both the games. Likewise, the challenge ciphertext  $\text{CT}_{\mathbf{m}_b^*} \in \mathcal{R} \times \mathcal{R}$  in the challenge phase as the simulation of  $(\text{pk}^*, \text{sk}^*, \text{up}^*)$  in the reveal phase are identically distributed in game  $\mathcal{G}_2$  and game  $\mathcal{G}_3$ . Consequently, the adversary's view to the distributions are same in both games which yields the following two cases.

**Case  $\beta = 0$ :** If  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  for  $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$ ,  $\mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n$ ,  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$  then the simulation follows as in game  $\mathcal{G}_3$  and therefore,

$$\begin{aligned} \Pr[\mathcal{D}(\mathbf{H}, \mathbf{a}) = 1 \mid \mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, \\ \mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n, \tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n, \tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n] \\ = \Pr[S_3]. \end{aligned}$$

**Case  $\beta = 1$ :** If  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  for  $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$ ,  $\mathbf{a} = (\tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}})$ ,  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  then the simulation follows as in game  $\mathcal{G}_2$  and therefore

$$\begin{aligned} \Pr[\mathcal{D}(\mathbf{H}, \mathbf{a}) = 1 \mid \mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, \\ \mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n, \tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2), \tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)] \\ = \Pr[S_2]. \end{aligned}$$

Note that

$$\begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix} (\tilde{\mathbf{r}}_1, \tilde{\mathbf{e}}, \tilde{\mathbf{r}}_2)^T = (\tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \tilde{\mathbf{e}} + \mathbf{s}\tilde{\mathbf{r}}_2)^T = (\mathbf{a}_1, \mathbf{a}_2)^T.$$

If the instance  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  is sampled from the  $3\text{-QCSDP}(n, w_e, w_r, b_1, b_2)$  distribution then  $\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2$ ,  $\mathbf{a}_2 = \tilde{\mathbf{s}}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}$  for some  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  and therefore

$$\begin{aligned} (\mathbf{u}_{\mu_i}, \mathbf{v}_{\mu_i} - \mu_i \mathbf{G}) &= (\mathbf{a}_{1,i} = \Pi_{\mu_{i-1}}(\mathbf{a}_{1,i-1}), \mathbf{a}_{2,i} = \mathbf{F}_{\mu_{i-1}}(\mathbf{a}_{2,i-1})) \\ &= (\tilde{\mathbf{r}}'_1 + \mathbf{h}_i \tilde{\mathbf{r}}'_2, \tilde{\mathbf{e}}' + \mathbf{s}_i \tilde{\mathbf{r}}'_2) \end{aligned}$$

for some  $\tilde{\mathbf{r}}'_1, \tilde{\mathbf{r}}'_2 \in \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}}' \in \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  by the Proposition 3 and Theorem 5 where  $\mathbf{a}_{1,i-1}$ ,  $\mathbf{a}_{2,i-1}$  the updated instance components formed by extracting  $\mathbf{a}_1 = \mathbf{a}_{1,0}$ ,  $\mathbf{a}_2 = \mathbf{a}_{2,0}$  from the instance and  $(\Pi_{\mu_{i-1}}, \mathbf{F}_{\mu_{i-1}}) = \text{Samp}(\mu_{i-1}) \in \text{Per}(\mathcal{R})$ . Then

$$\text{up}_{i+1}^{(u)} = \text{CT}_{\mu_i} = (\mathbf{a}_{1,i}, \mu_i \mathbf{G} + \mathbf{a}_{2,i}) = (\mathbf{u}_{\mu_i}, \mathbf{v}_{\mu_i} = \tilde{\mathbf{r}}'_1 + \mathbf{h}_i \tilde{\mathbf{r}}'_2, \mathbf{v}_{\mu_i} = \mu_i \mathbf{G} + \mathbf{s}_i \tilde{\mathbf{r}}'_2 + \tilde{\mathbf{e}}').$$

Hence the simulation of  $\text{up}_{i+1}^{(u)}$  is follows the same distribution as that in  $\mathcal{G}_2$ . On the other hand if the instance  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  is sampled from the uniform distribution over  $\mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n)$  then  $\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2$ ,  $\mathbf{a}_2 = \tilde{\mathbf{s}}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}$  for some  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$  and therefore

$$\begin{aligned} (\mathbf{u}_{\mu_i}, \mathbf{v}_{\mu_i} - \mu_i \mathbf{G}) &= (\mathbf{a}_{1,i} = \Pi_{\mu_{i-1}}(\mathbf{a}_{1,i-1}), \mathbf{a}_{2,i} = \mathbf{F}_{\mu_{i-1}}(\mathbf{a}_{2,i-1})) \\ &= (\tilde{\mathbf{r}}'_1 + \mathbf{h}_i \tilde{\mathbf{r}}'_2, \tilde{\mathbf{e}}' + \mathbf{s}_i \tilde{\mathbf{r}}'_2) \end{aligned}$$

for some  $\tilde{\mathbf{r}}'_1, \tilde{\mathbf{r}}'_2 \in \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}}' \in \mathbb{F}_{2,w_e}^n$  by the Proposition 3 and Theorem 5 where  $\mathbf{a}_{1,i-1}$ ,  $\mathbf{a}_{2,i-1}$  the updated instance components formed by extracted  $\mathbf{a}_1 = \mathbf{a}_{1,0}$ ,  $\mathbf{a}_2 = \mathbf{a}_{2,0}$  from the instance and  $(\Pi_{\mu_{i-1}}, \mathbf{F}_{\mu_{i-1}}) = \text{Samp}(\mu_{i-1}) \in \text{Per}(\mathcal{R})$ . Then

$$\text{up}_{i+1}^{(u)} = \text{CT}_{\mu_i} = (\mathbf{a}_{1,i}, \mu_i \mathbf{G} + \mathbf{a}_{2,i}) = (\mathbf{u}_{\mu_i}, \mathbf{v}_{\mu_i} = \tilde{\mathbf{r}}'_1 + \mathbf{h}_i \tilde{\mathbf{r}}'_2, \mathbf{v}_{\mu_i} = \mu_i \mathbf{G} + \mathbf{s}_i \tilde{\mathbf{r}}'_2 + \tilde{\mathbf{e}}').$$

Hence the simulation of  $\text{up}_{i+1}^{(u)}$  is follows the same distribution as in  $\mathcal{G}_3$ . Thus the probability of distinguishing game  $\mathcal{G}_2$  and game  $\mathcal{G}_3$  is same as the advantage of breaking the  $3\text{-DQCSDP}(n, w_e, w_r, b_1, b_2)$  problem. Hence,  $|\Pr[S_2] - \Pr[S_3]| = \text{Adv}_{\mathcal{D}}^{3\text{-DQCSDP}}(1^\lambda) < \varepsilon_3$  by the  $3\text{-DQCSDP}(n, w_e, w_r, b_1, b_2)$  assumption for some negligible function  $\varepsilon_3$ .

• *Proof of Claim 3:* We show that if there exists an adversary  $\mathcal{A}$  who can distinguish between the games  $\mathcal{G}_3$  and  $\mathcal{G}_4$  with advantage  $\varepsilon_3$  then one can build a distinguisher  $\mathcal{D}$  for the  $3\text{-DQCSDP}(n, w_e, w_r, b_1, b_2)$  problem with advantage  $\varepsilon_3$ . Let  $\mathcal{B}$  be the  $3\text{-DQCSDP}(n, w_e, w_r, b_1, b_2)$  challenger who provides an  $3\text{-DQCSDP}(n, w_e, w_r, b_1, b_2)$  instance  $\text{Ins} = (\mathbf{H}, \mathbf{a} = (\mathbf{a}_1, \mathbf{a}_2)) \in$

$\mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times \left( \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n \right)$  to the distinguisher  $\mathcal{D}$  where

$$\begin{aligned} \mathbf{H} &= \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, \quad \mathbf{h} \in \mathbb{F}_{2,b_1}^n, \quad \mathbf{s} \in \mathbb{F}_{2,b_2}^n, \quad b_1 = \mathbf{h} \cdot \mathbf{1} \bmod 2, \\ b_2 &= w + b_1 w \bmod 2, \quad b_3 = w_r + b_1 w_r \bmod 2, \quad b'_3 = w_e + b_2 w_r \bmod 2, \\ \mathbf{a} &= (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \quad \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n, \end{aligned}$$

and  $(\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2, \tilde{\mathbf{e}})$  is either a random tuple from  $\mathbb{F}_{2,w_r}^n \times \mathbb{F}_{2,w_r}^n \times \mathbb{F}_{2,w_e}^n$  or from  $\mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_e}^n(\mathbb{F}_2)$ . The distinguisher  $\mathcal{D}$  acts as a challenger to  $\mathcal{A}$  and simulates its views as follows.

– **Setup:** The distinguisher  $\mathcal{D}$  chooses a  $\tau$  error correcting decodable  $[n, k]$  linear code  $\mathcal{C}$  with generator matrix  $\mathbf{G}$ . It also chooses an encryption randomness size  $\alpha$ , fixed weights  $w, w_e, w_r$  along with  $\mathbf{P} : \{0, 1\}^* \rightarrow \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$  a randomness generator and a sampling function  $\text{Samp} : \mathbb{F}_2^k \rightarrow \text{Per}(\mathcal{R})$  and sets the public parameter

$$\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_e, w_r, \text{Samp}, \mathbf{G}, \mathbf{P}).$$

It sets

$$\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) = (\mathbf{h}, \mathbf{s}) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$$

extracting  $\mathbf{h}, \mathbf{s}$  from  $\mathbf{H}$  in the instance  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  and forwards  $\text{pp}_{\text{uhqcPKE}}, \text{pk}_0^{(u)}$  to  $\mathcal{A}$ . The distinguisher sets  $\mathbf{a}_{1,0} = \mathbf{a}_1$  and  $\mathbf{a}_{2,0} = \mathbf{a}_2$  where  $\mathbf{a} = (\mathbf{a}_1, \mathbf{a}_2) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n$  is extracted from the given instance  $(\mathbf{H}, \mathbf{a})$ .  $\mathcal{D}$  initializes a list  $\mathbf{a\_List} = \mathbf{a\_List} \cup \{\mathbf{a}_{1,0}, \mathbf{a}_{2,0}\}$ .

– **Pre-Challenge Query Phase:** On receiving a query  $\mu_i \in \mathbb{F}_2^k$  for  $i = \{0, 1, 2, \dots, t-1\}$  from  $\mathcal{A}$  the distinguisher  $\mathcal{D}$  updates the list  $\mathbf{a\_List} = \mathbf{a\_List} \cup \{\mathbf{a}_{1,i}, \mathbf{a}_{2,i}\}$  and computes

$$\text{up}_{i+1}^{(u)} = \mathbf{CT}_{\mu_i} = (\mathbf{r}'_1 + \mathbf{h}_i \mathbf{r}'_2, \mu_i \mathbf{G} + \mathbf{s}_i \mathbf{r}'_2 + \mathbf{e}'),$$

$$\text{pk}_{i+1}^{(u)} = (\Pi_{\mu_i}(\mathbf{h}_i) = \mathbf{h}_{i+1}, F_{\mu_i}(\mathbf{s}_i) = \mathbf{s}_{i+1}) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$$

$$\mathbf{a}_{1,i+1} = \Pi_{\mu_i}(\mathbf{a}_{1,i}), \quad \mathbf{a}_{2,i+1} = F_{\mu_i}(\mathbf{a}_{2,i})$$

and forwards  $\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)}$  to  $\mathcal{A}$  where  $\mathbf{r}'_1, \mathbf{r}'_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n, \mathbf{e}' \xleftarrow{\$} \mathbb{F}_{2,w_e}^n, (\Pi_{\mu_i}, F_{\mu_i}) = \text{Samp}(\mu_i) \in \text{Per}(\mathcal{R}), \text{pk}_i^{(u)} = (\mathbf{h}_i, \mathbf{s}_i) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  and  $\mathbf{a}_{1,i}, \mathbf{a}_{2,i}$  are the updated instance components at epoch  $i$ .

– **Challenge Phase:** The adversary  $\mathcal{A}$  submits a message pair  $(\mathbf{m}_0^*, \mathbf{m}_1^*) \in \mathbb{F}_2^k \times \mathbb{F}_2^k$  to  $\mathcal{D}$  at epoch  $t$  who in return samples a bit  $b \xleftarrow{\$} \{0, 1\}$ , computes

$$\boxed{\mathbf{CT}_{\mathbf{m}_b^*} = (\mathbf{u}_{\mathbf{m}_b^*} = \mathbf{a}_{1,t}, \quad \mathbf{v}_{\mathbf{m}_b^*} = \mathbf{m}_b^* \mathbf{G} + \mathbf{a}_{2,t}) \in \mathcal{R} \times \mathcal{R}}$$

and submits it to  $\mathcal{A}$  where  $\mathbf{a}_{1,t}$ ,  $\mathbf{a}_{2,t}$  are the updated instance component at epoch  $t$  denotes the current epoch.

– **Post-Challenge Query Phase:** The adversary  $\mathcal{A}$  can make  $\mathcal{O}_{\text{upd}}^{\text{uhqcPKE}}(\mu_i)$  queries for  $i = \{t, t+1, \dots, t'-1\}$  which are simulated exactly in the same way by  $\mathcal{D}$  as described in the pre-challenge query phase for  $\mu_i \in \mathbb{F}_2^k$ .

– **Reveal Phase:** The distinguisher  $\mathcal{D}$  samples  $\mu^* \xleftarrow{\$} \mathbb{F}_2^k$  at epoch  $t' + 1$ , independent of  $\mu^*$ , computes

$$\text{sk}^* = (\rho^*, \gamma^*) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$$

$$\text{pk}^* = (\mathbf{h}^* = \Pi_{\mu^*}(\mathbf{h}_{t'}), \mathbf{s}^* = \rho^* + \mathbf{h}^* \gamma^*) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n,$$

$$\text{up}^* = \text{CT}_{\mu^*} \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_{t'}^{(u)}, \mu^*; \theta'')$$

where  $\text{CT}_{\mu^*} = (\mathbf{u}_{\mu^*} = \mathbf{r}_1'' + \mathbf{h}_{t'} \mathbf{r}_2'', \mathbf{v}_{\mu^*} = \mu^* \mathbf{G} + \mathbf{s}_{t'} \mathbf{r}_2'' + \mathbf{e}'')$  and forwards the tuple  $(\text{pk}^*, \text{sk}^*, \text{up}^*)$  to  $\mathcal{A}$ .  $(\Pi_{\mu^*}, \mathbf{F}_{\mu^*}) = \text{Samp}(\mu^*) \in \text{Per}(\mathcal{R})$ ,  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$ ,  $\theta'' \in \mathbb{F}_2^\alpha$ ,  $\mathbf{P}(\theta'') = (\mathbf{e}'', \mathbf{r}_1'', \mathbf{r}_2'') \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ .

– **Guess Phase:** The adversary  $\mathcal{A}$  submits a guess bit  $b'$  for the bit  $b$  chosen by  $\mathcal{D}$  in the challenge phase,  $\mathcal{D}$  forwards to  $\mathcal{B}$ ,  $\beta = 0$  if  $b' \neq b$  and  $\beta = 1$  if  $b' = b$ .

Observe that  $\mathcal{D}$  perfectly simulates the experiment using the given instance of 3-DQCSDP( $n, w_e, w_r, b_1, b_2$ ). The public parameters  $\text{pp}_{\text{uhqcPKE}}$ , the initial public key  $\text{pk}_0^{(u)} \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  in the setup phase, the update ciphertext  $\text{up}_{i+1}^{(u)} \in \mathcal{R} \times \mathcal{R}$ , updated public key  $\text{pk}_{i+1}^{(u)} \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  in the pre-challenge and post-challenge query phase, the challenge ciphertext  $\text{CT}_{\mathbf{m}_b^*} \in \mathcal{R} \times \mathcal{R}$  in the challenge phase and the simulation of  $(\text{pk}^*, \text{sk}^*, \text{up}^*)$  in the reveal phase are identically distributed in both  $\mathcal{G}_3$  and  $\mathcal{G}_4$ . Thereby, adversary's view in both game are same and the following two cases hold.

**Case  $\beta = 0$ :** If  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  for  $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$ ,  $\mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h} \tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s} \tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n$ ,  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$  then the simulation follows as in game  $\mathcal{G}_3$  and therefore,

$$\Pr \left[ \mathcal{D}(\mathbf{H}, \mathbf{a}) = 1 \mid \mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix} \right],$$

$$\mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h} \tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s} \tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n, \tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n, \tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n \Big] \\ = \Pr[S_4].$$

**Case  $\beta = 1$ :** If  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  for  $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$ ,  $\mathbf{a} = (\tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}})$ ,  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  then the simulation follows as in game  $\mathcal{G}_2$  and therefore

$$\begin{aligned} & \Pr \left[ \mathcal{D}(\mathbf{H}, \mathbf{a}) = 1 \mid \mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix} \right. \\ & \left. \mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n, \tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2), \tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2) \right] \\ & = \Pr[S_3]. \end{aligned}$$

Note that,

$$\begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix} (\tilde{\mathbf{r}}_1, \tilde{\mathbf{e}}, \tilde{\mathbf{r}}_2)^T = (\tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \tilde{\mathbf{e}} + \mathbf{s}\tilde{\mathbf{r}}_2)^T = (\mathbf{a}_1, \mathbf{a}_2)^T.$$

Therefore if the instance  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  is from the 3-QCSDP( $n, w_e, w_r, b_1, b_2$ ) distribution then  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$

$$\begin{aligned} (\mathbf{u}_{\mathbf{m}_b^*}, \mathbf{v}_{\mathbf{m}_b^*} - \mathbf{m}_b^* \mathbf{G}) &= (\mathbf{a}_{1,t} = \Pi_{\mu_{t-1}}(\mathbf{a}_{1,t-1}), \mathbf{a}_{2,t} = \mathbf{F}_{\mu_{t-1}}(\mathbf{a}_{2,t-1})) \\ &= (\tilde{\mathbf{r}}'_1 + \mathbf{h}_t \tilde{\mathbf{r}}'_2, \tilde{\mathbf{e}}' + \mathbf{s}_t \tilde{\mathbf{r}}'_2) \end{aligned}$$

for some  $\tilde{\mathbf{r}}'_1, \tilde{\mathbf{r}}'_2 \in \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}}' \in \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  by the Proposition 3 and Theorem 5 where  $(\Pi_{\mu_{t-1}}, \mathbf{F}_{\mu_{t-1}}) = \text{Samp}(\mu_{t-1}) \in \text{Per}(\mathcal{R})$ ,  $\mathbf{a}_{1,t-1}, \mathbf{a}_{2,t-1}$  are the updated instance components at epoch  $t-1$  formed using  $\mathbf{a}_{1,0} = \mathbf{a}_1$  and  $\mathbf{a}_{2,0} = \mathbf{a}_2$ . Then

$$\text{CT}_{\mathbf{m}_b^*} = (\mathbf{u}_{\mathbf{m}_b^*} = \tilde{\mathbf{r}}'_1 + \mathbf{h}_t \tilde{\mathbf{r}}'_2, \mathbf{v}_{\mathbf{m}_b^*} = \mathbf{m}_b^* \mathbf{G} + \mathbf{s}_t \tilde{\mathbf{r}}'_2 + \tilde{\mathbf{e}}').$$

follows the same distribution as in  $\mathcal{G}_3$ . On the other hand if the instance  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  is from the uniform distribution over  $\mathbb{F}_{2,b_1}^{2n \times 3n} \times \left( \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n \right)$  then  $\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2$ ,  $\mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}$  for some  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$  and therefore

$$\begin{aligned} (\mathbf{u}_{\mathbf{m}_b^*}, \mathbf{v}_{\mathbf{m}_b^*} - \mathbf{m}_b^* \mathbf{G}) &= (\mathbf{a}_{1,t} = \Pi_{\mu_{t-1}}(\mathbf{a}_{1,t-1}), \mathbf{a}_{2,t} = \mathbf{F}_{\mu_{t-1}}(\mathbf{a}_{2,t-1})) \\ &= (\tilde{\mathbf{r}}'_1 + \mathbf{h}_t \tilde{\mathbf{r}}'_2, \tilde{\mathbf{e}}' + \mathbf{s}_t \tilde{\mathbf{r}}'_2) \end{aligned}$$

for some  $\tilde{\mathbf{r}}'_1, \tilde{\mathbf{r}}'_2 \in \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}}' \in \mathbb{F}_{2,w_e}^n$  by Proposition 3, and Theorem 5 where  $\mathbf{a}_{1,t-1}, \mathbf{a}_{2,t-1}$  the updated instance components formed by extracting  $\mathbf{a}_1 = \mathbf{a}_{1,0}, \mathbf{a}_2 = \mathbf{a}_{2,0}$  from the instance and  $(\Pi_{\mu_{t-1}}, \mathbf{F}_{\mu_{t-1}}) = \text{Samp}(\mu_{t-1}) \in \text{Per}(\mathcal{R})$ . Then

$$\text{CT}_{\mathbf{m}_b^*} = (\mathbf{a}_{1,t}, \mathbf{m}_b^* \mathbf{G} + \mathbf{a}_{2,t}) = (\mathbf{u}_{\mathbf{m}_b^*} = \tilde{\mathbf{r}}'_1 + \mathbf{h}_t \tilde{\mathbf{r}}'_2, \mathbf{v}_{\mathbf{m}_b^*} = \mu_t \mathbf{G} + \mathbf{s}_t \tilde{\mathbf{r}}'_2 + \tilde{\mathbf{e}}')$$

follows the same distribution as in  $\mathcal{G}_4$ . Thus the probability of distinguishing game  $\mathcal{G}_3$  and game  $\mathcal{G}_4$  is same as the advantage of breaking the 3-DQCSDP( $n, w_e, w_r, b_1, b_2$ ) game. Hence,  $|\Pr[S_3] - \Pr[S_4]| = \text{Adv}_{\mathcal{D}}^{3\text{-DQCSDP}}(1^\lambda) <$

$\varepsilon_3$  for some negligible function  $\varepsilon_3$  as 3-DQCDSP problem is hard.

• *Proof of Claim 4:* We show that if there exists an adversary  $\mathcal{A}$  capable of distinguishing the games  $\mathcal{G}_4$  and  $\mathcal{G}_5$  then a distinguisher  $\mathcal{D}$  can be constructed for the 3-DQCDSP( $n, w_e, w_r, b_1, b_2$ ) problem. Let  $\mathcal{B}$  be the 3-DQCDSP( $n, w_e, w_r, b_1, b_2$ ) (see Definition 15) challenger who provides an 3-DQCDSP( $n, w_e, w_r, b_1, b_2$ ) problem instance  $\text{Ins} = (\mathbf{H}, \mathbf{a} = (\mathbf{a}_1, \mathbf{a}_2)) \in \mathbb{F}_{2, b_1, b_2}^{2n \times 3n} \times (\mathbb{F}_{2, b_3}^n \times \mathbb{F}_{2, b'_3}^n)$  to the distinguisher  $\mathcal{D}$  where

$$\begin{aligned} \mathbf{H} &= \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, \quad \mathbf{h} \in \mathbb{F}_{2, b_1}^n, \quad \mathbf{s} \in \mathbb{F}_{2, b_2}^n, \quad b_1 = \mathbf{h} \cdot \mathbf{1} \bmod 2, \\ b_2 &= w + b_1 w \bmod 2, \quad b_3 = w_r + b_1 w_r \bmod 2, \quad b'_3 = w_e + b_2 w_r \bmod 2, \\ \mathbf{a} &= (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2, b_3}^n \times \mathbb{F}_{2, b'_3}^n, \end{aligned}$$

and  $(\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2, \tilde{\mathbf{e}})$  is either a random tuple from  $\mathbb{F}_{2, w_r}^n \times \mathbb{F}_{2, w_r}^n \times \mathbb{F}_{2, w_e}^n$  or from  $\mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_e}^n(\mathbb{F}_2)$ . The distinguisher  $\mathcal{D}$  acts as a challenger to  $\mathcal{A}$  and simulates its views as follows.

– **Setup:** The distinguisher  $\mathcal{D}$  chooses a  $\tau$  error correcting decodable  $[n, k]$  linear code  $\mathbf{C}$  with a generator matrix is  $\mathbf{G}$ . It also selects an encryption randomness size  $\alpha$ , fixed weights  $w, w_e, w_r$ , a randomness generator  $\mathbf{P} : \{0, 1\}^* \rightarrow \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$  along with a sampling function  $\text{Samp} : \mathbb{F}_2^k \rightarrow \text{Per}(\mathcal{R})$  and sets

$$\text{pp}_{\text{uhqcPKE}} = (n, k, \alpha, \tau, w, w_e, w_r, \text{Samp}, \mathbf{G}, \mathbf{P}).$$

It extracts  $\mathbf{h}, \mathbf{s}$  from instance  $(\mathbf{H}, \mathbf{a})$  and sets

$$\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) = (\mathbf{h}, \mathbf{s}) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n$$

and forwards  $\text{pp}_{\text{uhqcPKE}}, \text{pk}_0^{(u)}$  to its adversary  $\mathcal{A}$ . The distinguisher sets  $\mathbf{a}_1 = \mathbf{a}_{1,0}$  and  $\mathbf{a}_2 = \mathbf{a}_{2,0}$  where  $\mathbf{a} = (\mathbf{a}_1, \mathbf{a}_2) \in \mathbb{F}_{2, b_3}^n \times \mathbb{F}_{2, b'_3}^n$  extracted from the instance  $(\mathbf{H}, \mathbf{a})$ .  $\mathcal{D}$  initializes a list  $\mathbf{a\_List} = \mathbf{a\_List} \cup \{\mathbf{a}_{1,0}, \mathbf{a}_{2,0}\}$ .

– **Pre-Challenge Query Phase:** On receiving a query  $\mu_i \in \mathbb{F}_2^k$  for  $i = \{0, 1, 2, \dots, t-1\}$  from  $\mathcal{A}$  the distinguisher  $\mathcal{D}$  updates the list  $\mathbf{a\_List} = \mathbf{a\_List} \cup \{\mathbf{a}_{1,i}, \mathbf{a}_{2,i}\}$  and computes

$$\text{up}_{i+1}^{(u)} = (\mathbf{r}'_1 + \mathbf{h}_i \mathbf{r}'_2, \mu_i \mathbf{G} + \mathbf{s}_i \mathbf{r}'_2 + \mathbf{e}'),$$

$$\text{pk}_{i+1}^{(u)} = (\Pi_{\mu_i}(\mathbf{h}_i), \mathbf{F}_{\mu_i}(\mathbf{s}_i)) = (\mathbf{h}_{i+1}, \mathbf{s}_{i+1}) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n$$

$$\mathbf{a}_{1,i+1} = \Pi_{\mu_i}(\mathbf{a}_{1,i}), \quad \mathbf{a}_{2,i+1} = \mathbf{F}_{\mu_i}(\mathbf{a}_{2,i})$$

and forwards  $\text{up}_{i+1}^{(u)}, \text{pk}_{i+1}^{(u)}$  to  $\mathcal{A}$  where  $\mathbf{r}'_1, \mathbf{r}'_2 \xleftarrow{\$} \mathbb{F}_{2, w_r}^n, \mathbf{e}' \xleftarrow{\$} \mathbb{F}_{2, w_e}^n, (\Pi_{\mu_i}, \mathbf{F}_{\mu_i}) = \text{Samp}(\mu_i) \in \text{Per}(\mathcal{R}), \text{pk}_i^{(u)} = (\mathbf{h}_i, \mathbf{s}_i) \in \mathbb{F}_{2, b_1}^n \times \mathbb{F}_{2, b_2}^n, \mathbf{a}_{1,i}, \mathbf{a}_{2,i}$  are the updated

instance component at epoch  $i$ .

– **Challenge Phase:** On receiving challenge message pair  $(\mathbf{m}_0^*, \mathbf{m}_1^*) \in \mathbb{F}_2^k \times \mathbb{F}_2^k$  from  $\mathcal{A}$ , its distinguisher  $\mathcal{D}$  samples a bit  $b \xleftarrow{\$} \{0, 1\}$ , computes

$$\text{CT}_{\mathbf{m}_b^*} = (\mathbf{r}_1 + \mathbf{h}_t \mathbf{r}_2, \mathbf{m}_b^* \mathbf{G} + \mathbf{s}_t \mathbf{r}_2 + \mathbf{e}) \leftarrow \text{uhqcPKE}.\text{Enc}(\text{pp}_{\text{uhqcPKE}}, \text{pk}_t^{(u)}, \mathbf{m}_b^*; \theta),$$

and forwards it to  $\mathcal{A}$  where  $\theta \in \mathbb{F}_2^\alpha$  is a random  $\alpha$ -bit string,  $\text{P}(\theta) = (\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$  and  $\text{pk}_t^{(u)} = (\mathbf{h}_t, \mathbf{s}_t) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  is the public key at epoch  $t$ .

– **Post-Challenge Query Phase:** The adversary  $\mathcal{A}$  can make  $\mathcal{O}_{\text{upd}}^{\text{uhqcPKE}}(\mu_i)$  queries for  $i = \{t, t+1, \dots, t'-1\}$  which are simulated in the same manner by  $\mathcal{D}$  as described in the pre-challenge query phase for  $\mu_i \in \mathbb{F}_2^k$ .

– **Reveal Phase:** The distinguisher  $\mathcal{D}$  samples  $\mu^* \xleftarrow{\$} \mathbb{F}_2^k$ ,  $(\rho^*, \gamma^*) \xleftarrow{\$} \mathcal{S}_w^n(\mathbb{F}_2) \times \mathcal{S}_w^n(\mathbb{F}_2)$  at epoch  $t' + 1$  and sets

$$\text{sk}^* = (\rho^*, \gamma^*)$$

$$\text{pk}^* = (\mathbf{h}^* = \Pi_{\mu^*}(\mathbf{h}_{t'}), \mathbf{s}^* = \rho^* + \mathbf{h}^* \gamma^*) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$$

$$\boxed{\text{up}^* = (\mathbf{u}_{\mu^*} = \mathbf{a}_{1,t'}, \mathbf{v}_{\mu^*} = \mu^* \mathbf{G} + \mathbf{a}_{2,t'}) \in \mathcal{R} \times \mathcal{R},}$$

and forwards the tuple  $(\text{pk}^*, \text{sk}^*, \text{up}^*)$  to the adversary  $\mathcal{A}$ .  $(\Pi_{\mu^*}, \text{F}_{\mu^*}) = \text{Samp}(\mu^*) \in \text{Per}(\mathcal{R})$ ,  $\text{pk}_{t'}^{(u)} = (\mathbf{h}_{t'}, \mathbf{s}_{t'}) \in \mathcal{R} \times \mathcal{R}$ ,  $\theta'' \xleftarrow{\$} \mathbb{F}_2^\alpha$ ,  $\text{P}(\theta'') = (\mathbf{e}'', \mathbf{r}_1'', \mathbf{r}_2'') \in \mathcal{S}_{w_e}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2) \times \mathcal{S}_{w_r}^n(\mathbb{F}_2)$  where  $\mathbf{a}_{1,t}$ ,  $\mathbf{a}_{2,t}$  are the updated instance component at epoch  $t$  denotes the current epoch.

– **Guess Phase:** The adversary  $\mathcal{A}$  submits a guess bit  $b'$  for the bit  $b$  chosen by  $\mathcal{D}$  in the challenge phase,  $\mathcal{D}$  forwards to  $\mathcal{B}$ ,  $\beta = 0$  if  $b' \neq b$  and  $\beta = 1$  if  $b' = b$ .

Note that  $\mathcal{D}$  faithfully simulates the experiment using the given  $3\text{-DQCSDP}(n, w_e, w_r, b_1, b_2)$  instance. In the setup phase, the generated public parameters  $\text{pp}_{\text{uhqcPKE}}$  and the initial public key  $\text{pk}_0^{(u)} = (\mathbf{h}_0, \mathbf{s}_0) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  each have identical distribution in game  $\mathcal{G}_4$  and game  $\mathcal{G}_5$ . During the pre-challenge query phase and post-challenge query phase, the update ciphertext  $\text{up}_{i+1}^{(u)} \in \mathcal{R} \times \mathcal{R}$  and updated public key  $\text{pk}_{i+1}^{(u)} = (\mathbf{h}_{i+1}, \mathbf{s}_{i+1}) \in \mathbb{F}_{2,b_1}^n \times \mathbb{F}_{2,b_2}^n$  are simulated following to the same distribution in both the games. Likewise, the challenge ciphertext  $\text{CT}_{\mathbf{m}_b^*} \in \mathcal{R} \times \mathcal{R}$  in the challenge phase as well as the simulations of  $(\text{pk}^*, \text{sk}^*, \text{up}^*)$  reveal phases are identically distributed in both game  $\mathcal{G}_4$  and game  $\mathcal{G}_5$ . Therefore, the adversary's view the distributions are same in both games which yields the following two cases.

**Case  $\beta = 0$ :** If  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  for  $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$ ,  $\mathbf{a} = (\mathbf{a}_1 =$

$\tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n$ ,  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$  then the simulation follows as in game  $\mathcal{G}_5$  and therefore,

$$\Pr \left[ \mathcal{D}(\mathbf{H}, \mathbf{a}) = 1 \mid \mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, \right. \\ \left. \mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n, \tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n, \tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n \right] \\ = \Pr[S_5].$$

**Case  $\beta = 1$ :** If  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  for  $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$ ,  $\mathbf{a} = (\tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}})$ ,  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  then the simulation follows as in game  $\mathcal{G}_4$  and therefore

$$\Pr \left[ \mathcal{D}(\mathbf{H}, \mathbf{a}) = 1 \mid \mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, \right. \\ \left. \mathbf{a} = (\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}) \in \mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n, \tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2), \right. \\ \left. \tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2) \right] = \Pr[S_4].$$

Observe that,

$$\begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix} (\tilde{\mathbf{r}}_1, \tilde{\mathbf{e}}, \tilde{\mathbf{r}}_2)^T = (\tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2, \tilde{\mathbf{e}} + \mathbf{s}\tilde{\mathbf{r}}_2)^T = (\mathbf{a}_1, \mathbf{a}_2)^T.$$

Therefore if the instance  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  is from the 3-QCSDP( $n, w_e, w_r, b_1, b_2$ ) distribution then  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathcal{S}_{w_e}^n(\mathbb{F}_2)$

$$(\mathbf{u}_{\mu^*}, \mathbf{v}_{\mu^*} - \mu^* \mathbf{G}) = (\mathbf{a}_{1,t'} = \Pi_{\mu_{t'-1}}(\mathbf{a}_{1,t'-1}), \mathbf{a}_{2,t'} = \mathbf{F}_{\mu_{t'-1}}(\mathbf{a}_{2,t'-1})) \\ = (\tilde{\mathbf{r}}'_1 + \mathbf{h}_{t'}\tilde{\mathbf{r}}'_2, \tilde{\mathbf{e}}' + \mathbf{s}_{t'}\tilde{\mathbf{r}}'_2)$$

for some  $\tilde{\mathbf{r}}'_1, \tilde{\mathbf{r}}'_2 \in \mathcal{S}_{w_r}^n(\mathbb{F}_2)$ ,  $\tilde{\mathbf{e}}' \in \mathcal{S}_{w_e}^n(\mathbb{F}_2)$  using the Proposition 3 and Theorem 5 where  $(\Pi_{\mu^*}, \mathbf{F}_{\mu^*}) = \text{Samp}(\mu^*) \in \text{Per}(\mathcal{R})$ ,  $\mathbf{a}_{1,t'-1}, \mathbf{a}_{2,t'-1}$  are the updated instance components at epoch  $t' - 1$  formed using  $\mathbf{a}_{1,0} = \mathbf{a}_1$  and  $\mathbf{a}_{2,0} = \mathbf{a}_2$ . Therefore, the resulting ciphertext

$$\text{CT}_{\mu^*} = (\mathbf{u}_{\mu^*} = \tilde{\mathbf{r}}'_1 + \mathbf{h}_{t'}\tilde{\mathbf{r}}'_2, \mathbf{v}_{\mu^*} = \mu^* \mathbf{G} + \mathbf{s}_{t'}\tilde{\mathbf{r}}'_2 + \tilde{\mathbf{e}}').$$

follows the same distribution as in  $\mathcal{G}_4$ . On the other hand suppose that the instance  $\text{Ins} = (\mathbf{H}, \mathbf{a})$  is from the uniform distribution over  $\mathbb{F}_{2,b_1 \times 3n}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_{2,b'_3}^n)$  then  $\mathbf{a}_1 = \tilde{\mathbf{r}}_1 + \mathbf{h}\tilde{\mathbf{r}}_2$ ,  $\mathbf{a}_2 = \mathbf{s}\tilde{\mathbf{r}}_2 + \tilde{\mathbf{e}}$  for some  $\tilde{\mathbf{r}}_1, \tilde{\mathbf{r}}_2 \xleftarrow{\$} \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}} \xleftarrow{\$} \mathbb{F}_{2,w_e}^n$  and therefore

$$(\mathbf{u}_{\mu^*}, \mathbf{v}_{\mu^*} - \mu^* \mathbf{G}) = (\mathbf{a}_{1,t'} = \Pi_{\mu_{t'-1}}(\mathbf{a}_{1,t'-1}), \mathbf{a}_{2,t'} = \mathbf{F}_{\mu_{t'-1}}(\mathbf{a}_{2,t'-1})) \\ = (\tilde{\mathbf{r}}'_1 + \mathbf{h}_{t'}\tilde{\mathbf{r}}'_2, \tilde{\mathbf{e}}' + \mathbf{s}_{t'}\tilde{\mathbf{r}}'_2)$$

for some  $\tilde{\mathbf{r}}'_1, \tilde{\mathbf{r}}'_2 \in \mathbb{F}_{2,w_r}^n$ ,  $\tilde{\mathbf{e}}' \in \mathbb{F}_{2,w_e}^n$  using Proposition 3 and Theorem 5 where  $\mathbf{a}_{1,t'-1}, \mathbf{a}_{2,t'-1}$  the updated instance components formed by extracting  $\mathbf{a}_1 = \mathbf{a}_{1,0}, \mathbf{a}_2 = \mathbf{a}_{2,0}$  from the instance and  $(\Pi_{\mu_{t'-1}}, F_{\mu_{t'-1}}) = \text{Samp}(\mu_{t'-1}) \in \text{Per}(\mathcal{R})$ . Consequently, the ciphertext

$$\text{CT}_{\mu^*} = (\mathbf{a}_{1,t'}, \mu^* \mathbf{G} + \mathbf{a}_{2,t'}) = (\mathbf{u}_{\mu^*} = \tilde{\mathbf{r}}'_1 + \mathbf{h}_{t'} \tilde{\mathbf{r}}'_2, \mathbf{v}_{\mu^*} = \mu^* \mathbf{G} + \mathbf{s}_{t'} \tilde{\mathbf{r}}'_2 + \tilde{\mathbf{e}}')$$

follows the same distribution as in  $\mathcal{G}_5$ . Thus the probability of distinguishing game  $\mathcal{G}_4$  and game  $\mathcal{G}_5$  is same as the advantage of breaking the 3-DQCSDP( $n, w_e, w_r, b_1, b_2$ ) game.

Hence,  $|\Pr[S_4] - \Pr[S_5]| = \text{Adv}_{\mathcal{D}}^{3\text{-DQCSDP}}(1^\lambda) < \varepsilon_3$  for some negligible function  $\varepsilon_3$  considering the hardness of the 3-DQCSDP problem.

## 5 Algebraic Instantiations and Entropy Discussions of $\text{Per}(\mathcal{R})$

In Section 3.1, we mentioned that any concrete instantiation of the sampling algorithm  $\text{Samp}$  must satisfy some structural properties (homomorphism and weight preservation) as well as a distributional equivalence (high entropy). In this section, we present a construction of  $\text{Per}(\mathcal{R})$  using cyclic shifts and ring automorphisms. We furnish that the proposed construction satisfies the required algebraic properties and then discuss its entropy.

We first define two fundamental maps over the ring  $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$ .

**The Shift Map  $\Sigma^{(\delta_j)}$ :** Let  $\Sigma^{(\delta_j)} : \mathcal{R} \rightarrow \mathcal{R}$  be defined by

$$\Sigma^{(\delta_j)}(\gamma(x)) = \gamma(x)x^{\delta_j} \pmod{x^n - 1}$$

for some integer shift  $\delta_j \in M = \{0, 1, \dots, n-1\}$  and any  $\gamma(x) \in \mathcal{R}$ .

Fig. 5: Definition of the cyclic shift map  $\Sigma^{(\delta_j)}$

**Proposition 2.** Let  $\Sigma^{(\delta_j)} : \mathcal{R} \rightarrow \mathcal{R}$  be the map presented in Figure 5. Then, for every  $\mathbf{h}(x), \mathbf{y}(x) \in \mathcal{R}$ :

1. **Additivity:**  $\Sigma^{(\delta_j)}(\mathbf{h} + \mathbf{y}) = \Sigma^{(\delta_j)}(\mathbf{h}) + \Sigma^{(\delta_j)}(\mathbf{y})$ .
2. **Vector Multiplication:**  $\Sigma^{(\delta_j)}(\mathbf{h}\mathbf{y}) = \mathbf{h}\Sigma^{(\delta_j)}(\mathbf{y})$ .
3. **Weight Preservation:**  $\text{wt}(\mathbf{h}) = \text{wt}(\Sigma^{(\delta_j)}(\mathbf{h}))$ .

*Proof.* By the definition given in Figure 5,  $\Sigma^{(\delta_j)}(\mathbf{h}(x) + \mathbf{y}(x)) = (\mathbf{h}(x) + \mathbf{y}(x))x^{\delta_j} \pmod{x^n - 1}$ . Since multiplication distributes over addition in  $\mathcal{R}$ , this evaluates to  $\mathbf{h}(x)x^{\delta_j} + \mathbf{y}(x)x^{\delta_j} = \Sigma^{(\delta_j)}(\mathbf{h}(x)) + \Sigma^{(\delta_j)}(\mathbf{y}(x))$ , proving additivity.

Similarly, due to the associativity of multiplication in  $\mathcal{R}$ ,  $\Sigma^{(\delta_j)}(\mathbf{h}(x)\mathbf{y}(x)) = \mathbf{h}(x)(\mathbf{y}(x)x^{\delta_j}) \pmod{x^n - 1} = \mathbf{h}(x)\Sigma^{(\delta_j)}(\mathbf{y}(x))$ , proving the vector multiplication property.

Finally, let  $\mathbf{h}(x) = \sum_{i=0}^{n-1} h_i x^i$ . The operation  $\mathbf{h}(x)x^{\delta_j} \pmod{x^n - 1}$  corresponds to taking the exponents modulo  $n$ . Thus,  $\Sigma^{(\delta_j)}(\mathbf{h})$  is simply a cyclic shift of the coefficients of  $\mathbf{h}$ . Since a cyclic shift preserves the number of nonzero coefficients,  $\text{wt}(\Sigma^{(\delta_j)}(\mathbf{h})) = \text{wt}(\mathbf{h})$ .

**The Automorphism Map  $\Pi^{(p_j)}$ :** Let  $\Pi^{(p_j)} : \mathcal{R} \rightarrow \mathcal{R}$  be defined by

$$\Pi^{(p_j)}(\gamma(x)) = \gamma(x^{p_j}) \pmod{x^n - 1}$$

for an integer  $p_j \in N$ , where  $N = \{p_j \mid p_j \leq n \text{ and } \gcd(p_j, n) = 1\}$ .

Fig. 6: Definition of the ring automorphism  $\Pi^{(p_j)}$

**Proposition 3.** *The map  $\Pi^{(p_j)}$  presented in Figure 6 is a weight-preserving ring automorphism on  $\mathcal{R}$ . Specifically, for all  $\mathbf{h}, \mathbf{y} \in \mathcal{R}$ :*

1.  $\Pi^{(p_j)}(\mathbf{h} + \mathbf{y}) = \Pi^{(p_j)}(\mathbf{h}) + \Pi^{(p_j)}(\mathbf{y})$ .
2.  $\Pi^{(p_j)}(\mathbf{h}\mathbf{y}) = \Pi^{(p_j)}(\mathbf{h})\Pi^{(p_j)}(\mathbf{y})$ .
3.  $\text{wt}(\Pi^{(p_j)}(\mathbf{h})) = \text{wt}(\mathbf{h})$ .

*Proof.* We evaluate

$$\Pi^{(p_j)}(\mathbf{h} + \mathbf{y})(x) = (\mathbf{h} + \mathbf{y})(x^{p_j}) = \mathbf{h}(x^{p_j}) + \mathbf{y}(x^{p_j}) = \Pi^{(p_j)}(\mathbf{h}(x)) + \Pi^{(p_j)}(\mathbf{y}(x))$$

proving additivity. Similarly,

$$\Pi^{(p_j)}(\mathbf{h}\mathbf{y})(x) = (\mathbf{h}\mathbf{y})(x^{p_j}) = \mathbf{h}(x^{p_j})\mathbf{y}(x^{p_j}) = \Pi^{(p_j)}(\mathbf{h}(x))\Pi^{(p_j)}(\mathbf{y}(x))$$

proving multiplicativity. Because  $\gcd(n, p_j) = 1$ , the map  $x \mapsto x^{p_j}$  induces a bijection of the exponents modulo  $n$ , making  $\Pi^{(p_j)}$  a ring automorphism. Because this bijection simply permutes the coefficients without changing their values, the number of non-zero coefficients is invariant, yielding  $\text{wt}(\Pi^{(p_j)}(\mathbf{h})) = \text{wt}(\mathbf{h})$ .

Using the primitive maps  $\Sigma$  and  $\Pi$ , we construct a candidate set  $\text{Per}(\mathcal{R})$  through function composition.

**Theorem 5.** *For any  $(\Pi, F) \in \text{Per}(\mathcal{R})$  defined in Figure 7 and any  $\mathbf{x}, \mathbf{h}, \mathbf{y} \in \mathcal{R}$ , it holds that:*

$$F(\mathbf{x} + \mathbf{h}\mathbf{y}) = F(\mathbf{x}) + \Pi(\mathbf{h})F(\mathbf{y}).$$

*Furthermore, both  $\Pi$  and  $F$  strictly preserve Hamming weight.*

Let  $\text{Per}(\mathcal{R}) = \{(\Pi, F)\}$  where:

$$\Pi = \Pi^{(p_{k'})} \circ \Pi^{(p_{k'-1})} \circ \dots \circ \Pi^{(p_1)}$$

$$F = (\Pi^{(p_{k'})} \circ \Sigma^{(\delta_{k'})}) \circ (\Pi^{(p_{k'-1})} \circ \Sigma^{(\delta_{k'-1})}) \circ \dots \circ (\Pi^{(p_1)} \circ \Sigma^{(\delta_1)})$$

for uniformly sampled  $\delta_j \in M$  and  $p_j \in N$ .

Fig. 7:  $\text{Per}(\mathcal{R})$

*Proof.* Using Proposition 2 and Proposition 3, the additivity of both primitive functions yields  $F(\mathbf{x} + \mathbf{h}\mathbf{y}) = F(\mathbf{x}) + F(\mathbf{h}\mathbf{y})$ . We now evaluate  $F(\mathbf{h}\mathbf{y})$ .

By repeated application of the scalar property from Proposition 2 and the automorphism property from Proposition 3, the factor  $\mathbf{h}$  propagates through the composition. Each  $\Sigma$  operation leaves  $\mathbf{h}$  intact, while each  $\Pi$  operation transforms it. Thus:

$$F(\mathbf{h}\mathbf{y}) = (\Pi^{(p_{k'})} \circ \dots \circ \Pi^{(p_1)})(\mathbf{h}) \cdot F(\mathbf{y}) = \Pi(\mathbf{h})F(\mathbf{y}).$$

Finally, since every individual  $\Sigma$  and  $\Pi$  map preserves Hamming weight, any composition of these maps, including  $\Pi$  and  $F$ , must also preserve Hamming weight.

*Remark 1.* (Discussion on Entropy). While the construction in Figure 7 satisfies the required algebraic and weight-preserving properties, it can not achieve distributional equivalence due to low entropy. The operations  $\Sigma$  and  $\Pi$  generate a space of size at most  $|\text{Per}(\mathcal{R})| \leq n\phi(n)$ , since there are only  $n$  cyclic shifts and  $\phi(n)$  automorphisms, and composition does not expand this set. For cryptographic parameters ( $n \approx 10^4 - 10^5$ ), this is negligible compared to the  $\binom{n}{w}$  possibilities in  $\mathcal{S}_w^n(\mathbb{F}_2)$ . We leave the construction of  $\text{Per}(\mathcal{R})$  satisfying the above conditions along with sufficiently high entropy for forward security as an open question.

## 6 Conclusion

This article presents a design for an IND-CR-CPA secure updatable public key encryption scheme in code-based setting. In terms of storage, our work performs well compared to some other existing protocols. Besides, our protocol accommodates both asynchronous and unbounded key updates. However, achieving stronger security requires allowing the adversary access to a decryption oracle associated with the secret key of the current epoch. The formal definition of this enhanced security guarantee is captured by the indistinguishability under chosen-randomness and chosen-ciphertext attack (IND-CR-CCA) security notion. As future work, we plan to explore the updatable key encapsulation mechanism that attains IND-CR-CCA security.

## References

1. Abou Haidar, C., Libert, B., Passelègue, A.: Updatable public key encryption from dcr: Efficient constructions with stronger security. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. p. 11–22. CCS '22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3548606.3559376>, <https://doi.org/10.1145/3548606.3559376>
2. Albrecht, M.R., Benčina, B., Lai, R.W.F.: Hollow LWE: A new spin - unbounded updatable encryption from LWE and PCE. In: Fehr, S., Fouque, P. (eds.) Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques Madrid, Spain, May 4-8, 2025, Proceedings, Part VIII. Lecture Notes in Computer Science, vol. 15608, pp. 363–392. Springer (2025). [https://doi.org/10.1007/978-3-031-91101-9\\_13](https://doi.org/10.1007/978-3-031-91101-9_13), [https://doi.org/10.1007/978-3-031-91101-9\\_13](https://doi.org/10.1007/978-3-031-91101-9_13)
3. Alwen, J., Coretti, S., Dodis, Y., Tselekounis, Y.: Security analysis and improvements for the IETF MLS standard for group messaging. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part I. Lecture Notes in Computer Science, vol. 12170, pp. 248–277. Springer (2020). [https://doi.org/10.1007/978-3-030-56784-2\\_9](https://doi.org/10.1007/978-3-030-56784-2_9), [https://doi.org/10.1007/978-3-030-56784-2\\_9](https://doi.org/10.1007/978-3-030-56784-2_9)
4. Alwen, J., Fuchsbauer, G., Mularczyk, M.: Updatable public-key encryption, revisited. In: Joye, M., Leander, G. (eds.) Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI. Lecture Notes in Computer Science, vol. 14656, pp. 346–376. Springer (2024). [https://doi.org/10.1007/978-3-031-58754-2\\_13](https://doi.org/10.1007/978-3-031-58754-2_13), [https://doi.org/10.1007/978-3-031-58754-2\\_13](https://doi.org/10.1007/978-3-031-58754-2_13)
5. Asano, K., Watanabe, Y.: Updatable public key encryption with strong CCA security: Security analysis and efficient generic construction. In: Patra, A. (ed.) Topics in Cryptology - CT-RSA 2025 - Cryptographers' Track at the RSA Conference 2025, San Francisco, CA, USA, April 28-May 1, 2025, Proceedings. Lecture Notes in Computer Science, vol. 15598, pp. 223–246. Springer (2025). [https://doi.org/10.1007/978-3-031-88661-4\\_10](https://doi.org/10.1007/978-3-031-88661-4_10), [https://doi.org/10.1007/978-3-031-88661-4\\_10](https://doi.org/10.1007/978-3-031-88661-4_10)
6. Berlekamp, E., McEliece, R., van Tilborg, H.: On the inherent intractability of certain coding problems (corresp.). IEEE Transactions on Information Theory **24**(3), 384–386 (1978). <https://doi.org/10.1109/TIT.1978.1055873>
7. Berlekamp, E.R., McEliece, R.J., van Tilborg, H.C.A.: On the inherent intractability of certain coding problems. IEEE Transactions on Information Theory **24**(3), 384–386 (1978)
8. Canetti, R., Halevi, S., Katz, J.: Chosen-ciphertext security from identity-based encryption. In: Cachin, C., Camenisch, J. (eds.) Advances in Cryptology – EUROCRYPT 2004. Lecture Notes in Computer Science, vol. 3027, pp. 207–222. Springer, Heidelberg, Germany (2004)
9. Castryck, W., Decru, T.: An efficient key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V. Lecture Notes in Computer

- Science, vol. 14008, pp. 423–447. Springer (2023). [https://doi.org/10.1007/978-3-031-30589-4\\_15](https://doi.org/10.1007/978-3-031-30589-4_15), [https://doi.org/10.1007/978-3-031-30589-4\\_15](https://doi.org/10.1007/978-3-031-30589-4_15)
10. Diffie, W., Hellman, M.E.: New directions in cryptography. *IEEE Transactions on Information Theory* **22**(6), 644–654 (1976)
  11. Dodis, Y., Karthikeyan, H., Wichs, D.: Updatable public key encryption in the standard model. In: *Theory of Cryptography Conference*. pp. 254–285. Springer (2021)
  12. Duparc, M., Fouotsa, T.B., Vaudenay, S.: SILBE: an updatable public key encryption scheme from lollipop attacks. In: Eichlseder, M., Gambs, S. (eds.) *Selected Areas in Cryptography - SAC 2024 - 31st International Conference, Montreal, QC, Canada, August 28-30, 2024, Revised Selected Papers, Part I. Lecture Notes in Computer Science*, vol. 15516, pp. 151–177. Springer (2024). [https://doi.org/10.1007/978-3-031-82852-2\\_7](https://doi.org/10.1007/978-3-031-82852-2_7), [https://doi.org/10.1007/978-3-031-82852-2\\_7](https://doi.org/10.1007/978-3-031-82852-2_7)
  13. Eaton, E., Jao, D., Komlo, C., Mokrani, Y.: Towards post-quantum key-updatable public-key encryption via supersingular isogenies. In: AlTawy, R., Hülsing, A. (eds.) *Selected Areas in Cryptography - 28th International Conference, SAC 2021, Virtual Event, September 29 - October 1, 2021, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 13203, pp. 461–482. Springer (2021). [https://doi.org/10.1007/978-3-030-99277-4\\_22](https://doi.org/10.1007/978-3-030-99277-4_22), [https://doi.org/10.1007/978-3-030-99277-4\\_22](https://doi.org/10.1007/978-3-030-99277-4_22)
  14. Finiasz, M., Sendrier, N.: Distinguishing mceliece public keys. In: *Advances in Cryptology - ASIACRYPT 2009*. pp. 25–39. Springer (2009)
  15. Gaborit, P., Aguilar-Melchor, C., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Persichetti, E., Zémor, G., Bos, J., et al.: Hamming quasi-cyclic (hqc). NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA (2025)
  16. Gentry, C., Silverberg, A.: Hierarchical id-based cryptography. In: Zheng, Y. (ed.) *Advances in Cryptology – ASIACRYPT 2002. Lecture Notes in Computer Science*, vol. 2501, pp. 548–566. Springer, Heidelberg, Germany (2002)
  17. Goldwasser, S., Micali, S.: Probabilistic encryption. *Journal of Computer and System Sciences* **28**(2), 270–299 (1984). [https://doi.org/https://doi.org/10.1016/0022-0000\(84\)90070-9](https://doi.org/https://doi.org/10.1016/0022-0000(84)90070-9), <https://www.sciencedirect.com/science/article/pii/0022000084900709>
  18. Gulliver, T.A., Bhargava, V.K.: Some best rate  $1/p$  and rate  $(p-1)/p$  systematic quasi-cyclic codes. *IEEE Transactions on Information Theory* **37**(3), 552–555 (1991). <https://doi.org/10.1109/18.79914>
  19. Haidar, C.A., Passelègue, A., Stehlé, D.: Efficient updatable public-key encryption from lattices. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part V. Lecture Notes in Computer Science*, vol. 14442, pp. 342–373. Springer (2023). [https://doi.org/10.1007/978-981-99-8733-7\\_11](https://doi.org/10.1007/978-981-99-8733-7_11), [https://doi.org/10.1007/978-981-99-8733-7\\_11](https://doi.org/10.1007/978-981-99-8733-7_11)
  20. Horwitz, J., Lynn, B.: Toward hierarchical identity-based encryption. In: Knudsen, L.R. (ed.) *Advances in Cryptology – EUROCRYPT 2002. Lecture Notes in Computer Science*, vol. 2332, pp. 466–481. Springer, Heidelberg, Germany (2002)
  21. Jost, D., Maurer, U., Mularczyk, M.: Efficient ratcheting: Almost-optimal guarantees for secure messaging. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 11476,

- pp. 159–188. Springer (2019). [https://doi.org/10.1007/978-3-030-17653-2\\_6](https://doi.org/10.1007/978-3-030-17653-2_6), [https://doi.org/10.1007/978-3-030-17653-2\\_6](https://doi.org/10.1007/978-3-030-17653-2_6)
22. MacWilliams, F.J., Sloane, N.J.A.: The Theory of Error-Correcting Codes, North-Holland Mathematical Library, vol. 16. North-Holland, Amsterdam (1977)
  23. Maino, L., Martindale, C., Panny, L., Pope, G., Wesolowski, B.: A direct key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V. Lecture Notes in Computer Science, vol. 14008, pp. 448–471. Springer (2023). [https://doi.org/10.1007/978-3-031-30589-4\\_16](https://doi.org/10.1007/978-3-031-30589-4_16), [https://doi.org/10.1007/978-3-031-30589-4\\_16](https://doi.org/10.1007/978-3-031-30589-4_16)
  24. Melchor, C.A., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Gaborit, P., Persichetti, E., Zémor, G., Bourges, I.: Hamming quasi-cyclic (hqc). NIST PQC Round 2(4), 13 (2018)
  25. Misoczki, R., Tillich, J.P., Sendrier, N., Barreto, P.: Mdpcc-mceliece: New mceliece variants from moderate density parity-check codes. pp. 2069–2073 (07 2013). <https://doi.org/10.1109/ISIT.2013.6620590>
  26. Robert, D.: Breaking SIDH in polynomial time. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V. Lecture Notes in Computer Science, vol. 14008, pp. 472–503. Springer (2023). [https://doi.org/10.1007/978-3-031-30589-4\\_17](https://doi.org/10.1007/978-3-031-30589-4_17), [https://doi.org/10.1007/978-3-031-30589-4\\_17](https://doi.org/10.1007/978-3-031-30589-4_17)
  27. Shor, P.W.: Algorithms for quantum computation: Discrete logarithms and factoring. In: 35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, November 20-22, 1994. pp. 124–134. IEEE Computer Society (1994). <https://doi.org/10.1109/SFCS.1994.365700>, <https://doi.org/10.1109/SFCS.1994.365700>
  28. Statista: Quantum computing growth (2020), <https://www.statista.com/chart/17896/quantum-computing-developments/>, accessed: Apr. 4, 2020